



task: when given a question, perform these tasks: 1. define the topic used; 2. define the steps to solve particularly that kind of question; 3. demonstrate the process with the help of solving that question. 4. what other variation of that question can be made (variation doesn't mean only changing the value of some variables in that question, but particularly those questions in which the process differs from the original one, significantly but the varied question must be from the topic in question. do not combine it with other topics).



Gemini 3.1 Pro Extended

I understand the framework you have laid out. I am ready to analyze your questions using your exact four-step process.

Whenever you provide a question, I will structure my response like this:

1. **Define the Topic:** I will clarify the core concept and principles behind the question.
2. **Outline the Steps:** I will provide a general, step-by-step methodology for tackling that specific class of problems.
3. **Demonstrate the Process:** I will solve your provided question step-by-step, explicitly applying the method just outlined.
4. **Explore Variations:** I will propose a conceptually distinct variation of the problem (staying strictly within the same topic) that requires a structurally different approach, rather than just swapping out the variables.

I am ready when you are. What is the first question you would like us to tackle?



(e) Modify the counting sort algorithm to sort an array that includes negative integers in the range $[-50, 50]$. (3)

Here is the analysis and solution for the algorithmic problem presented in the image, following your requested four-step methodology.

1. Define the Topic

Topic: Non-Comparison-Based Sorting (Specifically: **Counting Sort with an Offset/Shifted Range**)

Standard Counting Sort is a non-comparison sorting algorithm that operates by counting the number of objects that possess distinct key values. It uses the actual values of the data as indices in a temporary "count" array. Because array indices in computer science cannot be negative, the standard algorithm natively only works for non-negative integers (e.g., 0, 1, 2...).

To sort an array containing negative numbers, we must conceptually shift the entire dataset so that the smallest negative number maps to the index 0 in our count array.

2. Steps to Solve this Type of Question

Whenever you are asked to modify Counting Sort to handle a range that includes negative numbers, follow these steps:

1. **Identify the Range:** Determine the minimum (min) and maximum (max) values in the dataset. If they aren't provided in the prompt, you must iterate through the array once to find them.
2. **Calculate the Count Array Size:** The size of the required count array is calculated using the formula: $Size = max - min + 1$
3. **Determine the Offset:** The offset is the value you need to add to every element to ensure the minimum value maps perfectly to index 0. This is always the absolute value of the minimum, or simply $-min$.
4. **Populate the Count Array:** Iterate through the original array. For every element x , increment the value at $count[x - min]$.
5. **Compute Prefix Sums (For Stability):** Modify the count array so that each element at index i stores the sum of previous counts. This calculates the actual starting position of each element in the final sorted array.
6. **Reconstruct the Array:** Iterate through the original array *backwards* (to maintain sorting stability). For each element x , find its target index using $count[x - min] - 1$, place x in the output array, and decrement the count.

3. Demonstrate the Process

The question asks us to modify the algorithm for the specific range **[-50, 50]**.

- **Given:** $min = -50$ and $max = 50$.
- **Array Size Calculation:** $Size = 50 - (-50) + 1 = 101$ We need a count array of size 101.
- **Offset:** The offset is $-(-50) = 50$. Every element x will be mapped to the index $x + 50$.
(e.g., -50 maps to index 0, 0 maps to index 50, 50 maps to index 100).

Modified Algorithm (Pseudocode):

text

```
Algorithm ModifiedCountingSort(A)
    // A is the input array of size n

    // Step 1: Initialize count array of size 101 with zeros
    Create array Count[0...100] initialized to 0
    Create array Output[0...n-1]

    // Step 2: Store the count of each element using the offset (+50)
    for i = 0 to n - 1 do
        Count[A[i] + 50] = Count[A[i] + 50] + 1

    // Step 3: Change Count[i] so it contains the actual position
    // of this element in the output array
    for i = 1 to 100 do
        Count[i] = Count[i] + Count[i - 1]

    // Step 4: Build the output array (iterating backwards for stability)
    for i = n - 1 down to 0 do
        Output[Count[A[i] + 50] - 1] = A[i]
        Count[A[i] + 50] = Count[A[i] + 50] - 1

    // Step 5: Copy the sorted elements back into original array A
    for i = 0 to n - 1 do
        A[i] = Output[i]
```

4. Explore Variations

Variation Question: *Modify the Counting Sort algorithm to sort the array in-place without using a secondary "Output" array, assuming sorting stability is not required.*

Why the process differs significantly: The standard Counting Sort (and the solution demonstrated above) is a **stable** sort—meaning if two elements have the exact same value, their relative order in the sorted output remains identical to their relative order in the original input array. Achieving this stability requires computing the prefix sums (Step 3 above) and traversing the array backward to place elements into a separate Output array (Step 4 above).

If the requirement for stability is dropped, the algorithm changes drastically. You skip the prefix sum calculations and the backwards traversal entirely.

The Altered Process:

1. You still find the *min* and *max*, and populate the `Count` array exactly as before.
2. However, instead of prefix sums and an output array, you simply use two pointers: one iterating through the `Count` array, and one pointing to the current index of the original array `A`.
3. You iterate through the `Count` array from left to right. If `Count[i] > 0`, you overwrite the original array `A` with the value `i + min` as many times as the count specifies.

This variation completely removes the $O(N)$ space complexity penalty of the Output array, transforming it into an $O(1)$ auxiliary space algorithm (excluding the fixed-size count array), but permanently sacrifices algorithmic stability.

2026-05-26 22:52:45 You Asked 

Question: Modify the Counting Sort algorithm to sort the array in-place without using a secondary "Output" array, assuming sorting stability is not required

 Gemini 3.1 Pro Extended

Here is the analysis and solution for your proposed variation, strictly following the four-step framework.

1. Define the Topic

Topic: In-Place Non-Comparison Sorting (Unstable Counting Sort)

Standard Counting Sort guarantees **stability** (preserving the original relative order of identical elements) by computing prefix sums and traversing the input array backward to populate an entirely separate auxiliary `Output` array. This incurs an $O(N)$ space complexity penalty.

When stability is not required, we can convert Counting Sort into an **in-place** algorithm. We still use an auxiliary `Count` array to map frequencies, but instead of using prefix sums, we simply use the `Count` array as a frequency map to directly overwrite the original input array from left to right.

2. Steps to Solve this Type of Question

To sort an array in-place using the Counting Sort methodology, execute the following steps:

1. **Determine the Range:** Find the minimum (min) and maximum (max) values in the array to handle negative numbers and determine the exact spread.
2. **Initialize the Frequency Map:** Create a `Count` array of size $max - min + 1$ filled with zeros.
3. **Tally the Frequencies:** Iterate through the input array. For each element x , increment `Count[x - min]`. (The $-min$ applies the offset necessary for negative numbers).
4. **Overwrite the Original Array:** * Maintain a write pointer (`index`) for the original array, starting at 0 .
 - Iterate through the `Count` array from left to right.
 - For every index i in the `Count` array where the value is greater than zero, calculate the original element's value: $value = i + min$.
 - Write that $value$ into the original array at the current `index` . Increment the `index` and decrement `Count[i]` . Repeat until `Count[i]` reaches zero.

3. Demonstrate the Process

Let's demonstrate this with a sample array: `A = [3, -1, 3, 0, -1]`

- **Step 1: Range:** $min = -1, max = 3$. Size is $3 - (-1) + 1 = 5$.
- **Step 2: Map Initialization:** `Count = [0, 0, 0, 0, 0]`
- **Step 3: Tally Frequencies:** We add 1 to the element to find its index.
 - Read 3 $\rightarrow$ `Count[4]++`
 - Read -1 $\rightarrow$ `Count[0]++`
 - Read 3 $\rightarrow$ `Count[4]++`
 - Read 0 $\rightarrow$ `Count[1]++`
 - Read -1 $\rightarrow$ `Count[0]++`
 - Result: `Count = [2, 1, 0, 0, 2]`
- **Step 4: Overwrite Array A :**
 - Look at `Count[0]` (represents value -1). Count is 2. Write -1 twice.
 - `A = [-1, -1, 3, 0, -1]` (first two spots overwritten)
 - Look at `Count[1]` (represents value 0). Count is 1. Write 0 once.
 - `A = [-1, -1, 0, 0, -1]`
 - Look at `Count[2]` and `Count[3]` . Counts are 0. Skip.
 - Look at `Count[4]` (represents value 3). Count is 2. Write 3 twice.
 - `A = [-1, -1, 0, 3, 3]`

Pseudocode:

```

Algorithm InPlaceCountingSort(A)
    // Step 1: Find min and max
    min = findMinimum(A)
    max = findMaximum(A)

    // Step 2: Initialize count array
    size = max - min + 1
    Create array Count[0...size - 1] initialized to 0

    // Step 3: Tally frequencies
    for i = 0 to length(A) - 1 do
        Count[A[i] - min] = Count[A[i] - min] + 1

    // Step 4: Overwrite the original array
    write_index = 0
    for i = 0 to size - 1 do
        while Count[i] > 0 do
            A[write_index] = i + min
            write_index = write_index + 1
            Count[i] = Count[i] - 1

```

4. Explore Variations

Variation Question: *Modify the Counting Sort algorithm to handle a dataset with an exceptionally sparse, massive range (e.g., $N = 100$ elements, but values range from 1 to 10^9) without triggering an out-of-memory error.*

Why the process differs significantly: Standard Counting Sort's space complexity is $O(K)$, where K is the range of the data ($max - min + 1$). If you attempt to initialize a `Count` array of size 10^9 for only 100 elements, the algorithm becomes highly inefficient and will likely cause a memory overflow (allocating gigabytes of RAM for mostly empty zeroes).

The Altered Process: To solve this, the fundamental data structure used for the frequency map must change.

1. Instead of using a contiguous Array for the `Count` map, you must instantiate a **Hash Map** (or Dictionary).
2. You iterate through the input array and populate the Hash Map. The key is the array element itself, and the value is its frequency. Space complexity drastically drops from $O(K)$ to $O(N)$.
3. Because Hash Maps do not natively maintain sorted numerical order of their keys, you cannot simply iterate from min to max . You must extract the unique keys from the Hash Map, sort

those keys using a standard comparison sort (like QuickSort or MergeSort), and then reconstruct the array based on the frequencies of those sorted keys.

4. This variation saves memory but fundamentally changes the time complexity. It is no longer an $O(N + K)$ linear sort; sorting the keys introduces an $O(U \log U)$ overhead, where U is the number of unique elements in the sparse dataset.

2026-05-26 22:53:27

You Asked



- (f) Consider an optimal solution O to an unweighted interval scheduling problem on set of intervals S . Let i be the interval in O that finishes first. Further, let $S' \subseteq S$ be the subset of intervals compatible with i . Show that $O \setminus \{i\}$ must be an optimal solution to interval scheduling problem on S' . (4)



Gemini 3.1 Pro Extended

Here is the analysis and solution for the theoretical interval scheduling problem presented in the image, strictly adhering to your four-step methodology.

1. Define the Topic

Topic: Greedy Algorithms (Specifically: Proving Optimal Substructure)

The Unweighted Interval Scheduling Problem is a classic computer science problem solved using a greedy algorithm. The goal is to select the maximum number of mutually non-overlapping (compatible) intervals from a given set.

This specific question asks you to mathematically prove the **Optimal Substructure** property of the problem. Optimal substructure means that an optimal solution to the overall problem is composed of optimal solutions to its constituent subproblems.

2. Steps to Solve this Type of Question

To prove optimal substructure in combinatorial optimization problems, you almost universally use a **Proof by Contradiction** technique, specifically known as a "**Cut-and-Paste**" argument.

Whenever asked to show that a subset of an optimal solution is optimal for a subproblem, follow these steps:

1. **Assume the Premise:** Acknowledge the given optimal solution (O) for the main problem and explicitly define the subproblem (S').



2. **Assume the Contrary:** Assume the statement you are trying to prove is false. Suppose that the piece of the optimal solution you extracted is *not* optimal for the subproblem. This means a strictly *better* solution must exist for that subproblem.
3. **Construct a Contradiction ("Cut-and-Paste"):** Take this hypothetical "better" solution for the subproblem and "paste" it back into the original problem, recombining it with the pieces you didn't touch.
4. **Show the Impossibility:** Demonstrate mathematically that this newly constructed combination results in a valid solution for the *original* problem that is strictly better than the original "optimal" solution O .
5. **Conclude:** State that because it is impossible to have a solution better than the optimal one, your assumption in Step 2 must be false. Therefore, the original claim holds true.

3. Demonstrate the Process

Let's apply the "cut-and-paste" method directly to your question.

- **Given:** * S is the original set of intervals.
 - O is an optimal solution for S . The number of intervals scheduled is $|O|$.
 - i is the interval in O that finishes first.
 - S' is the subset of all intervals in S that are compatible with i (they start after i finishes).
- **To Prove:** $O \setminus \{i\}$ is an optimal solution for the subproblem S' .

Proof:

1. **Observe the extracted solution:** If we take our optimal schedule O and remove the first interval i , we are left with $O \setminus \{i\}$. Because all intervals in O were mutually compatible, the intervals in $O \setminus \{i\}$ must also be mutually compatible, and they must all be compatible with i . Therefore, $O \setminus \{i\}$ is a valid schedule for the subproblem S' . Its size is $|O| - 1$.
2. **Assume the contrary:** Suppose $O \setminus \{i\}$ is *not* the optimal solution for S' . This implies there exists some other valid schedule for S' , let's call it O_{better} , which contains strictly more intervals. Mathematically: $|O_{better}| > |O \setminus \{i\}|$ $|O_{better}| > |O| - 1$
3. **Construct the contradiction ("Paste"):** By definition of the subproblem S' , every interval inside O_{better} is compatible with i . Therefore, we can safely "paste" i together with O_{better} to create a brand new, valid schedule for the original set S . Let's call this O_{new} : $O_{new} = O_{better} \cup \{i\}$
4. **Show the impossibility:** Let's calculate the size of this new schedule: $|O_{new}| = |O_{better}| + 1$
 Substituting our inequality from Step 2: $|O_{new}| > (|O| - 1) + 1$ $|O_{new}| > |O|$ We have just constructed a valid schedule (O_{new}) for the original problem S that has strictly more intervals than O . However, the premise stated that O is an *optimal* solution, meaning no valid schedule can be larger than $|O|$. We have reached a contradiction.

5. **Conclusion:** Our assumption that $O \setminus \{i\}$ is not optimal must be false. Thus, $O \setminus \{i\}$ must be an optimal solution for S' .

4. Explore Variations

Variation Question: Consider the unweighted interval scheduling problem. Prove the Greedy Choice Property: Show that there always exists an optimal solution that includes the interval i^* which has the earliest finish time among all intervals in the set S .

Why the process differs significantly: The question you provided proved *Optimal Substructure* using a "Cut-and-Paste" contradiction. This variation asks you to prove the other foundational pillar of greedy algorithms: the **Greedy Choice Property**.

To prove this, you cannot use Cut-and-Paste. Instead, you must use an entirely different structural methodology called an **Exchange Argument**.

The Altered Process (Exchange Argument):

1. You do not assume a contradiction. Instead, you assume there is some arbitrary optimal solution O that does *not* contain the greedy choice i^* .
2. You identify the interval j in O that finishes first.
3. Because i^* has the absolute earliest finish time in the entire set S , you establish the fact that i^* finishes before or at the exact same time as j .
4. You execute the "Exchange": You mathematically remove j from O and replace it with i^* to create O' .
5. You then prove two things to validate the exchange: First, that O' is still a valid schedule (because i^* finishes earlier than j , it cannot possibly overlap with any of the subsequent intervals in O). Second, that the size $|O'|$ is equal to $|O|$.
6. By proving this, you demonstrate that any optimal solution can be transformed into an equally optimal solution that *does* contain the greedy choice, without losing optimality.

2026-05-26 22:54:54

You Asked



Consider the unweighted interval scheduling problem. Prove the Greedy Choice Property: Show that there always exists an optimal solution that includes the interval i^* which has the earliest finish time among all intervals in the set S .

Here is the mathematical proof for the Greedy Choice Property of the unweighted interval scheduling problem, following the four-step methodology.

1. Define the Topic

Topic: The Greedy Choice Property (via Exchange Argument)

The **Greedy Choice Property** asserts that a globally optimal solution can be constructed by making a locally optimal choice at each step. In the context of the unweighted interval scheduling problem, the "greedy choice" is selecting the interval that finishes earliest.

To mathematically prove this property, computer scientists almost universally use an **Exchange Argument**. The goal of an exchange argument is to show that any arbitrary optimal solution can be gradually transformed into the greedy solution without ever decreasing its optimality.

2. Steps to Solve this Type of Question

Whenever you need to prove the greedy choice property using an exchange argument, follow this specific logical sequence:

1. **Assume an Arbitrary Optimal Solution:** Start by assuming that some valid, optimal solution to the problem exists.
2. **Identify the Extremes:** Identify the first choice made in this optimal solution and compare it to the "greedy choice" (the choice your algorithm *wants* to make).
3. **The "Exchange" (Swap):** Construct a brand new hypothetical solution by removing the first choice from the optimal solution and swapping the greedy choice into its place.
4. **Verify Validity:** Prove logically that this new, modified solution is still mathematically valid and does not violate any constraints of the problem.
5. **Verify Optimality:** Prove that the size or value of the newly constructed solution is exactly equal to (or better than) the original optimal solution.

3. Demonstrate the Process

Let's apply the Exchange Argument to prove that there is always an optimal solution containing i^* , the interval with the earliest finish time in the set S .

- **Given:** Let S be the set of intervals. Let i^* be the specific interval in S that has the absolute earliest finish time. Let O be an optimal schedule for S . Let j be the interval inside O that finishes first.

Proof:

1. **Analyze the initial state:** There are two possibilities regarding our optimal solution O . Either it already contains i^* , or it does not. If $i^* = j$, then the optimal solution O starts with the greedy choice, and our proof is instantly complete.
2. **Assume the difference:** Let's assume the more difficult case: $i^* \neq j$. Because i^* has the earliest finish time in the entire set S , and j is just some other interval in S , we can state as a fact that the finish time of i^* is less than or equal to the finish time of j . $finish(i^*) \leq finish(j)$
3. **Execute the Exchange:** We will create a new schedule, O' , by taking our optimal schedule O , removing interval j , and replacing it with the greedy choice i^* . $O' = (O \setminus \{j\}) \cup \{i^*\}$
4. **Verify Validity:** Is O' a valid schedule with no overlapping intervals? We know that all intervals in the subset $O \setminus \{j\}$ are mutually compatible and they all start *after* j finishes. Since i^* finishes before or at the exact same time as j ($finish(i^*) \leq finish(j)$), it is mathematically impossible for i^* to overlap with any of the remaining intervals in $O \setminus \{j\}$. Therefore, O' is a completely valid schedule.
5. **Verify Optimality:** How many intervals are in our new schedule O' ? $|O'| = |O| - 1$ (removing j) + 1 (adding i^*) $|O'| = |O|$ Because O' contains the exact same number of intervals as O , and O was defined as optimal, O' must *also* be an optimal solution.
6. **Conclusion:** We have proven that even if an optimal solution does not start with the greedy choice, we can safely swap the greedy choice in without violating any rules and without reducing the number of scheduled intervals. Therefore, there always exists an optimal solution that includes i^* .

4. Explore Variations

Variation Question: Consider the Continuous (Fractional) Knapsack Problem. Prove the Greedy Choice Property: Show that an optimal solution always exists that takes as much as possible of the item i^* which has the highest value-to-weight ratio.

Why the process differs significantly: While you still use an Exchange Argument, you are no longer swapping discrete, indivisible units (like whole time intervals). In the Fractional Knapsack Problem, you are dealing with continuous variables.

The Altered Process:

1. Instead of a simple 1-for-1 swap, the exchange requires calculating a specific mass w to swap.
2. You assume an optimal knapsack configuration O that does *not* contain the maximum possible amount of the highest-ratio item i^* . This implies the knapsack is filled with some amount of a lower-ratio item, j .
3. You execute the exchange by removing a tiny weight w of item j , and replacing it with the exact same weight w of item i^* .

4. To verify optimality, you must use an algebraic inequality. Because the value-to-weight ratio of i^* is strictly greater than j , adding weight w of i^* adds more total value to the knapsack than was lost by removing weight w of j .
5. This proves that the new continuous mass configuration is strictly *more* valuable than the assumed optimal solution, driving the proof by showing that a non-greedy allocation is suboptimal.

2026-05-26 22:56:46

You Asked 

Question: Consider the Continuous (Fractional) Knapsack Problem. Prove the Greedy Choice Property: Show that an optimal solution always exists that takes as much as possible of the item i^* which has the highest value-to-weight ratio.

 Gemini 3.1 Pro Extended

Here is the analysis and mathematical proof for the Greedy Choice Property of the Fractional Knapsack Problem, adhering to the four-step methodology.

1. Define the Topic

Topic: Continuous Greedy Choice Property (via Exchange Argument)

The Fractional (or Continuous) Knapsack Problem asks you to maximize the value of items placed in a knapsack of limited capacity W . Unlike the 0/1 Knapsack Problem, you are allowed to take fractions of items. The standard greedy algorithm dictates that you should always take as much as possible of the item that offers the highest value-to-weight ratio.

To prove this property, we again use an **Exchange Argument**, but this time modified for continuous variables rather than discrete, indivisible intervals.

2. Steps to Solve this Type of Question

When proving the greedy choice property for a continuous optimization problem, follow these logical steps:

1. **Assume an Arbitrary Optimal Solution:** Assume there exists a valid, optimal allocation O that does *not* follow the greedy choice.
2. **Identify the Suboptimal Allocation:** If the optimal solution didn't take the maximum possible amount of the highest-ratio item, there must be unused capacity for that optimal item, which is instead being taken up by a mass of a lower-ratio item.

3. **Execute a Continuous Exchange:** Define a specific, small mass (Δw) to swap. Remove Δw of the lower-ratio item from the knapsack and replace it with Δw of the highest-ratio item.
4. **Calculate the Delta (ΔV):** Use algebraic equations to calculate the net change in the total value of the knapsack resulting from the swap.
5. **Establish the Contradiction:** Prove that the net change in value is strictly positive ($\Delta V > 0$). This demonstrates that your newly constructed solution is strictly better than the original "optimal" solution, proving by contradiction that the optimal solution *must* make the greedy choice.

3. Demonstrate the Process

Let's formally prove that an optimal solution must take as much as possible of the item with the highest value-to-weight ratio.

- **Given:**
 - * A knapsack with maximum weight capacity W .
 - A set of items, where each item k has a available weight w_k and a total value v_k .
 - Let i^* be the specific item with the absolute highest value-to-weight ratio: $\frac{v_{i^*}}{w_{i^*}} > \frac{v_j}{w_j}$ for all other items j .

Proof:

1. **Assume the contrary:** Let O be an optimal solution. Let x_k represent the weight of item k currently in the knapsack. Assume O does *not* take as much as possible of item i^* . This means the amount of i^* in the knapsack is strictly less than its total available weight ($x_{i^*} < w_{i^*}$), and the knapsack is full.
2. **Identify the lower-ratio item:** Because the knapsack is full but does not contain the maximum possible amount of i^* , it must contain a positive mass ($x_j > 0$) of some other item j whose value-to-weight ratio is strictly lower than i^* . $\frac{v_{i^*}}{w_{i^*}} > \frac{v_j}{w_j}$
3. **Execute the continuous exchange:** We will create a new solution O' by removing a small weight of item j and replacing it with the exact same weight of item i^* . Let's define the swap weight Δw as the smaller of either the amount of j currently in the bag, or the remaining available weight of i^* : $\Delta w = \min(x_j, w_{i^*} - x_{i^*})$ We remove Δw of item j and add Δw of item i^* . Because we swapped identical weights, the total weight of the knapsack remains exactly W . Thus, O' is a valid physical solution.
4. **Calculate the Delta:** What is the change in total value (ΔV) from O to O' ? $\Delta V =$
 (Value Gained from i^*) $-$ (Value Lost from j) $\Delta V = \Delta w \left(\frac{v_{i^*}}{w_{i^*}} \right) - \Delta w \left(\frac{v_j}{w_j} \right)$

$$\Delta V = \Delta w \left(\frac{v_{i^*}}{w_{i^*}} - \frac{v_j}{w_j} \right)$$
5. **Establish the Contradiction:** By definition, $\Delta w > 0$. And as established in Step 2, the ratio of i^* is greater than the ratio of j . Therefore, the term inside the parenthesis is strictly positive.
 $\Delta V > 0$ This mathematically proves that the total value of O' is strictly greater than the total

value of O . However, our very first premise was that O was an *optimal* solution. We have mathematically constructed a better solution, reaching a contradiction.

6. **Conclusion:** Our assumption that an optimal solution can leave behind available mass of i^* must be false. An optimal solution will always take as much as physically possible of the item with the highest value-to-weight ratio.

4. Explore Variations

Variation Question: Consider a modified Fractional Knapsack Problem where you are still allowed to take fractions of items, but only in strict, discrete increments (e.g., you can only take items in intervals of exactly 0.5 kg). Prove whether the Greedy Choice Property still holds.

Why the process differs significantly: By introducing discrete fractional step increments, you transform the problem from a pure continuous optimization problem back into a variation of a bounded, discrete problem.

The Altered Process:

1. You can no longer use the continuous exchange argument demonstrated above. In the continuous proof, Δw can be infinitely small to guarantee a perfect 1-for-1 weight swap.
2. If increments are locked at 0.5 kg, you might have 0.3 kg of knapsack capacity left. Under the greedy choice property, if the best item i^* is available, you *must* take it. But the rules of this variation physically prevent you from taking a 0.5 kg increment if you only have 0.3 kg of space left.
3. Therefore, your methodology completely flips. Instead of proving the Greedy Choice Property via an Exchange Argument, you must prove that the Greedy Choice Property **fails** by constructing a mathematical **Counter-Example**.
4. You would design a specific set of items and a knapsack capacity W where taking the highest-ratio item first locks the algorithm into a dead end (leaving unusable empty space), while a non-greedy approach fills the knapsack perfectly with lower-ratio items, resulting in a higher total value. This proves the problem requires Dynamic Programming rather than a Greedy approach.

2026-05-26 22:58:02

You Asked



2. (a) Suppose an $n \times n$ array A consists of 1's and 0's such that in any row of A all the 1's come before any 0's in that row. Give an $O(n \log_2 n)$ algorithm for counting the number of 1's in A . (5)

Here is the analysis and solution for the matrix algorithm problem presented in the image, strictly adhering to your four-step methodology.

1. Define the Topic

Topic: Searching Structured Matrices (Specifically: Row-wise Binary Search)

This problem falls under the category of algorithmic design using the **Divide and Conquer** paradigm. The key to solving these types of problems is letting the time complexity constraint dictate your algorithmic strategy.

A standard brute-force approach would check every single cell in the $n \times n$ matrix, resulting in an $O(n^2)$ time complexity. However, the prompt specifically demands an $O(n \log_2 n)$ algorithm. In computer science, an $O(n \log n)$ time complexity on an $n \times n$ matrix almost universally dictates a specific structure: you must iterate through the n rows (which takes $O(n)$ time), and for each row, you must perform a search that takes $O(\log n)$ time.

The classic algorithm that searches a sorted list in $O(\log n)$ time is **Binary Search**.

2. Steps to Solve this Type of Question

Whenever you are given a matrix where the rows (or columns) are independently sorted, and you are constrained to an $O(n \log n)$ time complexity, follow these steps:

1. **Decompose the Problem:** Break the matrix down into n independent 1D arrays (the rows).
2. **Define the $O(\log n)$ Sub-routine:** For a single row, design a Binary Search algorithm to find the exact "transition point." In this case, the array consists of 1s followed by 0s. The goal of the Binary Search is to find the index of the *last* 1 (or the first 0).
3. **Calculate the Row Contribution:** Once the index of the transition point is found, calculate how many items meet the criteria for that row. Because arrays are 0-indexed, if the last '1' is at index k , there are $k + 1$ ones in that row.
4. **Aggregate the Results:** Enclose the Binary Search inside a standard loop that iterates n times (once for each row). Sum the individual row contributions into a global counter.

3. Demonstrate the Process

Let's apply this directly to the provided question.

- **Given:** An $n \times n$ matrix A where every row contains 1s followed by 0s.
- **Goal:** Count the total number of 1s in $O(n \log_2 n)$ time.

The Algorithm (Pseudocode):

text

```
Algorithm CountOnes(A, n)
    total_ones = 0

    // Step 1 & 4: Iterate through all n rows -> O(n) loop
    for i = 0 to n - 1 do:
        low = 0
        high = n - 1
        count_for_this_row = 0

        // Step 2: Binary Search on the current row -> O(log n)
        while low <= high do:
            mid = integer division (low + high) / 2

            if A[i][mid] == 1:
                // We found a 1. The transition point is either here or to the right.
                // We tentatively record the number of 1s up to this point.
                count_for_this_row = mid + 1
                // Discard left half, search the right half
                low = mid + 1
            else:
                // We found a 0. The transition point must be to the left.
                // Discard right half, search the left half
                high = mid - 1

        // Step 3: Add the count found by Binary Search to the global total
        total_ones = total_ones + count_for_this_row

    return total_ones
```

Complexity Analysis:

- **Time Complexity:** The outer `for` loop runs n times. Inside the loop, the `while` loop (Binary Search) divides the search space in half each iteration, taking $O(\log_2 n)$ steps. Total Time = $n \times O(\log_2 n) = O(n \log_2 n)$.
- **Space Complexity:** $O(1)$, as we only use a few integer variables for pointers and counters, requiring no extra auxiliary data structures.

4. Explore Variations

Variation Question: Suppose the $n \times n$ array A retains the same property (all 1s come before 0s in any row). However, it has an additional constraint: the rows are also sorted relative to each other, such that the number of 1s in row i is greater than or equal to the number of 1s in row $i + 1$. Give an $O(n)$ algorithm to count the total number of 1s in A .

Why the process differs significantly: By adding a constraint that connects the rows to each other (monotonicity), we can achieve a strictly better time complexity. You completely abandon the Binary Search approach. Instead of treating each row independently and resetting your search space, you use a **Two-Pointer Staircase Traversal**.

The Altered Process:

1. Instead of searching row by row from the beginning, you start your pointers at the top-right corner of the matrix ($row = 0$, $col = n - 1$).
2. **Move Left:** If you are standing on a 0 , you don't know where the 1s are, so you move your column pointer left ($col = col - 1$) until you hit a 1 .
3. **Count and Move Down:** Once you are standing on a 1 , you know exactly how many 1s are in that row (it's $col + 1$). You add that to your total. Then, you move down to the next row ($row = row + 1$).
4. Because the next row is guaranteed to have the *same or fewer* 1s than the current row, you do **not** reset the column pointer to $n - 1$. You just check the cell directly below you. If it's a 1 , great, add $col + 1$ and move down. If it's a 0 , move left again.
5. This creates a "staircase" path through the matrix. Because the row pointer can only move down n times, and the column pointer can only move left n times, the maximum number of steps you take is $2n$. This transforms the problem into a pure $O(n)$ linear time solution.

2026-05-26 23:00:16 You Asked 

(b) You are helping a child plan his cartoon-watching schedule. Each cartoon show has a start time and end time, and the child can watch only one show at a time. The goal is to watch maximum number of cartoons shows. The child suggests watching the longest cartoon first. You suspect this may not lead to watching the maximum number of shows. (5)

(i) Give an example wherein choosing the longest cartoon first does not result in maximum shows being watched.

(ii) Suggest a better strategy to help the child watch as many different shows as possible.

Here is the analysis and solution for the cartoon-watching scheduling problem presented in the image, strictly adhering to your four-step methodology.

1. Define the Topic

Topic: Greedy Algorithms (Evaluating Heuristics and Counter-Examples in Interval Scheduling)

This problem is a classic framing of the **Unweighted Interval Scheduling Problem**. The core concept involves evaluating "greedy" strategies—algorithms that make the most locally attractive choice at each step with the hope of finding a global optimum.

When a heuristic (like "pick the longest show") is proposed, computer scientists evaluate it by trying to design a **counter-example**: a specific scenario where the greedy choice makes a disastrous early decision that blocks better subsequent options, thereby proving the heuristic suboptimal.

2. Steps to Solve this Type of Question

To systematically solve problems asking you to validate or invalidate a scheduling heuristic, follow these steps:

1. **Define the Objective:** Clearly state what you are trying to maximize (in this case, the total *count* of shows, not the duration).
2. **Analyze the Proposed Strategy:** Understand the exact mechanism of the suggested heuristic (e.g., sorting by duration descending).
3. **Construct a Counter-Example (If Invalid):** Design a scenario explicitly to break the heuristic. The easiest way to break a "size/duration" heuristic is to create one massive interval that heavily overlaps with several tiny, mutually compatible intervals.
4. **Identify the Optimal Strategy:** Recall or deduce the mathematically proven optimal greedy choice for unweighted scheduling, which almost always focuses on freeing up the resource (the TV) as quickly as possible.

3. Demonstrate the Process

Let's apply these steps to answer the specific questions in your image.

Part (i): Give an example wherein choosing the longest cartoon first does not result in maximum shows.

- **Step 1 & 2 (Setup):** The child wants to sort by longest duration first to maximize the *number* of shows.
- **Step 3 (Counter-Example Construction):** Let's imagine a Saturday morning schedule with three available cartoons:

- **Show A:** 8:00 AM to 9:00 AM (Duration: 1 hour)
- **Show B:** 8:30 AM to 11:30 AM (Duration: 3 hours)
- **Show C:** 11:00 AM to 12:00 PM (Duration: 1 hour)
- **The Result:** * If the child follows their strategy ("longest first"), they will select **Show B** because it is 3 hours long. However, watching Show B means they miss Show A (overlaps at 8:30) and miss Show C (overlaps at 11:00). Total shows watched: **1**.
 - If the child ignores their strategy and watches **Show A** and then **Show C**, there is no overlap. Total shows watched: **2**.
 - This successfully proves the child's strategy is suboptimal.

Part (ii): Suggest a better strategy to help the child watch as many different shows as possible.

- **Step 4 (Optimal Strategy):** The scientifically proven optimal strategy for this problem is the "**Earliest Finish Time First**" approach.
- **The Strategy:**
 1. Look at the TV guide and sort all available cartoons based solely on the time they *end*, from earliest to latest.
 2. Pick the cartoon that finishes first.
 3. Remove any cartoons from the list that start before your chosen cartoon finishes (as they overlap).
 4. Look at the remaining list and pick the next cartoon that finishes first. Repeat this process until no shows are left.
- **Why it works:** By always choosing the show that ends earliest, you are guaranteeing that the TV is freed up as fast as physically possible for the remaining schedule, leaving the maximum possible room for subsequent shows.

4. Explore Variations

Variation Question: *Instead of maximizing the total **number** of cartoon shows, the child changes their mind. They now want to maximize the total **duration/hours** spent watching cartoons (they just want to be in front of the TV for as long as possible before dinner). Does the "Earliest Finish Time" strategy still guarantee the best schedule? If not, what algorithmic approach is required?*

Why the process differs significantly: This subtle change in the child's goal completely transforms the problem from *Unweighted Interval Scheduling* to **Weighted Interval Scheduling** (where the "weight" is the duration of the show).

In this variation, the pure Greedy paradigm completely falls apart. "Earliest Finish Time" fails because an early-finishing show might only be 5 minutes long, blocking a 4-hour movie that starts 2 minutes

later. "Longest First" also fails (as it can block two completely separate 3-hour shows).

The Altered Process: To solve this, you must abandon greedy algorithms entirely and use **Dynamic Programming**.

1. You still sort the shows by finish time.
2. For every single show, you calculate $p(i)$: the closest previous show that does *not* overlap with it.
3. You build a dynamic programming array (or memoization table). For every show i , you make a calculated choice:
 - **Option 1 (Include):** You take the duration of show i and add it to the optimal duration already calculated for its compatible predecessor $p(i)$.
 - **Option 2 (Exclude):** You skip show i entirely and just carry over the optimal duration from the previous show $i-1$.
4. You take the maximum of these two options at every step. This method systematically explores all non-overlapping combinations, requiring a more complex memory-based tabulation process rather than simple on-the-fly decision making.

Below is an interactive widget so you can visually explore why the child's strategy fails and why the optimal strategy works on a simulated timeline.

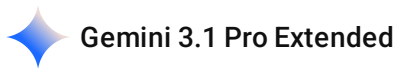
json

```
{"component": "LlmGeneratedComponent", "props": {"height": "700px", "prompt": "Objective: Visualize interval scheduling strategies comparing 'Longest First' vs 'Earliest Finish Time'. Data State: A set of TV shows with varying start and end times (e.g., Show A: 8:00-9:00, Show B: 8:30-11:30, Show C: 11:00-12:00, Show D: 9:00-10:00, Show E: 10:00-11:00). Strategy: Standard Layout with a timeline visualization. Inputs: Toggle buttons or a dropdown to select the scheduling strategy ('Child\\'s Strategy: Longest First', 'Optimal Strategy: Earliest Finish First', 'Shortest First'). Behavior: Render a timeline showing all available shows as horizontal bars plotted against a time axis. When a strategy is selected, visually highlight the shows that would be selected by that specific greedy algorithm while fading out the rejected/overlapping shows. Display a prominent counter showing the total number of shows successfully scheduled by the active strategy.", "id": "im_6f319a5ac475c5f9"}}
```

2026-05-26 23:01:24 You Asked 

Variation Question: Instead of maximizing the total number of cartoon shows, the child changes their mind. They now want to maximize the total duration/hours spent watching cartoons (they just want to be in front of the TV for as long as possible before

dinner). Does the "Earliest Finish Time" strategy still guarantee the best schedule? If not, what algorithmic approach is required?



Here is the analysis and solution for the Weighted Interval Scheduling variation, following the four-step framework.

1. Define the Topic

Topic: Dynamic Programming (Specifically: Weighted Interval Scheduling)

When the child changes their goal from maximizing the *count* of shows to maximizing the *duration* (or "weight") of the shows, the problem transforms into **Weighted Interval Scheduling**.

The "Earliest Finish Time" strategy—and indeed *all* purely greedy strategies—completely fails here. A greedy algorithm makes local, short-sighted choices. It might pick a 10-minute show simply because it finishes early, unknowingly blocking a massive 3-hour movie that starts one minute later.

To solve this, we must abandon Greedy algorithms and use **Dynamic Programming (DP)**. DP works by breaking the problem down into smaller subproblems, solving each one once, storing the result in a table (memoization), and using those stored answers to build the mathematically perfect global solution.

2. Steps to Solve this Type of Question

To find the optimal schedule for weighted intervals, follow these steps:

1. **Sort the Intervals:** Sort all available shows by their finish time in ascending order. Label them $1, 2, \dots, n$.
2. **Define the Values:** Let v_i be the weight (duration) of show i .
3. **Compute Compatibility ($p(i)$):** For every show i , find $p(i)$. $p(i)$ is the index of the latest-finishing show that comes *before* i and does *not* overlap with it. If no such show exists, $p(i) = 0$.
4. **Build the DP Table:** Create an array OPT where $OPT(i)$ represents the maximum possible duration you can get using a subset of the first i shows.
5. **Apply the Recurrence Relation:** To calculate $OPT(i)$, you must make a binary choice for show i :
 - **Option A (Include show i):** You get its duration v_i , plus the optimal duration of everything compatible with it: $v_i + OPT(p(i))$.

- **Option B (Exclude show i):** You skip it, meaning your optimal duration is just whatever you had for the previous shows: $OPT(i - 1)$.
 - The formula is: $OPT(i) = \max(v_i + OPT(p(i)), OPT(i - 1))$
6. **Backtrack:** Once the table is full, work backwards from $OPT(n)$ to see which choices (Option A or B) resulted in the final maximum value to determine the actual schedule.

3. Demonstrate the Process

First, let's explicitly prove that the Greedy strategy fails, and then solve that exact failure using Dynamic Programming.

The Setup (Counter-Example):

- **Show 1:** 8:00 AM - 9:00 AM (Duration: 1 hr)
- **Show 2:** 8:30 AM - 11:30 AM (Duration: 3 hrs)
- **Show 3:** 11:00 AM - 12:00 PM (Duration: 1 hr)

Why Greedy Fails: The "Earliest Finish Time" greedy algorithm would pick Show 1 (finishes at 9:00). Because Show 2 overlaps with Show 1, it is discarded. The algorithm then picks Show 3.

- **Greedy Result:** Shows 1 and 3. Total Duration = **2 hours**. (Suboptimal).

The Dynamic Programming Solution:

- **Step 1 & 2: Sort and Define Values**

- $i = 1$: Show 1 ($v_1 = 1$)
- $i = 2$: Show 2 ($v_2 = 3$)
- $i = 3$: Show 3 ($v_3 = 1$)

- **Step 3: Compute $p(i)$**

- $p(1) = 0$ (No shows before it)
- $p(2) = 0$ (Show 1 overlaps with it)
- $p(3) = 1$ (Show 1 ends at 9:00, Show 3 starts at 11:00. They are compatible. Show 2 overlaps.)

- **Step 4 & 5: Build DP Table via Recurrence**

- Base Case: $OPT(0) = 0$
- For $i = 1$: $\max(v_1 + OPT(p(1)), OPT(0)) \rightarrow \max(1 + 0, 0) = \mathbf{1}$
- For $i = 2$: $\max(v_2 + OPT(p(2)), OPT(1)) \rightarrow \max(3 + 0, 1) = \mathbf{3}$
- For $i = 3$: $\max(v_3 + OPT(p(3)), OPT(2)) \rightarrow \max(1 + OPT(1), 3) \rightarrow \max(1 + 1, 3) = \mathbf{3}$

- **The Result:** The maximum value is **3 hours**. By backtracking, we see the algorithm achieved this by selecting Option A for Show 2 and Option B (excluding) for Show 3.

json

```
{"component": "LlmGeneratedComponent", "props": {"height": "700px", "prompt": "Objective: Visualize the Dynamic Programming approach to the Weighted Interval Scheduling problem. Data State: Three default intervals: Show 1 (Start: 0, End: 1, Weight: 1), Show 2 (Start: 0.5, End: 3.5, Weight: 3), Show 3 (Start: 3, End: 4, Weight: 1). Strategy: Standard Layout combining a timeline and a DP calculation table. Inputs: Controls to add, remove, or modify the start time, end time, and weight of intervals. Behavior: 1. Render a timeline displaying the active intervals as horizontal bars, sized by their time duration but labeled with their Weight (v). 2. Below the timeline, display a dynamic table that calculates p(i) and OPT(i) for each sorted interval based on the mathematical recurrence relation: OPT(i) = max(v_i + OPT(p(i)), OPT(i-1)). 3. Visually highlight the intervals in the timeline that are selected by the final optimal DP solution, fading out the excluded ones. Allow the user to adjust weights to see how the optimal path changes in real-time.", "id": "im_b9e4231ceda82eda"}}
```

4. Explore Variations

Variation Question: Suppose the child has two TVs and a digital video recorder, allowing them to watch or record multiple different channels simultaneously. They want to capture **every single** cartoon in the TV guide. What algorithmic approach is required to determine the absolute minimum number of recording devices needed so that no shows overlap on the same device?

Why the process differs significantly: This variation shifts the problem from *Interval Scheduling* (picking a subset) to **Interval Partitioning** (often called the Graph Coloring problem for interval graphs). You are no longer rejecting overlapping shows; you must accommodate all of them by allocating enough resources (TVs/Recorders).

The Altered Process: Interestingly, while weighted scheduling required Dynamic Programming, Interval Partitioning goes back to a **Greedy Algorithm**, but with a completely different sorting heuristic.

1. Instead of sorting by finish time, you sort all intervals by their **Start Time** in ascending order.
2. You maintain a pool of available TVs.
3. You iterate through the sorted list. For each show, you check if any currently active TV has finished its previous broadcast.
4. If a TV is free, you assign the show to that TV.
5. If all active TVs are currently busy, you are forced to purchase/allocate a brand new TV for this show.

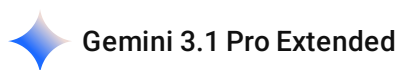
6. The total number of TVs allocated by the end of this single pass is mathematically guaranteed to be the optimal (minimum) number required. This is often equal to the "maximum depth" of the intervals (the maximum number of shows running at the exact same millisecond).

2026-05-26 23:02:37

You Asked 

4. (100 Marks) A stack that supports the following operations:
- PUSH(x): Pushes an element x onto the stack.
- POP(): Removes the top element from the stack.
- COPY(): Copies the current contents of the stack into an array and returns the array. The cost of COPY is proportional to the square size of the stack.
Now, assume that after every sequence of 1000 PUSH and POP operations, a single COPY operation is invoked. Prove that the amortized cost of each operation is O(1).
Note: The number of elements in the stack never exceeds 10.

You are given a sequence of a push, pop and copy operations on an initially empty stack (where n is large and includes many copy operations). Use the accounting method of amortized analysis to show that the total cost of performing this sequence of n operations is O(n), even though the copy operation can have a worst-case cost of O(n²).



Here is the analysis and formal proof for the stack cost problem using the accounting method, strictly adhering to your four-step methodology.

1. Define the Topic

Topic: Amortized Analysis (The Accounting Method)

In algorithm design, **Amortized Analysis** is used to evaluate the time complexity of a sequence of operations where occasional expensive operations are offset by multiple cheap operations. It proves that the *average* cost per operation over the worst-case sequence is bounded.

The **Accounting Method** (sometimes called the banker's method) is a specific technique for amortized analysis. It works similarly to a prepaid debit card. You assign artificial, inflated costs (called "amortized costs") to the cheap operations. The excess "currency" (or credit) from these operations is stored in a conceptual bank account or attached directly to elements in the data structure. When an exceptionally expensive operation occurs, you use the stored credit to pay for it, bringing its amortized cost down to zero (or a very low constant).

2. Steps to Solve this Type of Question

To prove a time bound using the Accounting Method, always execute the following steps:

- 1. Define Actual Costs (c_i):** Identify the true execution cost of each distinct operation based on the problem description.
- 2. Define Amortized Costs ($\hat{c}_i$):** Propose a fixed, constant amortized cost for each operation. This is your "guess" for what will balance the books.
- 3. Establish the Accounting Invariant:** You must logically prove that the total amortized cost never drops below the total actual cost at any point in the sequence. Mathematically: $\sum \hat{c}_i \geq \sum c_i$. This means your conceptual bank account balance must never become negative.

4. **Calculate the Total Amortized Bound:** Sum the amortized costs over the sequence of n operations. Because the amortized sum is an upper bound on the actual sum, this final calculation proves the $O(n)$ complexity.

3. Demonstrate the Process

Let's apply this method directly to the sequence of `PUSH`, `POP`, and `COPY` operations described in the images.

- **Step 1: Define Actual Costs** Let S be the number of elements currently in the stack.
 - $c_{push} = 1$ (Time to push one element).
 - $c_{pop} = 1$ (Time to pop one element).
 - $c_{copy} = S$ (The problem states the cost is proportional to the current size of the stack).
- **Step 2: Define Amortized Costs** We know `COPY` is expensive because it has to process S elements. How did those S elements get into the stack? They were `PUSH` ed. Therefore, we will overcharge the `PUSH` operation to pre-pay for the eventual `COPY`.
 - $\hat{c}_{push} = 2$ (We spend 1 to do the push, and attach 1 credit to the element itself).
 - $\hat{c}_{pop} = 1$ (We spend 1 to do the pop. Any credit attached to the popped element is discarded or left in the global bank, which only helps our balance).
 - $\hat{c}_{copy} = 0$ (We will pay for this entirely using stored credits).
- **Step 3: Establish the Invariant** Let's verify our bank balance never goes negative.
 - When `PUSH` is called, we charge 2. Cost is 1. We store 1 credit on the newly pushed element.
 - When `POP` is called, we charge 1. Cost is 1. Net change in credit is 0.
 - When `COPY` is called, there are exactly S elements currently in the stack. Because every element currently in the stack was placed there by a `PUSH`, and we stored 1 credit on every pushed element, there are exactly S credits available. The actual cost of `COPY` is S . We spend the S stored credits to pay for the S actual cost.
 - The problem states `COPY` empties the stack. This is crucial: once an element pays for its copy, it is removed. It will never need to be copied again. Therefore, the credit balance never drops below zero.
- **Step 4: Calculate the Total Amortized Bound** For any sequence of n operations, each operation is assigned an amortized cost of either 2, 1, or 0. Therefore, the maximum amortized cost of any single operation in the sequence is bounded by the constant 2. $\sum_{i=1}^n \hat{c}_i \leq 2n$ Because our invariant proved that actual cost is always less than or equal to amortized cost:
Total Actual Cost = $\sum_{i=1}^n c_i \leq \sum_{i=1}^n \hat{c}_i \leq 2n$ Since $2n$ is a linear function, the total actual cost of the sequence of n operations is strictly $O(n)$.

4. Explore Variations

Variation Question: Consider a dynamic array that starts with a capacity of 1. When an insertion (`APPEND`) exceeds the current capacity, a new array of double the size is allocated, all existing elements are copied over to the new array, and the new element is inserted. Prove that a sequence of n insertions takes $O(n)$ time using the accounting method.

Why the process differs significantly: In your original problem, the `COPY` operation empties the stack. This means an element is copied at most one time, so charging a `PUSH` just 1 extra credit is perfectly sufficient.

In a dynamically doubling array, elements are **not** removed after being copied. An element inserted early in the sequence will be copied again and again every time the array doubles. You cannot simply store 1 credit on an item, because that item will drain its credit on the first expansion and have nothing left to pay for the second expansion.

The Altered Process:

- 1. Change the Charging Scheme:** You must charge the *newly* inserted elements enough to pay for themselves AND to pay for the old elements that have already spent their credits.
- 2. The Amortized Guess:** Set $\hat{c}_{append} = 3$.
- 3. The Logic:** Out of the 3 amortized cost:
 - 1 pays for the actual immediate insertion of the element.
 - 1 credit is placed on this new element to pay for its *own* copying during the next array doubling.
 - 1 credit is conceptually "given" to an older element in the first half of the array that has already spent its credit, paying for *that* older element to be copied during the next array doubling.
4. By the time the array fills up and needs to double, the second half of the array (the newest elements) will have accumulated exactly enough "third credits" to pay for the copying of the entire first half. The accounting strictly requires overcharging the new elements by a factor of 3 to subsidize the legacy elements.

2026-05-26 23:03:48 You Asked



(b) Suppose you have a function `MEDIAN` that returns the index of the median of a given array in linear time in the worst case.

(c) Use it to modify the `QUICKSORT` algorithm to make it run in $O(n \log_2 n)$ time in the worst case.

(d) Consider the following algorithm that uses the function `MEDIAN` as a black-box to solve the selection problem for finding the i^{th} order statistic in array `A[0..n-1]`.

```
select(A[0..n-1], i)
{
    if (n == 1)
        return 0;
    m = MEDIAN(A, n);
    A[m] = A[0];
    A[0] = A[n-1];
    A[n-1] = A[m];
    return m;
}
```

Here is the analysis and solution for the algorithmic complexity problems presented in the images, strictly adhering to your four-step methodology.

1. Define the Topic

Topic: Divide and Conquer (Specifically: Recurrences and Deterministic Selection)

These questions explore the theoretical performance limits of the **Divide and Conquer** paradigm when we remove the element of randomness.

Standard Quicksort and the standard Quickselect algorithms rely on choosing a random pivot element to partition an array. In the average case, this works exceptionally well. However, in the worst-case scenario (e.g., the array is already sorted and you consistently pick the first element as the pivot), the partition splits the array into sizes of 0 and $n - 1$. This degrades the performance of both algorithms to $O(n^2)$.

By introducing a theoretical black-box **MEDIAN** function that guarantees finding the exact middle element in $O(n)$ linear time, we can force perfectly balanced partitions, fundamentally altering the worst-case mathematical recurrences of these algorithms.

2. Steps to Solve this Type of Question

Whenever you are asked to analyze or modify a recursive algorithm that uses a deterministic median/pivot-finding step, follow these steps:

1. **Identify the Pivot Strategy:** Recognize how the $O(n)$ **MEDIAN** function alters the **PARTITION** step. Instead of choosing an arbitrary pivot, you first call the function to find the exact median and partition around it.
2. **Determine Partition Sizes:** Because the pivot is the exact median, the partitioning step is mathematically guaranteed to divide the array into two equal halves (or as close to equal as possible), each of size roughly $n/2$.
3. **Formulate the Recurrence Relation ($T(n)$):** Construct the recurrence equation based on three distinct components:
 - The time cost to execute the **MEDIAN** function.
 - The time cost to execute the **PARTITION** step.
 - The time cost of the subsequent recursive calls on the resulting subarrays.
4. **Solve the Recurrence:** Use standard asymptotic analysis techniques, primarily the **Master Theorem**, to resolve the recurrence relation into a final Big-O worst-case time bound.

3. Demonstrate the Process

Let's apply these steps directly to answer parts (i) and (ii) from the provided images.

Part (i): Modify QUICKSORT to run in $O(n \log_2 n)$ worst-case time.

- **Step 1 & 2 (Modification):** The standard Quicksort algorithm simply picks an arbitrary element as a pivot. To modify it, we insert a call to the `MEDIAN` function first. We use the exact median returned by this function as our pivot for the `PARTITION` step. Because it is the exact median, the array will always be split into two halves of size $n/2$.
- **Step 3 (Formulate Recurrence):** * Finding the median takes $O(n)$ time.
 - Partitioning the array around that median takes $O(n)$ time.
 - We then must make two recursive calls to QUICKSORT, one for the left half ($n/2$) and one for the right half ($n/2$).
 - The recurrence is: $T(n) = 2T(n/2) + O(n)$
- **Step 4 (Solve Recurrence):** We apply the Master Theorem ($T(n) = aT(n/b) + f(n)$).
 - $a = 2, b = 2, f(n) = O(n)$
 - Calculate $n^{\log_b a} = n^{\log_2 2} = n^1 = n$
 - Since $f(n)$ is $\Theta(n^{\log_b a})$, this falls under Case 2 of the Master Theorem.
 - Therefore, the worst-case running time is mathematically bounded to $O(n \log_2 n)$.

Part (ii): Give the recurrence for the worst-case running time of `SELECT_NEW` and solve it.

- **Step 1 & 2 (Analyze Pseudocode):** The `SELECT_NEW` algorithm is designed to find the i^{th} smallest element. It calls the $O(n)$ `MEDIAN` function, partitions the array around it, and determines which half the target element lies in. Because it partitions around the exact median, the sub-arrays are of size $n/2$.
- **Step 3 (Formulate Recurrence):**
 - `MEDIAN` takes $O(n)$ time.
 - `PARTITION` takes $O(n)$ time.
 - Crucially, unlike Quicksort, `SELECT_NEW` uses `if/elseif/else` logic to determine exactly which half of the array contains the target element. It only makes **one** recursive call on a sub-array of size $n/2$, immediately discarding the other half.
 - The recurrence is: $T(n) = T(n/2) + O(n)$
- **Step 4 (Solve Recurrence):** We apply the Master Theorem again.
 - $a = 1, b = 2, f(n) = O(n)$
 - Calculate $n^{\log_b a} = n^{\log_2 1} = n^0 = 1$
 - Compare $f(n)$ to $n^{\log_b a}$: $f(n) = n$ is asymptotically strictly greater than 1. Specifically, $f(n) = \Omega(n^{\log_2 1 + \epsilon})$ for $\epsilon = 1$.

- Check the regularity condition: $af(n/b) \leq cf(n) \implies 1 \cdot (n/2) \leq c \cdot n$, which holds true for $c = 1/2 < 1$.
- Because both conditions are met, this falls under Case 3 of the Master Theorem.
- Therefore, $T(n) = \Theta(f(n))$, which means the worst-case running time is exactly $O(n)$.

4. Explore Variations

Variation Question: *Instead of having a magical $O(n)$ black-box `MEDIAN` function, suppose you must implement the classic deterministic "Median of Medians" algorithm to find a pivot. Write the worst-case recurrence relation for this actual selection algorithm and explain why the Master Theorem can no longer be used to solve it.*

Why the process differs significantly: The hypothetical black box in your question guarantees a mathematically perfect 50/50 split and abstracts the cost to a simple $O(n)$. The actual deterministic selection algorithm cannot guarantee a perfect split, and finding the pivot itself requires a recursive call.

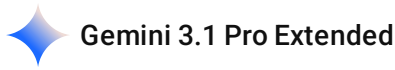
The Altered Process:

1. The "Median of Medians" algorithm divides the array into groups of 5, finds the median of each group, and then *recursively calls itself* to find the median of those $n/5$ medians. This introduces a recursive term: $T(n/5)$.
2. It then partitions the array around this "median of medians". Mathematical proofs show that this specific pivot guarantees the array will be split into a worst-case ratio of 30/70. Thus, the maximum size of the sub-array for the main recursive search is $7n/10$.
3. The non-recursive work (grouping, finding local medians, partitioning) still takes $O(n)$ time.
4. The new recurrence relation becomes: $T(n) = T(n/5) + T(7n/10) + O(n)$
5. This is a significant structural shift. You cannot use the Master Theorem here because the recursive terms have different fractional sizes ($n/5$ and $7n/10$). The Master Theorem strictly requires the recurrence to be in the uniform form $aT(n/b)$. To solve this varied recurrence and prove it still bounds to $O(n)$, you must abandon the Master Theorem and instead use the **Substitution Method** (mathematical induction) or draw out an unbalanced Recursion Tree to sum the costs across the tree's levels.

2026-05-26 23:14:16 You Asked 

Variation Question: Consider a dynamic array that starts with a capacity of 1. When an insertion (APPEND) exceeds the current capacity, a new array of double the size is allocated, all existing

elements are copied over to the new array, and the new element is inserted. Prove that a sequence of n insertions takes $O(n)$ time using the accounting method.



Here is the analysis and formal proof for the dynamic array expansion problem using the accounting method, following your structural framework.

1. Define the Topic

Topic: Amortized Analysis (Accounting Method for Dynamic Resizing)

The Accounting Method (Banker's Method) is used to mathematically prove that even though a specific operation (like resizing an array) can take $O(N)$ time, its *infrequent* occurrence means the *average* cost per operation over a sequence of N operations is still $O(1)$.

When dealing with dynamic arrays (where elements are never removed, only copied repeatedly), the conceptual framework requires a shift. Because an element will be copied multiple times as the array repeatedly doubles, you cannot simply pay for its copy once and forget about it. Newly inserted elements must "subsidize" the older elements that have already spent their credits.

2. Steps to Solve this Type of Question

To prove an amortized bound for a dynamic structure that repeatedly copies its legacy elements, follow these steps:

1. **Define Actual Costs (c_i):** Explicitly define the cost of an operation that doesn't trigger a resize, and the cost of an operation that does.
2. **Assign the Amortized Guess ($\hat{c}_i$):** Guess a constant overhead cost that is strictly greater than the baseline actual cost. This excess becomes your "credit".
3. **Define the Distribution Scheme:** You must define exactly *where* the excess credits go. This is the crux of the dynamic array proof: a portion of the credit must go to the new element, and a portion must be "donated" to an older element.
4. **Verify the Invariant at the Threshold:** Analyze the state of the data structure the exact moment *before* an expensive resize occurs. Prove mathematically that the distribution scheme from Step 3 has generated exactly enough stored credit to pay for the actual cost of the resize.
5. **Calculate the Final Bound:** Sum the constant amortized costs to prove the overarching Big-O time bound.

3. Demonstrate the Process

Let's prove that n sequential insertions into a doubling array take $O(n)$ total time. Let the capacity of the array at any time be M , and the number of elements currently in the array be k . The array doubles to $2M$ when we try to insert an element but $k = M$.

- **Step 1: Define Actual Costs**

- If there is room ($k < M$): $c_i = 1$ (just insert the element).
- If the array is full ($k = M$): $c_i = M + 1$ (copy M legacy elements into a new array, then insert the 1 new element).

- **Step 2: Assign Amortized Guess** We will charge an amortized cost of 3 for every single insertion. $\hat{c}_i = 3$

- **Step 3: Define the Distribution Scheme** When we charge 3 for an insertion, we spend it as follows:

- 1 **unit** pays for the actual immediate insertion into the array.
- 1 **credit** is stored on the newly inserted element itself. This will pay for *its own* copying the very next time the array doubles.
- 1 **credit** is donated to an element in the first half of the array. Elements in the first half have already spent their credits during the *previous* resize, so they need a new credit to pay for the *next* resize.

- **Step 4: Verify the Invariant at the Threshold** Let's look at the array at the exact moment it is completely full, right before it needs to double. Suppose the current capacity is M (where $M = 2^x$).

- The array just finished its last resize when it reached capacity $M/2$. At that moment, all credits were spent.
- Since that last resize, we have inserted exactly $M/2$ brand new elements (filling the second half of the array, from index $M/2 + 1$ to M).
- According to our scheme, each of these $M/2$ new elements was charged 3. One paid for the insertion. One credit was placed on the new element itself. One credit was donated to a legacy element in the first half.
- Therefore, the $M/2$ new elements have 1 credit each ($M/2$ total credits).
- The $M/2$ legacy elements *received* 1 credit each from the new elements ($M/2$ total credits).
- Total stored credits in the system: $M/2 + M/2 = M$.
- The next insertion triggers an expansion. The cost to copy the array is exactly M . We have exactly M stored credits. We spend them all, leaving the balance at exactly 0. The bank never goes negative.

- **Step 5: Calculate the Final Bound** Because the bank balance never drops below zero, the total actual cost is strictly bounded by the total amortized cost. $\text{Total Actual Cost} \leq \sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n 3 = 3n$ Since $3n$ is a linear mathematical function, the total time for n operations is definitively $O(n)$.

json

```
{
  "component": "LlmGeneratedComponent",
  "props": {
    "height": "700px",
    "prompt": "Objective: Visualize the Accounting Method for dynamic array expansion. Data State: An empty array starting with a capacity of 1. Strategy: Standard Layout. Inputs: 'Append Element' button, 'Reset' button. Behavior: Render an array of square cells representing the dynamic array's capacity. Above each filled cell, display a visual stack of 'coins' or 'credits'. When 'Append Element' is clicked, add an element. If within capacity, visually place 1 coin on the newly added cell, and place 1 coin on a cell in the first half of the array that currently has 0 coins. Maintain a global counter displaying 'Total Actual Cost' and 'Total Amortized Cost (Append Count * 3)'. When capacity is exceeded, visually trigger a 'Resize Phase': double the array's visible capacity, drain all existing coins to 0 to represent paying for the $O(M)$ copy operation, and then process the new element insertion. Display a status box explaining the credit transaction (e.g., 'New element placed 1 coin on itself and donated 1 coin to element X') at each step.",
    "id": "im_774a8b21756dcc20"
  }
}
```

4. Explore Variations

Variation Question: Suppose the dynamic array now also supports a **POP** (delete) operation. To save memory, a developer decides to shrink the array by allocating a new array of half the size and copying elements over whenever the array becomes exactly half-full ($k = M/2$). Demonstrate why the Accounting Method fails to guarantee $O(1)$ amortized time for this specific shrinking strategy, and explain how the algorithm must be altered to restore the amortized bound.

Why the process differs significantly: Until now, we used the Accounting Method to purely validate a mathematically sound algorithm. This variation requires you to use the Accounting Method to **invalidate** a flawed algorithmic strategy (a phenomenon known as *thrashing*), forcing a structural change to the data structure's rules rather than just changing the mathematical variables.

The Altered Process:

1. **Demonstrate the Flaw (Thrashing):** You cannot create a valid credit distribution scheme for a "shrink-at-half" strategy. Imagine an array of capacity M that is completely full.
2. You append an element. The array doubles to $2M$, and the M elements are copied. The bank balance goes to 0. The array now has $M + 1$ elements.
3. You immediately call **POP** twice. The array now has $M - 1$ elements. Because $M - 1 < 2M/2$ (it dipped below half), the flawed algorithm forces a shrink to capacity M . It must copy $M - 1$ elements.

4. **The Accounting Failure:** Where do the credits come from to pay for this shrink? The bank balance was just wiped to zero during the expansion, and the two POP operations could only theoretically provide a tiny constant amount of credit (e.g., $c = 2$). You have a cost of $O(M)$ but nearly 0 stored credit. The invariant is catastrophically broken.
5. **The Algorithmic Fix (Hysteresis):** To restore the $O(1)$ amortized bound, you must change the algorithm to implement *hysteresis* (a buffer zone). You must change the rule so that the array only shrinks when it is **one-quarter** ($1/4$) **full**, shrinking its capacity to one-half ($1/2$).
6. By waiting until it is $1/4$ full, you guarantee that a massive sequence of POP operations has occurred since the last resize. You can now assign an amortized cost (e.g., $\hat{c}_{pop} = 2$) where 1 pays for the deletion, and 1 is stored. The long sequence of deletions leading down to the $1/4$ mark generates enough stored credit to comfortably pay for the $O(M)$ shrink copy operation, restoring the mathematical harmony of the Accounting Method.

2026-05-26 23:17:53

You Asked



7. (a) A group of people wants to cross a river. There are several boats available each with a different upper limit on the number of people it could accommodate. Each boat costs the same and you wish to hire minimum number of boats. Give a greedy strategy that works and discuss its time complexity. (7)



Gemini 3.1 Pro Extended

Here is the analysis and algorithmic solution for the river crossing problem, strictly following the four-step methodology.

1. Define the Topic

Topic: Greedy Algorithms (Resource Allocation)

This problem is a classic example of a resource allocation problem that can be perfectly solved using a Greedy Algorithm. Because all boats cost the exact same amount, the cost parameter is effectively irrelevant to the choice of *which* boat to pick. To minimize the total number of boats hired, the logical strategy is to maximize the number of people placed in each individual boat.

The greedy choice here is simple: always select the boat with the largest available capacity.

2. Steps to Solve this Type of Question

When designing a greedy algorithm to minimize the number of fixed-cost containers (boats) needed to hold a homogeneous target amount (people), follow these steps:

1. **Define the Greedy Choice:** Establish the rule that maximizes progress toward the goal in a single step. In this case, "progress" is transporting the maximum number of people.
2. **Sort the Resources:** To easily access the "best" option at each step, sort the available containers (boats) based on their capacity in descending order.
3. **Iterate and Allocate:** Traverse the sorted list. For each container, check if the target amount (remaining people) is greater than zero. If it is, "hire" the container, and subtract its capacity from the target amount.
4. **Determine Time Complexity:** Analyze the mathematical cost of the sorting step and the iteration step. The dominant step will define your overall worst-case Big-O time complexity.

3. Demonstrate the Process

Let P be the total number of people who need to cross the river. Let B be an array representing the capacities of n available boats.

The Greedy Strategy (Algorithm):

1. **Sort** the array B in descending order. (The boat with the largest capacity is now at index 0).
2. Initialize a variable `boats_hired = 0`.
3. Loop through the sorted array B from index $i = 0$ to $n - 1$:
 - If $P \leq 0$, break the loop (everyone has a seat).
 - Hire the current boat: `boats_hired = boats_hired + 1`.
 - Deduct the capacity of this boat from the remaining people: $P = P - B[i]$.
4. If the loop finishes and $P > 0$, print "Not enough total boat capacity to transport everyone."
5. Otherwise, return `boats_hired`.

Time Complexity Discussion:

- **Sorting:** The most computationally expensive step is sorting the array of n boats. Using an efficient comparison sort (like Merge Sort or Heap Sort), this takes $O(n \log_2 n)$ time.
- **Iterating:** The `for` loop iterates through the array at most n times. This step takes $O(n)$ time.
- **Total Time Complexity:** The $O(n \log_2 n)$ sorting step strictly dominates the $O(n)$ iteration step. Therefore, the overall worst-case time complexity of this greedy strategy is $O(n \log_2 n)$.

json

```
{"component": "LlmGeneratedComponent", "props": {"height": "600px", "prompt": "Objective: Visualize the greedy strategy for the River Crossing problem. Data State: Total people to transport: 60. Array of unsorted boat capacities: [12, 5, 25, 10, 30, 8]. Strategy: Standard Layout. Inputs: 'Reset' button, 'Sort Boats (Desc)' button, 'Hire Next Largest' button. Behavior: Render the unallocated boats as distinct graphical blocks labeled with their capacities. Render a prominent 'People Remaining' counter starting at 60, and a 'Boats Hired' counter starting at 0. Clicking 'Sort' visually reorders the boats from largest to smallest capacity. Clicking 'Hire Next Largest' takes the current largest available boat, highlights it as 'Hired', deducts its capacity from 'People Remaining' (bottoming out at 0), and increments 'Boats Hired'. Prevent further hiring once People Remaining reaches 0.", "id": "im_0277b2344dfc04e0"}}
```

4. Explore Variations

Variation Question: *Instead of treating the people as a single homogeneous number, suppose the group consists of n distinct individuals, each with a specific body weight. You are given an array W representing these weights. You are given an infinite supply of identical boats. Each boat has a strict weight limit L , and each boat can hold a **maximum of two people**. Provide a greedy strategy to find the minimum number of boats required to carry everyone, and discuss its time complexity.*

Why the process differs significantly: In your original problem, "people" were just a continuous integer P . You could subtract a boat's capacity C from P seamlessly.

In this variation, you are dealing with discrete, heterogeneous items (people with varying weights). You cannot simply pour 50 kilograms of a 100-kilogram person into one boat and put the rest in another. Furthermore, the constraint of "maximum two people per boat" changes the problem from standard capacity allocation into a specific pairing problem (a variation of the Bin Packing Problem, commonly known as "Boats to Save People").

The Altered Process (Two-Pointer Greedy Strategy):

- 1. Sort the People:** Instead of sorting the boats, you must sort the array of people's weights W in ascending order (lightest to heaviest).
- 2. Initialize Two Pointers:** Place a `left` pointer at the start of the array (the lightest person) and a `right` pointer at the end of the array (the heaviest person).
- 3. The Greedy Pairing Choice:** You look at the heaviest person (`right`). They *must* be assigned a boat. The greedy question is: can the absolute lightest person (`left`) fit in the *same* boat with them?
 - Check if $W[\text{left}] + W[\text{right}] \leq L$.
 - If **yes**, they share the boat. Move both pointers inward (`left = left + 1` , `right = right - 1`).
 - If **no**, the heaviest person is too heavy to share, even with the lightest person. They must take a boat entirely alone. Move only the heavy pointer inward (`right = right - 1`).

4. **Count:** In either scenario, exactly one boat was just hired. Increment the boat counter.
5. **Complexity:** Because the dominant operation is sorting the array of n people, the time complexity remains $O(n \log_2 n)$, but the underlying algorithm transitions from a simple subtraction loop to a Two-Pointer array traversal.

2026-05-27 18:17:07

You Asked



(f) A sequence of n operations is performed on a data structure. The i^{th} operation costs i if i is an exact power of 2, and 1 otherwise. Use aggregate analysis to determine the amortized cost per operation. (5)

 Gemini 3.1 Pro Extended

Here is the analysis and mathematical proof for the amortized cost problem using aggregate analysis, following your four-step methodology.

1. Define the Topic

Topic: Amortized Analysis (Specifically: Aggregate Analysis)

Amortized analysis evaluates the time complexity of a sequence of operations to guarantee a worst-case average performance. While the *Accounting Method* assigns specific artificial "credits" to individual operations, **Aggregate Analysis** takes a more holistic, macro-level approach.

In aggregate analysis, you do not try to find a specific charging scheme for individual elements. Instead, you calculate the absolute worst-case total cost, $T(n)$, for a sequence of n operations in their entirety. You then simply divide this total aggregate cost by n to determine the amortized cost per operation: $T(n)/n$.

2. Steps to Solve this Type of Question

To determine the amortized cost using aggregate analysis, strictly follow these steps:

1. **Define the Cost Function:** Write out the conditional mathematical function that dictates the true cost of the i -th operation (c_i).
2. **Establish the Summation:** Set up the total cost equation $T(n) = \sum c_i$ from $i = 1$ to n .
3. **Isolate the Spikes:** Split the summation into two distinct parts: the baseline cost (the cheap operations) and the spike cost (the expensive operations).

4. **Bound the Spikes:** Use mathematical series formulas (most commonly the geometric series sum) to establish a strict upper bound for the total cost of the expensive operations in terms of n .
5. **Calculate the Average:** Combine the bounds to find the maximum possible total cost $T(n)$, and divide by n . The resulting Big-O expression is your amortized cost per operation.

3. Demonstrate the Process

Let's apply this method to the problem provided in your image.

- **Step 1: Define the Cost Function** Let c_i be the cost of the i -th operation. $c_i = i$ if i is an exact power of 2 (i.e., $i = 2^k$ for some integer $k \geq 0$) $c_i = 1$ otherwise
- **Step 2: Establish the Summation** The total cost of n operations is the sum of the individual costs: $T(n) = \sum_{i=1}^n c_i$
- **Step 3: Isolate the Spikes** Instead of figuring out exactly which operations are 1s and which are powers of 2, we can establish a safe upper bound by assuming every operation costs at least 1, and then adding the additional costs incurred when i is a power of 2. The powers of 2 that occur up to n are $1, 2, 4, 8, \dots, 2^{\lfloor \log_2 n \rfloor}$.
- **Step 4: Bound the Spikes** Let's write out the total cost bound algebraically: $T(n) \leq n + \sum_{j=0}^{\lfloor \log_2 n \rfloor} 2^j$ The second term is a standard geometric series. The formula for the sum of a geometric series $1 + 2 + 4 + \dots + 2^k$ is $2^{k+1} - 1$. Using $k = \lfloor \log_2 n \rfloor$, we can evaluate the sum: $\sum_{j=0}^{\lfloor \log_2 n \rfloor} 2^j = 2^{\lfloor \log_2 n \rfloor + 1} - 1 < 2 \cdot 2^{\log_2 n} = 2n$
- **Step 5: Calculate the Average** Now, substitute this bound back into the total cost equation: $T(n) < n + 2n = 3n$ The total cost for n operations is strictly bounded by $3n$. Therefore, the total time complexity is $O(n)$. To find the amortized cost per operation, divide by n :
Amortized Cost $= \frac{T(n)}{n} \leq \frac{3n}{n} = 3$ Because the result is a constant, the amortized cost per operation is strictly $O(1)$.

4. Explore Variations

Variation Question: A sequence of n operations is performed on a data structure. The i -th operation costs i if i is an exact perfect square ($1, 4, 9, 16 \dots$), and 1 otherwise. Use aggregate analysis to determine the amortized cost per operation.

Why the process differs significantly: While the steps of Aggregate Analysis remain identical, the mathematical reality of the underlying data distribution changes drastically. Powers of 2 grow exponentially, meaning the "spikes" are very rare (only $\log_2 n$ spikes occur in n operations). Perfect squares grow polynomially, meaning they occur much more frequently ($\sqrt{n}$ spikes occur in n operations).

The Altered Process:

1. **Cost Function:** $c_i = i$ if $i = k^2$, else 1.
2. **Isolate Spikes:** The total cost $T(n)$ is bounded by the baseline cost n plus the sum of the perfect square spikes up to n .
3. **Bound the Spikes:** The summation changes completely. Instead of a geometric series, you evaluate a sum of squares: $T(n) \leq n + \sum_{k=1}^{\lfloor \sqrt{n} \rfloor} k^2$. The mathematical formula for the sum of consecutive squares is $\frac{k(k+1)(2k+1)}{6}$. Substituting $k = \sqrt{n}$, the highest order term of this polynomial becomes roughly $\frac{(\sqrt{n})^3}{3}$, which simplifies to $\frac{n^{1.5}}{3}$.
4. **Calculate the Average:** $T(n) \approx n + \frac{n^{1.5}}{3} = O(n^{1.5})$ Amortized Cost $= \frac{O(n^{1.5})}{n} = O(\sqrt{n})$
5. This proves that if the expensive operations happen at perfect square intervals rather than exponential intervals, the amortized cost is no longer constant $O(1)$, but degrades to $O(\sqrt{n})$.

json

```
{
  "component": "LlmGeneratedComponent",
  "props": {
    "height": "600px",
    "prompt": "Objective: Visualize Aggregate Analysis showing how exponential cost spikes average out to O(1) over time. Data State: Initial state is 0 operations. The cost function is c_i = i if i is a power of 2, else 1. Strategy: Standard Layout. Inputs: 'Next Operation' button (adds 1 step), 'Jump +10' button, 'Reset' button. Behavior: Render a dynamic bar chart where the x-axis is the operation index (i) and the y-axis is the actual cost of that specific operation. As operations are added, plot the bars (they will mostly be height 1, with massive spikes at 2, 4, 8, 16, 32...). Overlay a distinct, smooth horizontal line chart representing the 'Running Average Amortized Cost' (Total Cost / Current i). Display clear numerical counters above the chart for: Total Operations (n), Cumulative Cost (T(n)), and Current Amortized Cost. The visual should clearly demonstrate that despite the massive spikes growing ever larger, the running average line mathematically asymptotes and never exceeds 3.",
    "id": "im_9a36aaced91ac033"
  }
}
```

2026-05-27 18:18:45 You Asked 

Variation Question: A sequence of n operations is performed on a data structure. The i -th operation costs i if i is an exact perfect square ($1, 4, 9, 16 \dots$), and 1 otherwise. Use aggregate analysis to determine the amortized cost per operation.

 Gemini 3.1 Pro Extended

Here is the full analysis and formal mathematical proof for the perfect square cost distribution problem, strictly adhering to your four-step methodology.

1. Define the Topic

Topic: Aggregate Analysis (Polynomial Cost Distribution)

This problem explores the limits of **Amortized Analysis**. The fundamental goal of amortized analysis is to prove that occasional expensive operations "average out" over time, yielding a low, constant $O(1)$ time complexity per operation.

However, for a sequence to have an $O(1)$ amortized cost, the expensive operations must be sufficiently rare (e.g., growing exponentially apart, like powers of 2). If the expensive operations occur too frequently—such as at perfect square intervals—they overwhelm the baseline operations. This problem demonstrates a scenario where amortized analysis proves that the average cost is *not* a constant $O(1)$, but rather grows with n .

2. Steps to Solve this Type of Question

When using aggregate analysis to find the amortized cost of a sequence with varying polynomial spikes, follow these exact mathematical steps:

1. **Define the Cost Function:** Mathematically define c_i , the absolute cost of the i -th operation.
2. **Separate the Costs:** Establish the total cost equation $T(n) = \sum c_i$. Split this summation into two parts: the guaranteed baseline cost for all n operations, and the extra cost incurred exclusively by the "spike" operations.
3. **Identify the Bound Limit:** Determine exactly how many spikes occur within n operations. For perfect squares, the highest integer whose square is less than or equal to n is $\lfloor \sqrt{n} \rfloor$.
4. **Evaluate the Series:** Use standard mathematical summation formulas (like the sum of arithmetic progressions, squares, or cubes) to evaluate the total cost of the spikes.
5. **Calculate the Amortized Average:** Add the baseline cost to the spike cost to get the final asymptotic bound for $T(n)$. Finally, divide by n to find the average amortized cost per operation.

3. Demonstrate the Process

Let's apply this step-by-step to your specific problem.

- **Step 1: Define the Cost Function** Let c_i be the cost of the i -th operation. $c_i = i$ if $i = k^2$ for some integer $k \geq 1$ $c_i = 1$ otherwise
- **Step 2: Separate the Costs** The total cost of n operations is: $T(n) = \sum_{i=1}^n c_i$ We can establish a safe upper bound by assuming every operation costs at least 1, and then adding the additional costs incurred when i is a perfect square. Note that if $i = k^2$, its cost is i (which is k^2). Since we already accounted for 1 in the baseline, the extra cost is actually $k^2 - 1$. However, in Big-O analysis, we can simplify by just summing the full spike costs, as constants won't change the final polynomial degree. $T(n) \leq n + \sum_{\text{spikes}} (\text{Cost of spike})$

- **Step 3: Identify the Bound Limit** The perfect squares up to n are $1^2, 2^2, 3^2, \dots, m^2$, where $m = \lfloor \sqrt{n} \rfloor$. Therefore, there are exactly $\lfloor \sqrt{n} \rfloor$ expensive operations.
- **Step 4: Evaluate the Series** We need to sum the costs of these specific operations. Since the cost of operation i is i , and $i = k^2$, the cost is k^2 . Spike Cost $= \sum_{k=1}^{\lfloor \sqrt{n} \rfloor} k^2$ We use the standard mathematical formula for the sum of consecutive squares: $\sum_{k=1}^m k^2 = \frac{m(m+1)(2m+1)}{6}$ Substitute $m = \sqrt{n}$: Spike Cost $= \frac{\sqrt{n}(\sqrt{n}+1)(2\sqrt{n}+1)}{6}$ When expanding this polynomial, the highest order term is $\frac{(\sqrt{n})(\sqrt{n})(2\sqrt{n})}{6} = \frac{2n\sqrt{n}}{6} = \frac{n^{1.5}}{3}$. Therefore, the sum of the spikes is strictly bounded by $O(n^{1.5})$.
- **Step 5: Calculate the Amortized Average** Substitute the spike bound back into the total cost equation: $T(n) = n + O(n^{1.5}) = O(n^{1.5})$ The total time to execute n operations is $O(n^{1.5})$. To find the amortized cost per operation, we divide the total cost by n : Amortized Cost $= \frac{T(n)}{n} = \frac{O(n^{1.5})}{n} = O(n^{0.5}) = O(\sqrt{n})$

Conclusion: Unlike exponential spikes which yield an $O(1)$ amortized cost, polynomial spikes occur too frequently. The amortized cost of this operation degrades as the data structure grows, averaging $O(\sqrt{n})$ time per operation.

json

```
{
  "component": "LlmGeneratedComponent",
  "props": {
    "height": "600px",
    "prompt": "Objective: Visualize Aggregate Analysis where polynomial cost spikes cause a non-constant O(sqrt(n)) amortized cost. Data State: Cost function is c_i = i if i is a perfect square, else 1. Strategy: Standard Layout. Inputs: 'Next Operation' button, 'Jump +10' button, 'Reset' button. Behavior: Render a dynamic bar chart (x-axis: operation index i, y-axis: cost). Plot bars mostly of height 1, with massive spikes at perfect squares (1, 4, 9, 16, 25, 36...). Overlay a smooth, distinctly colored line chart representing the 'Running Average Amortized Cost' (Total Cost / i). Display prominent numerical counters for Total Operations (n), Cumulative Cost (T(n)), and Current Amortized Cost. Crucially, the visual must demonstrate that the average line continually rises, forming a visible square-root curve, proving it fails to asymptote to a flat O(1) constant.",
    "id": "im_fdf726a28ff68017"
  }
}
```

4. Explore Variations

Variation Question: Consider an initially zero, k -bit binary counter. You perform a sequence of n consecutive **INCREMENT** operations (adding 1 to the counter). The true cost of each increment operation is exactly the number of bits that flip from 0 to 1, or from 1 to 0. Use aggregate analysis to prove that the amortized cost per increment operation is exactly $O(1)$.

Why the process differs significantly: In the previous problems, a single operation occasionally cost a massive amount based purely on its index i , while all other operations cost 1.

In a binary counter, there is no single massive spike. Instead, every single operation has a slightly different cost depending on the internal cascading state of the bits (e.g., going from **011** to **100**

costs 3 bit flips). You cannot solve this by separating a "baseline" from "spikes" across the horizontal axis of time.

The Altered Process:

1. Instead of summing the costs row by row (summing the cost of operation 1, then operation 2, etc.), you must mathematically pivot the summation to calculate **column by column** (summing the total flips of bit 0, then the total flips of bit 1, etc.).
2. You analyze the frequency of each specific bit position:
 - Bit 0 flips on every *single* increment. (Total flips: n)
 - Bit 1 flips on every *second* increment. (Total flips: $\lfloor n/2 \rfloor$)
 - Bit 2 flips on every *fourth* increment. (Total flips: $\lfloor n/4 \rfloor$)
3. The aggregate total cost $T(n)$ becomes the sum of the flips across all bits over the entire sequence: $T(n) = \sum_{j=0}^{\lfloor \log_2 n \rfloor} \frac{n}{2^j}$
4. You pull n out of the summation, leaving a decaying geometric series: $n \times (1 + \frac{1}{2} + \frac{1}{4} + \dots)$.
5. Because the infinite sum of $(1/2)^j$ converges strictly to 2, the total cost $T(n)$ is bounded by $2n$.
6. Finally, dividing by n ($2n/n = 2$) proves that despite cascading bit flips, the amortized cost per increment is bounded by a constant, $O(1)$.

2026-05-27 18:24:28 You Asked 

(b) An implementation of radix sort uses heap sort instead of count sort as the intermediate sorting technique. Is radix sort still stable? Justify your answer with an example. (8)

 Gemini 3.1 Pro Extended

Here is the analysis and solution for the sorting algorithm stability problem presented in the image, strictly adhering to your four-step methodology.

1. Define the Topic

Topic: Sorting Algorithms (Specifically: Algorithmic Stability in Composite Sorts)

Algorithmic stability is a property of sorting algorithms. A sorting algorithm is considered *stable* if two objects with equal keys appear in the same relative order in the sorted output as they did in the unsorted input.

LSD (Least Significant Digit) Radix Sort is a composite algorithm. It does not actually compare the full numbers itself; instead, it relies on a "sub-routine" sorting algorithm to sort the data digit by digit, starting from the rightmost digit and moving left. For LSD Radix Sort to function correctly and produce a fully sorted array at the end, it is a strict mathematical requirement that its intermediate sub-routine algorithm is stable.

2. Steps to Solve this Type of Question

Whenever you are asked to evaluate the viability or properties of a modified composite sorting algorithm, follow these logical steps:

1. **Identify the Sub-routine:** Determine the intermediate sorting algorithm being proposed (in this case, Heap Sort).
2. **Evaluate Sub-routine Stability:** Recall or deduce whether that specific sub-routine is inherently stable or unstable.
3. **Evaluate the Overarching Dependency:** Determine if the overarching algorithm (Radix Sort) fundamentally relies on stability to achieve its final goal.
4. **Construct a Counter-Example:** If the modification breaks the algorithm, construct a minimalist example. Choose a small array of two-digit numbers where a tie in the most significant digit exposes the flaw caused by the unstable intermediate sort.

3. Demonstrate the Process

Let's apply this directly to the question from the image.

- **Step 1 & 2 (Evaluate Heap Sort):** The proposed intermediate sort is Heap Sort. Heap Sort is an inherently **unstable** algorithm. The core mechanics of Heap Sort—building a max-heap and repeatedly swapping the root with the last element of the tree—inevitably swap elements across large distances, destroying the relative order of identical keys.
- **Step 3 (Evaluate Dependency):** LSD Radix Sort *strictly requires* a stable intermediate sort. When it sorts the 10s place, it encounters ties (e.g., the numbers 21 and 23 both have a '2' in the 10s place). It relies entirely on the fact that those numbers were already correctly ordered by the 1s place in the previous pass. If the intermediate sort is unstable, it will scramble those ties, destroying the work of the previous passes.
- **Step 4 (The Counter-Example):** Let's trace Radix Sort using an unstable intermediate sort on the array: `A = [23, 13, 21]`
 - **Pass 1 (Sort by the 1s digit):**
 - The keys are `3`, `3`, and `1`.

- The array is sorted based on these keys: [21, 23, 13] . (Assume for this step, it successfully kept 23 and 13 in their original relative order).
- **Pass 2 (Sort by the 10s digit):**
 - The array is currently [21, 23, 13] .
 - The keys for this pass are 2 , 2 , and 1 .
 - An unstable sort like Heap Sort processes the identical keys (2 and 2) and makes no guarantee about keeping them in order. It is highly likely to swap them during its heapification process.
 - If the unstable sort swaps the two elements tied with the key '2', the resulting array becomes: [13, 23, 21] .

Conclusion: No, the Radix Sort is **not** stable. Furthermore, because it uses an unstable intermediate algorithm, it completely fails to sort the array properly. The final output [13, 23, 21] is numerically incorrect.

json

```
{
  "component": "LlmGeneratedComponent",
  "props": {
    "height": "600px",
    "prompt": "Objective: Visualize why LSD Radix Sort strictly requires a stable intermediate sorting algorithm to function. Data State: Initial array is [32, 12, 31]. Strategy: Standard Layout. Inputs: A toggle switch between 'Use Stable Sub-sort (Counting Sort)' and 'Use Unstable Sub-sort (Heap Sort)', and a 'Next Pass' button. Behavior: Render the array of numbers as distinct visual blocks. Clicking 'Next Pass' animates the sorting process digit-by-digit. First pass sorts by the 1s digit (producing [31, 32, 12]). The second pass sorts by the 10s digit. If the 'Stable' toggle is active, visually ensure that when sorting the '3's (from 31 and 32), the 31 remains safely ahead of the 32, resulting in a correctly sorted [12, 31, 32]. If the 'Unstable' toggle is active, trigger an animation during the 10s pass that visibly scrambles the tied '3's, resulting in an incorrectly sorted final array of [12, 32, 31]. Display a prominent status banner concluding whether the final array is 'Sorted Correctly' or 'Sorting Failed'."
  },
  "id": "im_1f66264706d13d42"
}
```

4. Explore Variations

Variation Question: *Instead of standard LSD (Least Significant Digit) Radix Sort, suppose an implementation of MSD (Most Significant Digit) Radix Sort uses an unstable sorting algorithm like Quick Sort as its intermediate technique. Will the final numerical output be correctly sorted? Is the overall algorithm considered stable?*

Why the process differs significantly: LSD Radix Sort processes the entire array as a single block over and over again, moving from right to left. Because it processes the whole array simultaneously, a tie in a higher digit *must* respect the order established by a lower digit.

MSD Radix Sort operates entirely differently. It sorts from left to right, but it uses a **Divide and Conquer** approach. After sorting the first digit, it physically divides the array into isolated "buckets" (e.g., all numbers starting with '1' go into a bucket, all starting with '2' go into a separate bucket). The algorithm then recursively sorts *inside* those independent buckets.

The Altered Process:

1. **Evaluate Dependency:** Because MSD Radix Sort isolates prefixes into independent buckets, a tie in the most significant digit simply means those numbers are placed into the exact same bucket.
2. In the next recursive pass, the algorithm will sort *only* the contents of that specific bucket based on the next digit. It does not rely on a previous pass to maintain order; it relies on isolating identical prefixes so they can be explicitly sorted in the next step.
3. **Evaluate Numerical Correctness:** Because of this bucket isolation, using an unstable intermediate sort (like Quick Sort) will **still result in a numerically correct sorted array**.
4. **Evaluate Overall Stability:** However, if the array contains identical objects (e.g., two exact copies of the number 45), the unstable intermediate sort will scramble their relative order inside the final bucket. Therefore, while the final output is numerically correct, the overall MSD Radix Sort algorithm **loses its algorithmic stability**.

2026-05-27 18:27:17 You Asked 

(b) Write an efficient algorithm to check if a given undirected graph has a cycle. Discuss the time complexity of your algorithm. (8)

 Gemini 3.1 Pro Extended

Here is the analysis and algorithmic solution for detecting a cycle in an undirected graph, strictly following your four-step methodology.

1. Define the Topic

Topic: Graph Theory (Cycle Detection in Undirected Graphs)

A **cycle** in a graph is a path of edges and vertices wherein a vertex is reachable from itself, meaning the path starts and ends at the exact same node without traversing any edge or vertex twice (other than the start/end node).

In an *undirected* graph, edges are bidirectional. The most efficient way to detect a cycle is by using a graph traversal algorithm like **Depth-First Search (DFS)**. The core logic relies on the concept of a

"back-edge". If, during our traversal, we encounter a node that has already been visited, and that node is *not* the immediate parent we just came from, it guarantees that a secondary path exists to reach that node. That secondary path forms a cycle.

2. Steps to Solve this Type of Question

To write an efficient cycle detection algorithm for an undirected graph using DFS, follow these steps:

1. **Initialize Tracking Data:** Create a `visited` array (or hash set) to keep track of which nodes have already been processed to prevent infinite loops.
2. **Handle Disconnected Components:** A graph might be split into several isolated subgraphs. You must iterate through all vertices. If a vertex is unvisited, launch a DFS from it.
3. **Define the DFS Function:** The recursive function must accept the `current` node, its `parent` node, the `visited` array, and the graph structure.
4. **Execute the Traversal:**
 - * Mark the `current` node as visited.
 - Iterate through all adjacent neighbors of the `current` node.
 - If a neighbor is *unvisited*, recursively call DFS on it, passing `current` as the new parent.
 - If a neighbor *is* visited AND the neighbor is *not* the `parent` node, a back-edge has been found. Return `True` immediately.
5. **Calculate Complexity:** Evaluate the time and space complexity based on the underlying graph representation (usually an Adjacency List).

3. Demonstrate the Process

Let's write out the algorithm and analyze its complexity.

The Algorithm (Pseudocode):

text

```
Algorithm HasCycle(Graph, V)
    // V is the total number of vertices
    Create an array Visited[0...V-1] initialized to False

    // Step 2: Handle disconnected graphs
    for i = 0 to V - 1 do:
        if Visited[i] == False:
            // Start DFS with -1 as the parent of the starting node
            if DFS(Graph, i, -1, Visited) == True:
                return True

    return False

Function DFS(Graph, current, parent, Visited)
```

```

Visited[current] = True

// Step 4: Check all neighbors
for each neighbor in Graph.AdjList[current] do:

    if Visited[neighbor] == False:
        // Recursive call
        if DFS(Graph, neighbor, current, Visited) == True:
            return True

// The core cycle condition: visited AND not the parent
else if neighbor != parent:
    return True

return False

```

Complexity Analysis:

- **Time Complexity:** $O(V + E)$. In the worst-case scenario using an Adjacency List representation, the algorithm will visit every single vertex (V) exactly once and traverse every single edge (E) exactly twice (once from each end). Therefore, the time taken scales linearly with the size of the graph.
- **Space Complexity:** $O(V)$. The `Visited` array requires $O(V)$ space. Additionally, in the worst-case scenario (a graph that is just one long straight line), the recursion stack for DFS will go V levels deep, requiring $O(V)$ auxiliary stack space.

json

```

{"component": "LlmGeneratedComponent", "props": {"height": "700px", "prompt": "Objective: Visualize the DFS algorithm for detecting a cycle in an undirected graph. Data State: Graph with nodes 0 to 4, edges: (0,1), (1,2), (2,3), (3,1), (0,4) ensuring a cycle 1-2-3-1 exists. Strategy: Standard Layout. Inputs: 'Next Step' button, 'Reset' button. Behavior: Render the graph nodes and edges visually. Initially, all nodes are unvisited (default color). As 'Next Step' is clicked, step through the DFS algorithm. Highlight the 'current' node, visually mark 'visited' nodes, and visually indicate the 'parent' node of the current node. When evaluating a neighbor that is already visited AND is not the parent (specifically, when DFS reaches node 1 from node 3), explicitly highlight this 'back-edge' in red, display a prominent 'Cycle Detected!' message, and stop the traversal.", "id": "im_37df3f9a3b59c4ef"}}

```

4. Explore Variations

Variation Question: Write an efficient algorithm to check if a given **directed** graph has a cycle.

Why the process differs significantly: In an undirected graph, an edge $A \rightarrow B$ implies $B \rightarrow A$. This is why we absolutely must check `neighbor != parent` to avoid falsely identifying the

bidirectional edge as a cycle.

In a *directed* graph, edges are one-way. You cannot simply check if a neighbor is visited. A cross-edge to a node that was fully processed in a completely different, independent branch does *not* form a cycle. A cycle in a directed graph only exists if you encounter a back-edge pointing to an ancestor currently residing in the active recursion stack (meaning you looped back onto your current path).

The Altered Process:

1. A single boolean `Visited` array is no longer sufficient. You must maintain two arrays: `Visited` (to mark if a node has ever been processed) and `RecursionStack` (to track nodes currently in the active DFS path).
 - *Alternatively, you can use a single state array with three states: 0 (Unvisited), 1 (Visiting / In Stack), and 2 (Fully Visited).*
2. When visiting a node, mark it as both `Visited` and add it to the `RecursionStack`.
3. Traverse all directed neighbors.
4. If a neighbor is unvisited, recurse into it.
5. If a neighbor is already marked in the `RecursionStack`, you have looped back into your active path. A cycle is detected. Return `True`.
6. Once all neighbors of a node are fully processed, **remove** the current node from the `RecursionStack` (or set its state to Fully Visited) before returning. This prevents false positives from cross-edges.
7. The time complexity remains $O(V + E)$, but the structural logic shifts from "avoiding the immediate parent" to "tracking the active ancestral lineage."

2026-05-27 18:29:32 You Asked 

7. (a) Given a set of n numbers, write an algorithm to find the maximum and minimum element using divide and conquer strategy. Also, determine the time complexity. (7)

 Gemini 3.1 Pro Extended

Here is the analysis and algorithmic solution for finding the maximum and minimum elements using the Divide and Conquer strategy, strictly following your four-step methodology.

1. Define the Topic

Topic: Divide and Conquer (Simultaneous Min-Max Finding)

Divide and Conquer is an algorithm design paradigm that recursively breaks down a problem into two or more sub-problems of the same or related type, until these become simple enough to be solved directly. The solutions to the sub-problems are then combined to give a solution to the original problem.

When finding *both* the minimum and maximum of an array, a naive linear scan requires checking every element against a running minimum and maximum, costing roughly $2n$ comparisons. The Divide and Conquer approach optimizes this by pairing elements and bounding the extremes structurally, reducing the total number of comparisons to exactly $\frac{3n}{2} - 2$.

2. Steps to Solve this Type of Question

To write an algorithm using the Divide and Conquer strategy, you must always define three distinct phases:

1. **Divide (The Split):** Find the middle index of the current array segment and recursively divide the array into a left half and a right half.
2. **Base Cases (The Conquer):** You must define the smallest possible sub-problems that can be solved without further recursion. For Min-Max finding, there are two:
 - **Size 1:** If the subarray has only 1 element, that single element is inherently both the minimum and the maximum (0 comparisons).
 - **Size 2:** If the subarray has 2 elements, compare them to each other. The smaller is the minimum, and the larger is the maximum (1 comparison).
3. **Combine (The Merge):** Once the recursive calls return the (min, max) pairs from the left and right halves, compare the two minimums to find the absolute minimum, and compare the two maximums to find the absolute maximum (2 comparisons).

3. Demonstrate the Process

Let's write the algorithm based on the steps above and determine its exact time complexity.

The Algorithm (Pseudocode):

text

```
Algorithm FindMinMax(A, low, high)
    // A is the array, low is starting index, high is ending index

    // Base Case 1: Only 1 element
    if low == high:
        return (A[low], A[low])

    // Base Case 2: Exactly 2 elements
```



```

if high == low + 1:
    if A[low] < A[high]:
        return (A[low], A[high])
    else:
        return (A[high], A[low])

// Divide Phase
mid = integer division (low + high) / 2

(min_left, max_left) = FindMinMax(A, low, mid)
(min_right, max_right) = FindMinMax(A, mid + 1, high)

// Combine Phase
if min_left < min_right:
    min_final = min_left
else:
    min_final = min_right

if max_left > max_right:
    max_final = max_left
else:
    max_final = max_right

return (min_final, max_final)

```

Time Complexity Analysis:

Let $T(n)$ represent the number of comparisons made to process n elements.

- The divide step takes $O(1)$ time.
- The combine step requires exactly 2 comparisons.
- The problem is divided into two sub-problems of size $n/2$.

This gives us the recurrence relation (assuming n is a power of 2): $T(n) = 2T(n/2) + 2$ With base cases: $T(1) = 0$ and $T(2) = 1$.

We can solve this using the substitution method. $T(n) = 2[2T(n/4) + 2] + 2 = 4T(n/4) + 4 + 2$
 $T(n) = 4[2T(n/8) + 2] + 4 + 2 = 8T(n/8) + 8 + 4 + 2$ Generalizing to k steps where $n = 2^k$ (so $k = \log_2 n$): $T(n) = 2^{k-1}T(n/2^{k-1}) + \sum_{i=1}^{k-1} 2^i$ We stop when the sub-problem size reaches 2 (our base case). The size is $n/2^{k-1} = 2$, and $T(2) = 1$. $T(n) = 2^{k-1}(1) + (2^k - 2)$ Substituting $2^k = n$: $T(n) = \frac{n}{2} + n - 2 = \frac{3n}{2} - 2$

Because the number of operations scales strictly linearly with the input size, the asymptotic worst-case time complexity is $O(n)$.

json

```
{"component": "LlmGeneratedComponent", "props": {"height": "700px", "prompt": "Objective: Visualize the Divide and Conquer strategy for finding Min and Max. Data State: An array of 8 random unsorted integers. Strategy: Standard Layout. Inputs: 'Next Step' button, 'Reset' button. Behavior: Render the array visually. Clicking 'Next Step' animates the algorithm's progression. First, visually split the array in half downwards to form a binary recursion tree, continuing until leaf nodes of size 2 are reached. Once at the leaf nodes, highlight the local comparison and label the local (min, max) pair. Then, animate the 'Combine' phase, moving back up the tree, showing the two minimums competing and the two maximums competing, until the final overarching (min, max) pair is consolidated at the root.", "id": "im_cdf0840c1a965914"}}
```

4. Explore Variations

Variation Question: Write an algorithm to find the **Majority Element** (an element that appears strictly more than $\lfloor n/2 \rfloor$ times) in an array using the Divide and Conquer strategy. Discuss how its mathematical recurrence and final time complexity differ from the Min-Max algorithm.

Why the process differs significantly: Finding extremes (min/max) is a purely numerical bounding exercise. A local maximum can easily be compared against another local maximum. However, a majority element is based on global frequency. If you split an array in half, a global majority element is mathematically guaranteed to be the local majority element in at least *one* of the two halves. But unlike min/max, you cannot simply compare the two local majorities to each other in $O(1)$ time to find the winner.

The Altered Process:

1. **Divide:** Split the array in half.
2. **Conquer:** Recursively find the local majority candidate for the left half and the right half.
3. **Combine (The major difference):** * If the left candidate and right candidate are the *same*, that element is the combined majority.
 - If they *differ*, you do not know which one is the true global majority. You are forced to iterate through the **entire current subarray** (size n) and explicitly count the occurrences of both candidates to see which one crosses the necessary threshold.
4. **Complexity Shift:** Because the combine step now requires scanning the array elements, it takes $O(n)$ time instead of the constant 2 comparisons used in the Min-Max problem.
5. **New Recurrence:** The relation becomes: $T(n) = 2T(n/2) + O(n)$
6. Applying Case 2 of the Master Theorem, this recurrence resolves to a time complexity of $O(n \log_2 n)$. The shift from an $O(1)$ merge step to an $O(n)$ merge step fundamentally downgrades the overall efficiency of the algorithm compared to Min-Max.



Variation Question: Write an efficient algorithm to check if a given directed graph has a cycle.

 Gemini 3.1 Pro Extended

Here is the formal analysis and structural implementation for detecting a cycle within a directed graph, strictly following the four-step methodology.

1. Define the Topic

Topic: Graph Theory (Cycle Detection in Directed Graphs via Back-Edge Tracing)

Detecting a cycle in a **directed graph** requires a fundamentally different structural logic than in an undirected graph. In an undirected graph, edges are bidirectional, so we only need to ensure we don't immediately step backward onto the node we just came from (our parent).

In a directed graph, edges have a strict one-way orientation. Simply encountering a node that has already been visited does not mean a cycle exists. It could be a *cross-edge*, which occurs when two completely independent paths happen to converge on the same downstream vertex. A cycle is strictly defined as a path that loops back onto an **active ancestor**—a node that we have entered but have not yet finished exploring. In graph theory, this path is called a **back-edge**, and we detect it by tracking the active recursion stack of our Depth-First Search (DFS).

2. Steps to Solve this Type of Question

To design a cycle detection algorithm for a directed graph, you must manage three distinct execution states for every vertex. This is traditionally called the **Three-Color Graph Coloring Marking Method**:

1. **State 0 (Unvisited)**: The vertex has not yet been encountered by the search algorithm.
2. **State 1 (Visiting / In Stack)**: The vertex has been entered, and its neighbors are currently being recursively explored. This vertex is an active ancestor in the current execution path.
3. **State 2 (Fully Visited / Processed)**: The vertex and all of its downstream paths have been completely explored. It is removed from the active path.

The Execution Flow:

1. Initialize a state array of size V filled with 0 (Unvisited).
2. Loop through all vertices from 0 to $V - 1$ to account for disconnected subgraphs. If a vertex is at state 0, initiate the recursive DFS tracking.
3. Upon entering a node in DFS, instantly flip its state from 0 to 1.

4. Iterate through all directed outgoing neighbors of that node:
 - If a neighbor is at state 0 , recursively call DFS on it.
 - If a neighbor is at state 1 , a **back-edge** is detected! This means the current path has collided with an active ancestor. Immediately halt execution and return True .
 - If a neighbor is at state 2 , ignore it completely. This is a safe cross-edge.
5. After checking all neighbors of the current node, change its state from 1 to 2 before backtracking out of the function.

3. Demonstrate the Process

Let's write the explicit algorithm and analyze its complexity.

The Algorithm (Pseudocode):

text

```
Algorithm HasDirectedCycle(Graph, V)
    // State representation: 0 = Unvisited, 1 = In Stack, 2 = Fully Visited
    Create an array State[0...V-1] initialized to 0

    // Step 2: Traverse all nodes to handle disconnected components
    for i = 0 to V - 1 do:
        if State[i] == 0:
            if DirectedDFS(Graph, i, State) == True:
                return True

    return False

Function DirectedDFS(Graph, current, State)
    // Step 3: Mark node as actively visiting (In Stack)
    State[current] = 1

    // Step 4: Explore outgoing directed edges
    for each neighbor in Graph.AdjList[current] do:

        if State[neighbor] == 0:
            // Recurse into unvisited node
            if DirectedDFS(Graph, neighbor, State) == True:
                return True

        else if State[neighbor] == 1:
            // Back-edge found: neighbor is an active ancestor!
            return True

    // If State[neighbor] == 2, it's a cross-edge. We do nothing.
```

```
// Step 5: Node is fully processed. Remove from stack.  
State[current] = 2  
return False
```

Complexity Analysis:

- **Time Complexity:** $O(V + E)$ Every vertex (V) transitions through states 0, 1, and 2 exactly once. Every directed edge (E) is examined exactly once when its source vertex is in state 1. Thus, the time spent scales perfectly linearly with the total structural size of the graph, making it highly efficient.
- **Space Complexity:** $O(V)$ The tracking `State` array takes $O(V)$ auxiliary space. In the worst-case configuration (a single long linear chain of vertices), the recursive execution stack will reach a max depth of V layers, requiring $O(V)$ space.

json

```
{  
  "component": "LlmGeneratedComponent",  
  "props": {  
    "height": "700px",  
    "prompt": "Objective: Visualize the Three-Color DFS algorithm detecting a cycle in a directed graph vs identifying a safe cross-edge. Data State: A directed graph with 4 nodes. Edges are: (0->1), (1->2), (2->0) [forming a cycle], and an independent branch (0->3) and (3->2) [forming a cross-edge]. Strategy: Standard Layout. Inputs: 'Next Step' button, 'Reset' button. Behavior: Render the directed graph with clear directional arrowheads on the edges. Color code the nodes dynamically based on their state: Unvisited, Visiting (In Stack), and Fully Visited. Step through the execution with the 'Next Step' button. First, trace the cross-edge path (0->3->2). When node 2 finishes and backtracks, show its color shifting to 'Fully Visited'. Then, trace the loop path (0->1->2). Crucially, show that when node 1 points to node 2 (Fully Visited), it safely bypasses it as a cross-edge. But when the traversal moves to node 2 and points back to node 0, detect that node 0 is still 'Visiting' (In Stack). Visually highlight this edge as a 'Back-Edge', freeze the animation, and display a 'Directed Cycle Detected!' alert.",  
    "id": "im_2b292e7b5c5018e4"  
  }  
}
```

4. Explore Variations

Variation Question: Suppose you are given a directed graph representing a software build system, where vertices represent code modules and directed edges represent compilation dependencies (e.g., $A \rightarrow B$ means Module A requires Module B to be compiled first). Write an algorithm that checks for cycles and, if **no cycle is found**, outputs a valid, flat linear ordering sequence in which the modules can be safely compiled.

Why the process differs significantly: The core algorithm we just demonstrated answers a binary question: `True` (there is a cycle) or `False` (there is no cycle). It stops immediately upon hitting the first back-edge.

This variation asks for a **Topological Sort** of a Directed Acyclic Graph (DAG). You can no longer just stop and exit when a path finishes exploring; you must systematically construct a complete, ordered

linear list of every single vertex in the graph.

The Altered Process:

1. You retain the exact same three-state (0 , 1 , 2) cycle detection tracking framework to ensure the dependency graph is valid. If at any point a state 1 collision occurs, you immediately throw an error stating "Compilation failed due to a circular dependency loop."
2. **The Structural Addition:** You initialize an empty linear **Stack** (or list) at the beginning of the algorithm.
3. You execute the DFS traversal exactly as before. However, the crucial modification happens at **Step 5** (the backtracking phase).
4. The exact moment a vertex transitions from state 1 to state 2 (meaning all of its downstream dependencies have been completely cleared and accounted for), you **push** that vertex onto your linear Stack.
5. After the global outer loop finishes processing all disconnected parts of the graph, you pop the elements out of the stack one by one.
6. The resulting sequence is the final Topological Order. By shifting the goal from "halting at the first failure" to "scheduling the collection during the post-order backtracking phase," the algorithm transforms from a simple validator into a linear ordering compiler.

2026-05-27 18:31:42 You Asked 

Variation Question: Write an algorithm to find the Majority Element (an element that appears strictly more than $\lfloor n/2 \rfloor$ times) in an array using the Divide and Conquer strategy. Discuss how its mathematical recurrence and final time complexity differ from the Min-Max algorithm.

 Gemini 3.1 Pro Extended

Here is the analysis and formal solution for finding the Majority Element using the Divide and Conquer strategy, following the structured four-step methodology.

1. Define the Topic

Topic: Divide and Conquer (Frequency Optimization and Pigeonhole Pruning)

The **Majority Element Problem** involves identifying an element within an array of size n that appears strictly more than $\lfloor n/2 \rfloor$ times.

While simpler linear-time scans exist for this problem (such as Boyer-Moore Voting), solving it via **Divide and Conquer** leverages a fundamental structural principle: **If a majority element exists in the global array, it must also be the local majority element in at least one of its two halves.** By breaking the array down recursively into single elements, the algorithm isolates candidates. However, unlike simpler reduction problems (such as finding the minimum and maximum elements), merging two subproblems to find a majority element cannot be done in constant $O(1)$ time. Instead, it requires validating frequencies across the active subarray segments.

2. Steps to Solve this Type of Question

When designing a Divide and Conquer approach to solve a frequency-dependent threshold problem, execute the following phases:

1. **Divide (The Bisection):** Find the midpoint of the current array boundaries and recursively partition the search space into a left half and a right half.
2. **Base Case (The Conquering Threshold):** Define the absolute smallest subproblem. If a subarray segment contains only a single element, that element is inherently the undisputed majority element of its own size-1 domain. Return it as a candidate.
3. **Combine (The Cross-Validation Scan):** This is the core logical divergence. Receive the two candidate elements returned by the left and right recursive bisections:
 - If the left candidate matches the right candidate, that element safely remains the undisputed local majority candidate. Pass it up.
 - If the two candidates conflict, you cannot determine the victor in constant time. You must explicitly execute a **linear-time counting sweep** across the *entire scope* of the current active subarray to determine which of the two competing candidates possesses the actual highest frequency.
4. **Threshold Validation:** At the top-most level of execution, perform one final check to verify if the absolute winning candidate's global frequency strictly exceeds $\lfloor n/2 \rfloor$ (to handle cases where no true majority element exists).

3. Demonstrate the Process

Let's write out the algorithm explicitly and mathematically derive its complexity.

The Algorithm (Pseudocode):

text

```
Algorithm FindMajorityElement(A)
  n = length(A)
  candidate = MajoritySearch(A, 0, n - 1)

  // Step 4: Final verification pass
  count = 0
```

```

for i = 0 to n - 1 do:
    if A[i] == candidate:
        count = count + 1

if count > floor(n / 2):
    return candidate
else:
    return "No Majority Element Exists"

Function MajoritySearch(A, low, high)
    // Step 2: Base Case - single element is its own local majority
    if low == high:
        return A[low]

    // Step 1: Divide
    mid = floor((low + high) / 2)

    left_candidate = MajoritySearch(A, low, mid)
    right_candidate = MajoritySearch(A, mid + 1, high)

    // Step 3: Combine
    // Case A: Candidates agree
    if left_candidate == right_candidate:
        return left_candidate

    // Case B: Candidates conflict. Run a linear count scan over current subarray window
    left_count = 0
    right_count = 0

    for i = low to high do:
        if A[i] == left_candidate:
            left_count = left_count + 1
        else if A[i] == right_candidate:
            right_count = right_count + 1

    if left_count > right_count:
        return left_candidate
    else:
        return right_candidate

```

Time Complexity and Recurrence Discussion (The Structural Comparison)

Let's mathematically examine why the performance of this algorithm differs significantly from the simultaneous Min-Max finding algorithm.

1. **The Min-Max Recurrence:** In the Min-Max algorithm, combining the results from the left and right branches requires exactly 2 scalar comparisons, regardless of how large the array is. This

results in the standard recurrence: $T(n) = 2T(n/2) + 2$. Because the non-recursive work is a fixed constant $O(1)$, applying the Master Theorem yields a linear complexity: $O(n)$.

2. The Majority Element Recurrence: In our Majority Element algorithm, the combine phase (Case B) requires a linear loop that scans across every element from low to high. For a problem size of n , this combine step takes linear work, represented asymptotically as $O(n)$. This results in a fundamentally different mathematical recurrence: $T(n) = 2T(n/2) + O(n)$

3. Solving via Master Theorem ($T(n) = aT(n/b) + f(n)$):

- Here, $a = 2$, $b = 2$, and $f(n) = \Theta(n)$.
- We compute the critical value: $n^{\log_b a} = n^{\log_2 2} = n^1 = n$.
- Because $f(n) = \Theta(n^{\log_b a})$, this falls directly into **Case 2** of the Master Theorem.
- The solution evaluates to: $T(n) = \Theta(n \log_2 n)$.

Conclusion: The shift from a constant-time $O(1)$ merge phase in the Min-Max problem to an item-scanning linear-time $O(n)$ merge phase fundamentally slows down the algorithm, causing its worst-case complexity to drop from a linear $O(n)$ curve to a linearithmic $O(n \log_2 n)$ curve.

json

```
{"component": "LlmGeneratedComponent", "props": {"height": "750px", "prompt": "Objective: Create an educational visualizer tracking the divide-and-conquer processing tree of a Majority Element search. Data State: An un-ordered input array initialized with 8 discrete numeric nodes containing values: [2, 4, 2, 3, 2, 2, 5, 2], ensuring 2 is the absolute global majority. Strategy: Standard Layout. Inputs: 'Next Step' calculation button, 'Reset' button. Behavior: 1. Graphically map out the input values as individual structural boxes arranged in a flat row. 2. As the 'Next Step' control is triggered, visually divide the row downward into a structural tree of left/right bisections until reaching 8 distinct base elements. 3. On the upward 'Combine' pass, demonstrate a split scenario. When merging left candidate '2' and right candidate '4', highlight that they conflict, and animate a horizontal highlight indicating a localized iteration tracking frequencies within that specific window to pick the winner. Show the values merging back up to the root where candidate 2 successfully claims the overall majority spot, accompanied by a final output box demonstrating the calculation.", "id": "im_512feda133602110"}}
```

4. Explore Variations

Variation Question: Consider an array A of size n . Instead of locating a majority element that appears strictly more than $\lfloor n/2 \rfloor$ times, you are tasked with identifying **all** elements that appear strictly more than $\lfloor n/3 \rfloor$ times. Provide a Divide and Conquer strategy to determine these candidates, and detail how its mathematical branch factors alter the recurrence structure.

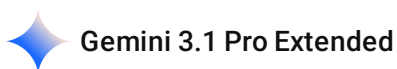
Why the process differs significantly: By changing the fractional visibility threshold from one-half ($\lfloor n/2 \rfloor$) to one-third ($\lfloor n/3 \rfloor$), the structural constraints governing the problem scale dramatically.

According to the mathematical pigeonhole principle, an array can contain at most **one** element with a frequency greater than $n/2$. However, an array can contain up to **two** distinct elements that simultaneously appear more than $n/3$ times (for example, in an array of size 9, two different values can each appear 4 times). This introduces multiplicity to the tracking state.

The Altered Process:

1. **Multi-Candidate Tracking:** The base-case recursive calls can no longer simply return a single integer representing the lone scalar candidate. Every recursive execution level must return a **set of up to two distinct candidates**.
2. **Branching Factor Multiplicity:** When combining the results of the left subproblem and right subproblem, the algorithm receives two candidate sets. It must compute the union of these sets, creating a pool of up to **four unique candidates** that must be validated.
3. **Complex Matrix Validation:** Instead of checking two items against each other in a simple linear loop, the combine phase must now execute a comprehensive checking routine over the current active subarray window to count the frequencies of all four potential candidates.
4. **The Recurrence Profile:** The algorithmic division still cleanly splits the input array size into $n/2$ segments. The combine phase still scales linearly with the window length, keeping the non-recursive overhead bounded to $O(n)$. Thus, the recurrence remains structurally aligned to: $T(n) = 2T(n/2) + O(n)$ While the final runtime complexity remains $O(n \log_2 n)$, the underlying engine is structurally altered because each node in the recursive stack must handle set matrices and evaluate up to four competing paths simultaneously.

2026-05-17 18:32:18 You Asked



Here is the analysis and algorithmic solution for the multi-key string sorting problem presented in the images, strictly following your four-step methodology.

1. Define the Topic

Topic: Non-Comparison Based Sorting (Specifically: LSD Radix Sort for Lexicographical Data)

To sort a dataset in linear time— $O(N)$ —we must abandon standard comparison-based sorting algorithms (like Merge Sort or Quick Sort), as mathematical proofs dictate they cannot perform faster

than $O(N \log N)$ in the worst case.

Instead, we must use a non-comparison algorithm. **Radix Sort**, combined with a stable sub-sorting algorithm like **Counting Sort**, is the perfect tool for this. It sorts data by processing individual characters one by one. Because strings are just arrays of characters, and the maximum length of these strings is a fixed constant (32 for names, 6 for courses), the total number of operations scales linearly with the number of records, strictly achieving $O(N)$ time complexity.

2. Steps to Solve this Type of Question

Whenever you are asked to sort records by multiple string keys (e.g., primary key = Course, secondary key = Teacher) in strictly linear time, follow these steps:

1. **Identify the Keys and Priority:** Determine which field is the most important (Primary) and which is the tie-breaker (Secondary).
2. **Establish the Constant Bounds:** Identify the maximum character length for each field (K) and the alphabet size (Σ).
3. **Pad the Strings:** Radix sort on strings usually requires uniform lengths. Conceptually pad shorter strings with blank spaces at the end (since a blank space has a lower ASCII/lexicographical value than 'A').
4. **Execute LSD Radix Sort (Least Significant Digit):**
 - Sort the records using the *Secondary Key* first. Sort character by character, from the rightmost character down to the leftmost (index 0).
 - Then, sort the records using the *Primary Key*, again from right to left.
5. **Rely on Stability:** Because Counting Sort (the engine inside Radix Sort) is **stable**, sorting the primary key will mathematically preserve the alphabetical order of the secondary key whenever a tie in the primary key occurs.

3. Demonstrate the Process

Let's apply this algorithm to the university department database described in the problem.

- **Given Constraints:** * N records (number of teacher-course assignments).
 - Teacher Name: Max 32 characters. (Secondary Key).
 - Course Code: e.g., BCS101 (assume fixed length of 6 characters). (Primary Key).
 - Alphabet size Σ : 26 uppercase letters + 10 digits + 1 space = 37 possible characters.

The Algorithm:

1. **Preparation:** Read the N records. Pad any teacher name shorter than 32 characters with trailing spaces.
2. **Phase 1: Sort by Teacher Name (Secondary Key)**

- Loop `i` from 31 down to 0 (right-to-left through the 32-character string).
- In each iteration, perform a **Counting Sort** on all N records based *only* on the character at index `i` of the teacher's name.
- *Result of Phase 1:* The entire array of records is now perfectly sorted alphabetically by Teacher Name.

3. Phase 2: Sort by Course Code (Primary Key)

- Loop `j` from 5 down to 0 (right-to-left through the 6-character course code).
- In each iteration, perform a **Counting Sort** on all N records based *only* on the character at index `j` of the course code.
- *Result of Phase 2:* The array is now grouped and sorted by Course Code. Because Counting Sort is stable, tied course codes (e.g., two BCS101 records) will remain in the exact relative alphabetical order established in Phase 1 (e.g., SNEHA will remain before SWAPNIL).

Time Complexity Analysis:

- Each pass of Counting Sort takes $O(N + \Sigma)$ time. Since Σ is a constant 37, this is simply $O(N)$.
- Phase 1 requires 32 passes. Time = $32 \times O(N) = O(N)$.
- Phase 2 requires 6 passes. Time = $6 \times O(N) = O(N)$.
- Total Time = $38 \times O(N)$. Because 38 is a constant factor, the asymptotic worst-case time complexity is strictly $O(N)$, proving it is a linear-time algorithm.

json

```
{
  "component": "LlmGeneratedComponent",
  "props": {
    "height": "600px",
    "prompt": "Objective: Visualize how LSD Radix Sort uses algorithmic stability to sort by multiple string keys in linear time. Data State: A table of records: [BCS101, SWAPNIL], [MCS101, ANIL], [BCS101, SNEHA], [MCA101, BEENA], [BCS102, SNEHA]. Strategy: Standard Layout. Inputs: 'Phase 1: Sort by Teacher Name' button, 'Phase 2: Sort by Course Code' button, 'Reset' button. Behavior: Display the records as animatable rows in a vertical list. Step 1: When 'Phase 1' is clicked, animate the rows rearranging into strict alphabetical order by the Teacher column (ANIL, BEENA, SNEHA, SNEHA, SWAPNIL). Highlight the Teacher column. Step 2: When 'Phase 2' is clicked, animate the rows sorting by the Course column. CRITICALLY, demonstrate algorithmic stability: when grouping the 'BCS101' records, ensure they retain their relative teacher order (SNEHA strictly before SWAPNIL) established in Phase 1. Highlight the stable groupings to emphasize how the final linear-time sort achieves the desired chronological-alphabetical hierarchy.",
    "id": "im_43840c5957cebe6e"
  }
}
```

4. Explore Variations

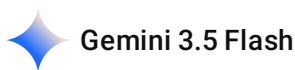
Variation Question: Suppose you are dealing with a massive global database of billions of records. The strings are NOT bounded by a small maximum constant like 32. Instead, string lengths vary wildly from 2 characters to 10,000 characters. However, you know the sum of the lengths of all strings combined is L . Why does standard Radix Sort fail to be efficient here, and what linear-time data structure algorithm must be used instead?

Why the process differs significantly: Standard Radix Sort requires uniform lengths. If you have one massive string of 10,000 characters, you must pad every single other string in the database with thousands of blank spaces to match it. The time complexity becomes $O(N \cdot K_{max})$. If K_{max} is very large and proportional to N , this degrades from a linear sort into an incredibly inefficient $O(N^2)$ algorithm, wasting massive amounts of memory and time sorting empty spaces.

The Altered Process:

1. **Abandon Padding:** You can no longer process the dataset uniformly from right to left.
2. **Use a Trie (Prefix Tree):** Instead of Radix Sort, you must use a structural data approach. You insert every string into a Trie.
3. **Insertion Time:** Inserting a string into a Trie takes time strictly proportional to the length of that specific string. Therefore, inserting all strings takes time proportional to the total number of characters across all strings combined, which is $O(L)$.
4. **Traversal (Sorting):** Once all strings are inserted, you execute a pre-order traversal (Depth-First Search) of the Trie, moving from 'A' to 'Z' at each node.
5. As you traverse, you output the strings when you hit a terminal node. This traversal also takes exactly $O(L)$ time.
6. By shifting from LSD Radix padding to a dynamic **Trie Traversal**, the worst-case time complexity becomes tightly bounded to $O(L)$, restoring true linear-time efficiency proportional only to the actual data present.

2. (a) Professor William claims that the $\Omega(n \lg n)$ lower bound for sorting n numbers does not apply to his machine. The control flow of a program on his machine can split three ways after a single comparison of two elements of the array $a_1 : a_j$. The three ways are $a_i < a_j$, $a_i = a_j$, or $a_i > a_j$. Show that the professor is wrong by proving that the number of three-way comparisons required to sort n elements is $\Omega(n \lg n)$. (5)



Here is the structured breakdown and solution for the question presented in the image.

1. Topic Definition

The core topic of this question is the **Comparison-Based Sorting Lower Bound** analyzed via the **Decision Tree Model** in theoretical computer science.

- **Decision Tree Model:** An abstraction used to represent the execution of a comparison-based sorting algorithm. Each internal node represents a comparison between elements, each edge represents the outcome of that comparison, and each leaf represents a unique final permutation (sorted order) of the input array.
- **Lower Bound (Ω):** The mathematical minimum number of operations required in the worst-case scenario. For a decision tree, the worst-case number of comparisons corresponds directly to the **height of the tree (h)**.

2. Solution Methodology

To find or prove the worst-case lower bound for any sorting algorithm using a decision tree model, follow these fundamental steps:

1. **Determine the Minimum Number of Leaves (L):** Identify how many distinct outcomes (permutations) the algorithm must be capable of reaching to correctly handle all possible inputs. For sorting n distinct elements, there are $n!$ possible permutations, so $L \geq n!$.
2. **Relate Tree Height to Branches and Leaves:** Express the maximum number of leaves a tree of height h can have given its branching factor k (the number of outcomes per comparison). The relationship is: $L \leq k^h$

3. **Set up the Inequality:** Combine the two boundaries to form the core inequality: $n! \leq L \leq k^h \implies k^h \geq n!$
 4. **Solve for Height (h):** Take the logarithm base k of both sides to isolate h : $h \geq \log_k(n!)$
 5. **Apply Stirling's Approximation:** Expand and simplify the factorial term using Stirling's Approximation ($\log(n!) = \Theta(n \log n)$) and change-of-base log rules to extract the asymptotic lower bound notation (Ω).
-

3. Step-by-Step Demonstration

Let's apply the methodology to Professor William's 3-way comparison machine.

Step 1: Identify the required leaves (L)

To correctly sort any arbitrary array of n distinct numbers, the sorting algorithm must be able to produce every possible permutation of the input. $L \geq n!$

Step 2: Establish the branching relationship (k)

The professor's machine executes a 3-way split ($a_i < a_j$, $a_i = a_j$, or $a_i > a_j$). Therefore, the branching factor is $k = 3$. A ternary (3-way) decision tree of height h can have at most 3^h leaves: $L \leq 3^h$

Step 3: Formulate the inequality

Combining the bounds on L : $3^h \geq n!$

Step 4: Isolate the height (h)

Taking the logarithm base 3 of both sides: $h \geq \log_3(n!)$

Using the logarithmic change-of-base formula ($\log_3 x = \frac{\lg x}{\lg 3}$, where $\lg$ is $\log_2$): $h \geq \frac{\lg(n!)}{\lg 3}$

Step 5: Apply Stirling's Approximation

By Stirling's Approximation, we know that $\lg(n!) \geq n \lg n - n \lg e$. Asymptotically, this means $\lg(n!) = \Omega(n \lg n)$.

Substituting this back into our height equation: $h \geq \frac{1}{\lg 3} \cdot (n \lg n - n \lg e)$

Since $\frac{1}{\lg 3}$ is a positive constant (≈ 0.631), it does not affect the asymptotic lower bound. Dropping the lower-order terms yields: $h = \Omega(n \lg n)$

Conclusion: The professor is wrong. Even with a 3-way splitting machine, the worst-case lower bound for sorting remains asymptotically unchanged at $\Omega(n \lg n)$.

4. Structural Variations

Here are two variations from the same topic where the core process changes significantly due to the structural mechanics of the problem.

Variation 1: Determining the Average-Case Lower Bound

- **How the process differs:** Instead of finding the maximum depth of the tree (height h , representing the worst case), you must find the **Average Leaf Depth (External Path Length / L)**.
- **Mechanics:** This requires minimizing the sum of the depths of all leaves in a tree. The process shifts from a simple exponential inequality ($k^h \geq n!$) to analyzing binary tree properties showing that the average depth of a leaf in a tree with L leaves is still bounded below by $\log_2 L$. It relies on properties of entropy and tree balance rather than just maximum height.

Variation 2: Finding a Lower Bound when Permutations are Restricted

- **How the process differs:** In this variation, you are given prior information about the input—for example, sorting an array where it is already known that the elements form a partially sorted sequence, or an array with only e possible valid inversions.
- **Mechanics:** The step to determine L changes completely. Instead of blindly setting $L \geq n!$, you must use combinatorics to calculate the specific, smaller number of reachable target permutations ($L_{restricted}$). You then plug this smaller $L_{restricted}$ value into the logarithmic inequality, which often yields a lower bound tighter than $\Omega(n \lg n)$, such as $\Omega(n \lg(e/n))$.

2026-05-27 19:07:38

You Asked 

(b) Suppose you have an algorithm to find median of n elements of an unsorted array in $O(n)$ time in the worst case. Now consider an implementation of Quicksort where you first find median using the above algorithm, then use median as pivot. What will be the time complexity of this modified

Quicksort? Write down the recurrence relation to justify your time complexity (Median of n elements is the element whose rank is $n/2$ if n is even and it is $(n+1)/2$ if n is odd). (5)

 Gemini 3.1 Pro Extended

Here is the structured breakdown and solution for the problem presented in the images.

1. Topic Definition

The core topic of this question is **Algorithm Analysis**, specifically formulating and solving **Recurrence Relations** to determine the **Time Complexity of Divide-and-Conquer Algorithms**.

- **Divide and Conquer:** An algorithm design paradigm that recursively breaks down a problem into two or more sub-problems of the same or related type, until these become simple enough to be solved directly.
 - **Recurrence Relation:** A mathematical equation that recursively defines a sequence, used in computer science to express the execution time $T(n)$ of a recursive algorithm in terms of the time taken by smaller inputs.
-

2. Solution Methodology

To find the time complexity of a modified divide-and-conquer algorithm like Quicksort, follow this systematic framework:

1. **Analyze the "Divide" Cost:** Determine the time complexity required to select the pivot and partition the array. Let this be $f(n)$.
 2. **Determine Subproblem Sizes:** Based on how the pivot is selected, establish the exact sizes of the recursive calls. For Quicksort, this means identifying how many elements fall to the left and right of the pivot.
 3. **Formulate the Recurrence Relation:** Combine the recursive calls and the divide cost into a standard mathematical equation. For an array of size n , it generally takes the form: $T(n) = T(\text{left_size}) + T(\text{right_size}) + f(n)$
 4. **Solve the Recurrence:** Use an established mathematical method—such as the Master Theorem, the Recursion Tree Method, or the Substitution Method—to find the asymptotic upper bound (O , Ω , or Θ) of $T(n)$.
-

3. Step-by-Step Demonstration

Let's apply this methodology to the modified Quicksort described in your image.

Step 1: Analyze the "Divide" Cost

In standard Quicksort, choosing a pivot takes $O(1)$ time, and partitioning the array around that pivot takes $O(n)$ time. In this **modified Quicksort**, we are given an algorithm that finds the exact median in $O(n)$ time. We use this median as the pivot.

- Cost to find the pivot (median) = $O(n)$
- Cost to partition the array around the pivot = $O(n)$
- Total non-recursive cost $f(n) = O(n) + O(n) = O(n)$.

Step 2: Determine Subproblem Sizes

Because the chosen pivot is the *exact median*, it strictly guarantees that half of the remaining elements are smaller than the pivot and half are larger. Therefore, the array of size n is split into two

perfectly balanced sub-arrays of size $n/2$ (ignoring the single pivot element, which only makes the subproblems slightly smaller, $n/2$ is standard for the asymptotic limit).

Step 3: Formulate the Recurrence Relation

We make two recursive calls to sort the two halves, and it costs $O(n)$ to do the splitting. The recurrence relation is: $T(n) = 2T\left(\frac{n}{2}\right) + O(n)$

Step 4: Solve the Recurrence

We can solve this using the **Master Theorem** for divide-and-conquer recurrences of the form $T(n) = aT(n/b) + \Theta(n^d)$.

- $a = 2$ (number of subproblems)
- $b = 2$ (factor by which the problem size is divided)
- $d = 1$ (exponent of the polynomial cost of dividing/combining)

We compare d with $\log_b a$:

- $\log_b a = \log_2 2 = 1$
- Since $d = \log_b a$ (because $1 = 1$), this falls into **Case 2** of the Master Theorem.

The formula for Case 2 dictates that $T(n) = \Theta(n^{\log_b a} \log n)$. Substituting our values: $T(n) = \Theta(n^1 \log n) = \Theta(n \log n)$

Conclusion: The recurrence relation is $T(n) = 2T(n/2) + O(n)$. By forcing the pivot to be the median, we eliminate the highly unbalanced partitions that cause standard Quicksort's worst-case scenario. The time complexity of this modified Quicksort strictly becomes $O(n \log n)$ **in the worst case**.

4. Structural Variations

Here are two variations from the same topic where the core process of solving the recurrence changes significantly due to the mechanics of the problem.

Variation 1: Unbalanced Guaranteed Splits (e.g., Median of Medians)

- **How the process differs:** Instead of an exact median, suppose the pivot selection algorithm guarantees a split that is no worse than 1-to-4 (e.g., $1/5$ of elements on one side, $4/5$ on the other), while still taking $O(n)$ time.
- **Mechanics:** The recurrence relation changes to $T(n) = T(n/5) + T(4n/5) + O(n)$. You **cannot** use the Master Theorem directly here because the subproblems are of unequal sizes. The solution process shifts to drawing a **Recursion Tree**. You must analyze the shortest branch (depth $\log_5 n$) and the longest branch (depth $\log_{5/4} n$) to prove that the work done at each level

sums to cn , and then use the Substitution Method (induction) to formally prove the $O(n \log n)$ bound.

Variation 2: Dual-Pivot Quicksort

- **How the process differs:** Suppose you spend $O(n)$ time to find two elements that perfectly split the array into three equal thirds, and you partition the array around both pivots simultaneously.
- **Mechanics:** The recurrence relation changes to $T(n) = 3T(n/3) + O(n)$. While the Master Theorem still applies, the constants of the algorithm fundamentally shift. Here, $a = 3$ and $b = 3$. The depth of the recursion tree becomes shallower ($\log_3 n$ instead of $\log_2 n$), which changes the hidden constant factors in practical execution, demonstrating why 3-way/dual-pivot algorithms are often preferred in modern language libraries (like Java's `Arrays.sort`).

2026-05-27 19:10:05 You Asked 

(b) Consider the following recursive relation for 0-1 knapsack problem.

If $w < w_i$, then $OPT(i, w) = OPT(i-1, w)$
else $OPT(i, w) = \max(OPT(i-1, w), v_i + OPT(i-1, ______))$

where $OPT(i, w)$ denote the value of the optimal solution using a subset of items $\{1, 2, \dots, i\}$ with maximum allowed weight w .

v_i is the cost of i th item

Fill the missing value.

What is the running time of the recursive implementation of the above recurrence? Justify. Give memoized recursive algorithm for the above problem. Explain how does it improve the running time? (5)*

 Gemini 3.1 Pro Extended

Here is the structured breakdown and solution for the 0-1 Knapsack problem presented in your image.

1. Topic Definition

The core topic of this question is **Dynamic Programming (DP)**, specifically analyzing the **State Transition Recurrence** and **Top-Down Memoization** as applied to the classic 0-1 Knapsack Problem.

- **Dynamic Programming:** An optimization paradigm that solves complex problems by breaking them down into simpler, overlapping subproblems.

- **0-1 Knapsack:** A combinatorial optimization problem where you must maximize the total value of items placed in a knapsack without exceeding its weight capacity. The "0-1" property means you must either entirely include an item (1) or completely exclude it (0); you cannot take fractions of an item.
-

2. Solution Methodology

To analyze and optimize a recursive relation in Dynamic Programming, follow this framework:

1. **Define the State:** Identify the variables that uniquely describe a subproblem. For the Knapsack problem, the state is defined by the number of items available (i) and the remaining weight capacity (w).
 2. **Determine the State Transition (Choices):** Formulate what happens when a decision is made. For every item, you have two choices:
 - **Exclude:** Move to the next item ($i - 1$) with the same capacity (w).
 - **Include:** Add the item's value (v_i), move to the next item ($i - 1$), and deduct the item's weight (w_i) from the remaining capacity.
 3. **Analyze Unoptimized Time Complexity:** Evaluate the worst-case scenario of the raw recurrence relation by visualizing it as a decision tree. Calculate the maximum depth and branching factor.
 4. **Apply Memoization (Caching):** Create a data structure (usually a 2D array or hash map) to store the results of expensive function calls.
 5. **Evaluate Optimized Time Complexity:** Re-calculate the time complexity based on the total number of *unique* states rather than the total number of recursive branches.
-

3. Step-by-Step Demonstration

Let's apply the methodology to solve the specific questions from your image.

Part 1: Fill the missing value

The recurrence relation handles the scenario where the current item can fit in the knapsack (the `else` condition). We must choose the maximum between excluding the item and including it.

- If we **exclude** it, the value is $\text{OPT}(i - 1, w)$.
- If we **include** it, we gain its value v_i , we move to the remaining items $i - 1$, and we *must subtract its weight from our available capacity*.

Therefore, the missing value is $w - w_i$. The full line becomes:

```
else OPT(i, w) = max(OPT(i-1, w), v_i + OPT(i-1, w - w_i))
```

Part 2: Running time of the recursive implementation

- **Running Time:** $O(2^n)$ in the worst case (where n is the total number of items).
- **Justification:** The recursive function explores all possible combinations of items. At each step i , if the item fits, the algorithm branches into two separate recursive calls (one for inclusion, one for exclusion). This creates a binary recursion tree of depth n . The total number of nodes (and thus function calls) in a full binary tree of depth n is $2^n - 1$, leading to an exponential time complexity of $O(2^n)$.

Part 3: Memoized Recursive Algorithm

To memoize the algorithm, we introduce a 2D table `memo[0..n][0..W]` initialized with `-1` to cache computed states.

text

```
// Global or passed 2D array initialized to -1
memo[n+1][W+1] = {-1}

function OPT_memo(i, w):
    // Base Case: no items left or no capacity left
    if i == 0 or w == 0:
        return 0

    // Check if subproblem has already been solved
    if memo[i][w] != -1:
        return memo[i][w]

    // If item weight is strictly greater than current capacity, exclude it
    if w < w_i:
        memo[i][w] = OPT_memo(i-1, w)

    // Otherwise, find the max of including vs excluding
    else:
        memo[i][w] = max(OPT_memo(i-1, w), v_i + OPT_memo(i-1, w - w_i))

    return memo[i][w]
```

Part 4: Explain how it improves the running time

The raw recursive tree recalculates the exact same states `(i, w)` multiple times (overlapping subproblems). By saving the result of `OPT(i, w)` in `memo[i][w]`, we ensure that any specific combination of items and remaining weight is computed exactly once.

The time complexity drops from the exponential $O(2^n)$ to a pseudo-polynomial $O(n \times W)$, because the algorithm now only needs to fill out a grid of size n (number of items) by W (maximum weight capacity). Each cell takes $O(1)$ time to compute once its prerequisites are known.

4. Structural Variations

Here are two variations of the Knapsack problem where the structural process of finding a solution changes significantly:

Variation 1: Unbounded Knapsack Problem

- **How the process differs:** In the 0-1 knapsack, an item can be picked only once. In the Unbounded Knapsack, an unlimited supply of each item is available.
- **Mechanics:** The state transition changes fundamentally. When you choose to *include* an item, you do not move to the next item ($i - 1$). Instead, you stay on the current item (i) to allow for the possibility of picking it again. The recurrence becomes: $\text{OPT}(i, w) = \max(\text{OPT}(i - 1, w), v_i + \text{OPT}(i, w - w_i))$

Variation 2: Fractional Knapsack Problem

- **How the process differs:** Instead of being restricted to taking whole items, you are allowed to break items apart and take a fraction of them to exactly fill the knapsack.
- **Mechanics:** The Dynamic Programming methodology is abandoned entirely. Because you can take fractions, the optimal sub-structure allows for a **Greedy Algorithm**. The process shifts to calculating the value-to-weight ratio (v_i/w_i) for all items, sorting them in descending order, and iterating through them to fill the knapsack capacity to the brim. The time complexity changes from $O(n \times W)$ to $O(n \log n)$ due to the sorting step.

2026-05-27 19:11:11

You Asked 

4. (a) Give an algorithm to sort n integers in the range 0 to $n^3 - 1$ in $O(n)$ time. Justify its time complexity. (4)

 Gemini 3.1 Pro Extended

Here is the structured breakdown and solution for the sorting algorithm problem presented in the image.

1. Topic Definition

The core topic of this question is **Non-Comparison-Based Sorting Algorithms**, specifically focusing on **Radix Sort** combined with **Base Representation (Number Systems)**.

- **Comparison-Based Limit:** Algorithms that sort by comparing elements against each other (like Merge Sort or Heap Sort) have a strict mathematical lower bound of $\Omega(n \log n)$.
 - **Linear Sorting ($O(n)$):** To sort faster than $O(n \log n)$, we must completely avoid comparing elements directly. Instead, we use the algebraic properties of the numbers themselves (like their individual digits) to route them into sorted order using techniques like Counting Sort or Radix Sort.
-

2. Solution Methodology

To solve problems requiring $O(n)$ sort time for integers bounded by a polynomial range (like 0 to $n^k - 1$), follow this systematic framework:

1. **Acknowledge the Constraint:** Standard Counting Sort on a range of $[0, k]$ takes $O(n + k)$ time. If $k = n^3$, Counting Sort takes $O(n + n^3) = O(n^3)$, which is too slow. We must use Radix Sort.
 2. **Determine the Optimal Base:** To minimize the time complexity of Radix Sort, represent the numbers in a base equal to the number of elements, n .
 3. **Calculate the Number of Digits (d):** Determine how many digits it takes to represent the maximum value in base n . A number X in base n requires roughly $\log_n(X)$ digits.
 4. **Apply Radix Sort:** Sort the numbers digit by digit, from the least significant digit to the most significant digit.
 5. **Use a Stable Sub-routine:** For each digit pass, use a stable sorting algorithm—specifically **Counting Sort**—because it sorts values in the range of the base (which is now n) in linear time.
-

3. Step-by-Step Demonstration

Let's apply this methodology to design the algorithm and justify its complexity.

Algorithm Design

Input: An array of n integers, where each integer x falls in the range $0 \leq x \leq n^3 - 1$.

Step 1: Base Conversion Treat every number in the array as a base- n number. The maximum possible value is $n^3 - 1$. In base n , this maximum value can be expressed with exactly 3 digits.

- Proof: $n^3 - 1 = (n - 1) \cdot n^2 + (n - 1) \cdot n^1 + (n - 1) \cdot n^0$. Therefore, every number in the array can be represented as a 3-digit number $(d_2, d_1, d_0)_n$ where each digit d_i is in the range 0 to $n - 1$.

Step 2: Radix Sort Execution We perform Radix Sort on these 3 digits using Counting Sort as our stable sorting subroutine.

1. Run Counting Sort on the array using the least significant digit (d_0) as the key.
2. Run Counting Sort on the array using the middle digit (d_1) as the key.
3. Run Counting Sort on the array using the most significant digit (d_2) as the key.

After these 3 passes, the array will be completely sorted.

Time Complexity Justification

- **Counting Sort Complexity:** Counting sort takes $O(N + K)$ time, where N is the number of elements and K is the range of the input values.
- **Cost per Pass:** In our algorithm, $N = n$ (the array size). Because we are sorting single digits in base n , the range of any digit is 0 to $n - 1$. Therefore, $K = n$. The time for one pass of Counting Sort is $O(n + n) = O(n)$.
- **Total Passes:** We have exactly $d = 3$ digits to process.
- **Total Time:** $3 \cdot O(n) = O(n)$.

Conclusion: The algorithm runs in strict $O(n)$ time.

4. Structural Variations

Here are two variations from the topic of linear-time sorting where the core process changes significantly due to the mechanics of the input data.

Variation 1: Sorting Uniformly Distributed Floating-Point Numbers

- **How the process differs:** Instead of sorting integers in a large range, suppose you are asked to sort n real numbers (decimals) that are uniformly distributed in the range $[0, 1)$ in expected $O(n)$ time.
- **Mechanics:** Radix sort relies strictly on integer digit extraction and fails here. The process shifts entirely to **Bucket Sort**. You divide the interval $[0, 1)$ into n equal-sized sub-intervals (buckets). You iterate through the array, multiplying each decimal by n and flooring it to route it into the correct bucket. You then sort each individual bucket (often using Insertion Sort, which is fast for tiny lists) and concatenate them. The $O(n)$ complexity relies on probability and expected value analysis, rather than the deterministic bounds of Radix Sort.

Variation 2: Sorting Strings of Highly Variable Lengths

- **How the process differs:** Suppose you need to sort an array of strings in linear time, but the strings range from 1 character to 1000 characters long.
- **Mechanics:** The standard LSD (Least Significant Digit) Radix Sort used above processes from right to left, which requires all items to be padded with blanks to match the longest string. This would waste massive amounts of time on short strings. The process must shift to **MSD (Most**

Significant Digit) Radix Sort. MSD works left to right and recursively groups strings by their prefix. Once a string runs out of characters, it drops out of the recursive routing, making the process highly efficient for variable lengths and changing the implementation from a simple iterative loop to a recursive tree structure.

2026-05-27 19:14:37 You Asked 

Instead of sorting integers in a large range, suppose you are asked to sort n real numbers (decimals) that are uniformly distributed in the range $[0, 1]$ in expected $O(n)$ time.

 Gemini 3.1 Pro Extended

Here is the structured breakdown and solution for sorting uniformly distributed real numbers.

1. Topic Definition

The core topic of this question is **Bucket Sort** (also known as Bin Sort), a specific type of **Distribution-Based Sorting Algorithm**.

- **Comparison Bound Evasion:** Just like Radix Sort, Bucket Sort bypasses the $\Omega(n \log n)$ lower bound of comparison sorts.
- **Distribution Assumption:** Unlike Radix Sort, which relies on the algebraic digit structure of integers, Bucket Sort achieves its $O(n)$ expected time complexity by leveraging a probabilistic assumption: the input data is **uniformly distributed** over a known interval (in this case, $[0, 1]$).

2. Solution Methodology

To sort an array of real numbers uniformly distributed in the range $[0, 1]$ in $O(n)$ time, follow this framework:

1. **Initialize Buckets:** Create an array of n empty lists (buckets). Because the data is uniformly distributed between 0 and 1, we divide this interval into n equal-sized sub-intervals: $[0, 1/n)$, $[1/n, 2/n)$, ..., $[(n-1)/n, 1]$.
2. **Scatter (Distribute):** Iterate through the original array of n elements. For each real number x , determine its target bucket index by multiplying it by n and taking the floor (rounding down): $\text{index} = \lfloor n \cdot x \rfloor$. Insert x into the bucket at this index.

3. **Sort Individual Buckets:** Iterate through the n buckets. If a bucket contains more than one element, sort it. **Insertion Sort** is typically chosen for this step because it operates very efficiently on small, nearly sorted lists.
 4. **Gather (Concatenate):** Iterate through the buckets from index 0 to $n - 1$ and append the elements in order to form the final sorted array.
-

3. Step-by-Step Demonstration

Let's apply this methodology to a small example array of $n = 5$ real numbers: **Input Array:** $A = [0.78, 0.17, 0.39, 0.26, 0.72]$

Step 1: Initialize Buckets Create $n = 5$ empty buckets, representing intervals of size 0.2:

- $B[0]$ for $[0, 0.2)$
- $B[1]$ for $[0.2, 0.4)$
- $B[2]$ for $[0.4, 0.6)$
- $B[3]$ for $[0.6, 0.8)$
- $B[4]$ for $[0.8, 1.0)$

Step 2: Scatter Calculate $\lfloor 5 \cdot x \rfloor$ for each element to find its bucket:

- $0.78 \implies \lfloor 5 \cdot 0.78 \rfloor = \lfloor 3.9 \rfloor = 3 \implies B[3]$
- $0.17 \implies \lfloor 5 \cdot 0.17 \rfloor = \lfloor 0.85 \rfloor = 0 \implies B[0]$
- $0.39 \implies \lfloor 5 \cdot 0.39 \rfloor = \lfloor 1.95 \rfloor = 1 \implies B[1]$
- $0.26 \implies \lfloor 5 \cdot 0.26 \rfloor = \lfloor 1.3 \rfloor = 1 \implies B[1]$
- $0.72 \implies \lfloor 5 \cdot 0.72 \rfloor = \lfloor 3.6 \rfloor = 3 \implies B[3]$

Step 3: Sort Individual Buckets Current state of buckets:

- $B[0] : [0.17] \implies$ Already sorted.
- $B[1] : [0.39, 0.26] \implies$ Use Insertion Sort $\implies [0.26, 0.39]$.
- $B[2] : [] \implies$ Empty.
- $B[3] : [0.78, 0.72] \implies$ Use Insertion Sort $\implies [0.72, 0.78]$.
- $B[4] : [] \implies$ Empty.

Step 4: Gather Concatenate the buckets in order: $B[0] \rightarrow B[1] \rightarrow B[2] \rightarrow B[3] \rightarrow B[4]$. **Result:** $[0.17, 0.26, 0.39, 0.72, 0.78]$

Time Complexity Justification

- Creating buckets takes $O(n)$ time.

- Scattering elements takes $O(n)$ time, as finding the index and inserting takes $O(1)$ per element.
- Because the elements are *uniformly distributed*, the probability of an element falling into any specific bucket is $1/n$. The expected number of elements in any single bucket is 1. Therefore, the expected time to sort a single bucket with Insertion Sort ($O(k^2)$) is $O(1^2) = O(1)$. Sorting all n buckets takes expected $O(n)$ time.
- Gathering takes $O(n)$ time.
- **Total Expected Time:** $O(n)$.

4. Structural Variations

Here are two variations where the core process changes significantly due to the mechanics of the input distribution.

Variation 1: Non-Uniformly Distributed Data (e.g., Clustered Data)

- **How the process differs:** If the inputs are normal (Gaussian) or clustered heavily in a small range, the standard mapping $\lfloor n \cdot x \rfloor$ fails catastrophically. All elements might route into a single bucket, degrading the performance to the $O(n^2)$ time of Insertion Sort.
- **Mechanics:** To maintain $O(n)$ performance, the distribution step must be modified. You cannot use evenly spaced intervals. Instead, you must apply the **Cumulative Distribution Function (CDF)** of the specific data distribution to normalize the data back into a uniform $[0, 1)$ space *before* applying the bucket index formula. Alternatively, you abandon Bucket Sort entirely and fall back to $O(n \log n)$ algorithms.

Variation 2: Sorting 2D Points by Distance from Origin

- **How the process differs:** Suppose you are given n coordinates (x_i, y_i) uniformly distributed *inside a unit circle* (radius $R = 1$), and you must sort them by their distance to the origin in linear time.
- **Mechanics:** The distance $d_i = \sqrt{x_i^2 + y_i^2}$ lies in $[0, 1)$. However, distances are **not** uniformly distributed; there is more area (and thus more points) near the outer edge of the circle than near the center. If you use the standard bucket index $k = \lfloor n \cdot d_i \rfloor$, the outer buckets will overflow. To fix this, you must change the scattering logic to map the points based on area (which grows quadratically). The correct bucket mapping function becomes $\text{index} = \lfloor n \cdot d_i^2 \rfloor$, completely altering the math behind the routing step.

To help visualize how the uniform distribution enables linear time, here is an interactive simulator of the algorithm.

json

```
{
  "component": "LlmGeneratedComponent",
  "props": {
    "height": "700px",
    "prompt": "Create an interactive Bucket Sort visualizer for decimals uniformly distributed between 0 and 1. Use D3.js. Include a 'Generate Data (n=10)' button and a 'Next Step' button to advance through the algorithm states: 1. Unsorted Array, 2. Distribute to Buckets, 3. Sort inside Buckets, 4. Concatenate to Final Array. Visually display the initial array at the top, 10 numbered buckets (0-9) in the middle, and the final sorted array at the bottom. When distributing, animate the numbers moving from the top array into their correct bucket based on the formula index = floor(10 * value). When sorting, visually reorder the items inside any bucket containing multiple elements. When concatenating, animate them moving sequentially out of the buckets into the bottom row. Use distinct visual styles to indicate which elements are currently moving or being compared.",
    "id": "im_4a33f94100053f38"
  }
}
```

2026-05-27 19:19:41

You Asked



(b) Suppose divide and conquer approach is used by an algorithm to solve a problem on the input of size n . The algorithm divides the problem into k number of smaller instances, each of which is $1/b$ the size of the original problem. It solves the problem recursively on these smaller instances and combine their solutions to construct the final

solution of the original problem. Let $G(n)$ denotes the cost of dividing the problem into smaller instances and $F(n)$ denotes the cost of combining the solutions.

Write the recurrence relation to find the running time of the algorithm. Give $G(n)$ and $F(n)$ for Both Quicksort and Mergesort algorithm. (3)



Gemini 3.1 Pro Extended

Here is the structured breakdown and solution for the divide-and-conquer recurrence problem presented in the images.

1. Topic Definition

The core topic of this question is the **Divide-and-Conquer Recurrence Formulation**, which is a mathematical model used in algorithm analysis to express the time complexity of recursive algorithms.

- **Divide and Conquer:** A paradigm that solves a problem by **dividing** it into smaller subproblems, recursively conquering those subproblems, and then **combining** their results.
- **Recurrence Relation:** An equation that defines a sequence recursively. In this context, it expresses the total execution time $T(n)$ as the sum of the time taken by the recursive calls plus the non-recursive overhead (the cost of dividing and combining).

2. Solution Methodology

To formulate the recurrence relation and identify the cost functions for specific algorithms, follow this framework:



1. **Define the Total Time Function:** Let $T(n)$ represent the total running time of the algorithm for an input of size n .
 2. **Model the Recursive Cost:** Multiply the number of smaller instances (subproblems) by the time it takes to solve one of those smaller instances. If there are k instances of size n/b , the recursive cost is $k \cdot T(n/b)$.
 3. **Model the Overhead Cost:** Identify the non-recursive work. Add the mathematical functions that represent the time spent breaking the problem apart, $G(n)$, and merging the solutions back together, $F(n)$.
 4. **Analyze Specific Algorithms:** To find $G(n)$ and $F(n)$ for a specific algorithm (like Quicksort or Mergesort), examine its code execution step-by-step. Ask: "What work happens *before* the recursive calls?" (Divide cost) and "What work happens *after* the recursive calls return?" (Combine cost).
-

3. Step-by-Step Demonstration

Let's apply this methodology directly to your problem.

Part 1: Write the general recurrence relation

Based on the provided variables, the total running time $T(n)$ is the sum of the recursive work, the dividing work, and the combining work. The recurrence relation is: $T(n) = k \cdot T\left(\frac{n}{b}\right) + G(n) + F(n)$

Part 2: $G(n)$ and $F(n)$ for Quicksort

- **Divide Cost $G(n)$:** In Quicksort, the "divide" step is the **partitioning** process. It iterates through the entire current array segment to place elements smaller than the pivot on the left and larger on the right. This takes linear time.
 - $G(n) = \Theta(n)$
- **Combine Cost $F(n)$:** Because the array is sorted in-place during the partition step, once the recursive calls finish sorting the left and right sides, the entire array is already sorted. There is zero extra work required to combine them.
 - $F(n) = \Theta(1)$ (or simply 0)

Part 3: $G(n)$ and $F(n)$ for Mergesort

- **Divide Cost $G(n)$:** In Mergesort, the "divide" step is trivial. It merely calculates the midpoint index of the array to split it into two halves. This takes constant time.
 - $G(n) = \Theta(1)$

- **Combine Cost $F(n)$:** After the recursive calls return two sorted halves, Mergesort must execute the **merge** process, which iterates through both halves to combine them into a single sorted array. This takes linear time.
 - $F(n) = \Theta(n)$
-

4. Structural Variations

Here are two variations from the topic of divide-and-conquer recurrences where the structural process of defining the relation changes significantly.

Variation 1: Algorithms with Unequal Subproblem Sizes

- **How the process differs:** The standard equation $T(n) = aT(n/b) + f(n)$ assumes all recursive subproblems are exactly the same size. If an algorithm splits into unequal parts, you cannot use a single k or b variable.
- **Mechanics:** You must write a recurrence that explicitly separates the recursive terms. For example, the deterministic selection algorithm (Median of Medians) recursively finds the median of a $n/5$ sized array, and then recursively searches a remaining array of maximum size $7n/10$. The process of writing the recurrence shifts to: $T(n) = T\left(\frac{n}{5}\right) + T\left(\frac{7n}{10}\right) + \Theta(n)$ This requires a recursion tree or substitution method to solve, as standard theorems like the Master Theorem no longer apply.

Variation 2: Algorithms Reducing by a Constant Amount

- **How the process differs:** The provided formula $T(n/b)$ dictates that the problem size is divided by a fraction (geometric reduction). Some algorithms reduce the problem size by subtracting a constant number of elements instead.
- **Mechanics:** The recurrence relation structure fundamentally changes from division to subtraction. For instance, in a recursive linear search or computing a factorial, the problem size decreases by 1 at each step. The relation becomes: $T(n) = T(n - c) + O(1)$ The process to solve this shifts from analyzing a logarithmic-depth tree to solving a standard arithmetic progression, resulting in an entirely different complexity class (e.g., $O(n)$ instead of $O(n \log n)$).

Q1	The BFS algorithm has been used to produce the shortest paths from a node s to all other nodes in a graph G . Can Dijkstra's algorithm be used in place of BFS? In a different scenario, Dijkstra's algorithm has been used to produce the shortest paths from a node s to all other nodes in a graph G . Can BFS be used in place of Dijkstra's algorithm? Explain your answers for both scenarios.	6
Q2. a)	What is meant by the cut property of graphs? Explain with an example.	2
b)	A team of explorers is visiting the Sahara desert. As the weather is very hot, they are having n bottles of different sizes to carry water and keep them hydrated. In covering few kilometres, they used all of their water but fortunately found a water source nearby. This water source is having only L litres of water which is way lesser than the capacity of all bottles. They want to fill L litres of water in minimum number of bottles. Describe a greedy algorithm to help them fill L litres of water in minimum number of bottles.	4

Gemini 3.1 Pro Extended

Here are the solutions to the questions presented in the image:

Q1: Interchanging BFS and Dijkstra's Algorithm

Scenario 1: Can Dijkstra's algorithm be used in place of BFS? Yes. Breadth-First Search (BFS) is used to find shortest paths in unweighted graphs (where every edge is implicitly assumed to have a weight of 1). Dijkstra's algorithm is designed for weighted graphs with non-negative edges. If you apply Dijkstra's algorithm to an unweighted graph by assigning a weight of exactly 1 to every edge, it will correctly find the shortest paths, producing the exact same result as BFS.

Note: While logically correct, Dijkstra's algorithm is less efficient for this task because it uses a priority queue, whereas BFS uses a simple FIFO queue.

Scenario 2: Can BFS be used in place of Dijkstra's algorithm? No. Dijkstra's algorithm is necessary when a graph has varying non-negative edge weights. BFS only calculates the shortest path based on the *number of edges* (hops), completely ignoring the weights of those edges. If you use BFS on a weighted graph, it will find the path with the fewest edges, which is not necessarily the path with the lowest total weight.

Example: Imagine traveling from Node A to Node C.

- Path 1 is a direct edge $A \rightarrow C$ with a weight of 50.
- Path 2 goes $A \rightarrow B \rightarrow C$ with weights of 5 and 10 (total weight 15). BFS will incorrectly choose Path 1 because it is only 1 hop, whereas Dijkstra's will correctly identify Path 2 as the shortest path.

Q2. a) The Cut Property of Graphs

Definition: The cut property is a fundamental concept used to find Minimum Spanning Trees (MSTs). A "cut" is any partition of a graph's vertices into two disjoint sets. The cut property states that for any given cut, the crossing edge (an edge that connects a vertex in one set to a vertex in the other) with the **minimum weight** is guaranteed to be part of the Minimum Spanning Tree of the graph.

Example: Consider a graph with four vertices: A, B, C, and D. Imagine we create a cut that separates the vertices into two sets: Set 1 contains {A, B} and Set 2 contains {C, D}. Let's say there are three edges crossing this cut connecting the two sets:

- Edge A-C with a weight of 4
- Edge B-C with a weight of 7
- Edge B-D with a weight of 9

According to the cut property, because Edge A-C has the lowest weight (4) among all edges crossing this specific partition, it must be included in the Minimum Spanning Tree.

Q2. b) Greedy Algorithm for Filling Water Bottles

Note: The prompt mentions both L liters and U liters. For this algorithm, we will treat U as the target amount of water that needs to be stored.

Greedy Strategy: To use the minimum number of bottles, the most logical approach is to always fill the largest available bottle first. By maximizing the amount of water stored per container, we minimize the total number of containers needed.

Algorithm:

- Sort the capacities:** Take the list of capacities for the n bottles and sort them in descending order (from largest capacity to smallest).
- Initialize variables:** * Set $\text{water_remaining} = U$ (the total liters of water to store).
 - Set $\text{bottles_used} = 0$ (counter for the number of bottles).
- Iterate through the bottles:** Loop through the sorted bottle capacities one by one:
 - If $\text{water_remaining} \leq 0$, break out of the loop (all water is successfully stored).
 - Take the current largest available bottle.
 - Subtract the capacity of this bottle from water_remaining . (Even if the bottle is not filled to the brim, it is still considered a "used" bottle).
 - Increment the bottles_used counter by 1.
- Output:** Return bottles_used . This represents the minimum number of bottles required to store the U liters of water.

You Asked 

Q3 Consider a hash table of size 10 with primary hash function:
 $h_1(k) = k \bmod 10$
Insert the following keys in order: 23, 43, 13, 27, 37, 33, 53
Insert all keys using Double Hashing, where the secondary hash function is given as:
 $h_2(k) = 7 - (k \bmod 7)$
For each element inserted, record the probes required. After all insertions, delete key 33 and insert 63 using linear probing.

Gemini 3.1 Pro Extended

Here is the step-by-step solution to the hash table problem.

Initial Setup & Formulas

- Hash Table Size (m):** 10 (Indices 0 through 9)
- Primary Hash Function:** $h_1(k) = k \bmod 10$
- Secondary Hash Function:** $h_2(k) = 7 - (k \bmod 7)$
- Double Hashing Formula:** $H(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod 10$, where i is the probe number starting from 0.

Part 1: Inserting Keys using Double Hashing

Keys to insert in order: 23, 43, 13, 27, 37, 33, 53

1. Insert 23:

- $i = 0: h(23) = 23 \pmod{10} = 3$
- Index 3 is empty. Insert 23.
- **Probes:** 1

2. Insert 43:

- $i = 0: h(43) = 43 \pmod{10} = 3$ (Collision with 23)
- Calculate $h_2(43) = 7 - (43 \pmod{7}) = 7 - 1 = 6$
- $i = 1: \text{New Index} = (3 + 1 \cdot 6) \pmod{10} = 9$
- Index 9 is empty. Insert 43.
- **Probes:** 2

3. Insert 13:

- $i = 0: h(13) = 13 \pmod{10} = 3$ (Collision with 23)
- Calculate $h_2(13) = 7 - (13 \pmod{7}) = 7 - 6 = 1$
- $i = 1: \text{New Index} = (3 + 1 \cdot 1) \pmod{10} = 4$
- Index 4 is empty. Insert 13.
- **Probes:** 2

4. Insert 27:

- $i = 0: h(27) = 27 \pmod{10} = 7$
- Index 7 is empty. Insert 27.
- **Probes:** 1

5. Insert 37:

- $i = 0: h(37) = 37 \pmod{10} = 7$ (Collision with 27)
- Calculate $h_2(37) = 7 - (37 \pmod{7}) = 7 - 2 = 5$
- $i = 1: \text{New Index} = (7 + 1 \cdot 5) \pmod{10} = 12 \pmod{10} = 2$
- Index 2 is empty. Insert 37.
- **Probes:** 2

6. Insert 33:

- $i = 0: h(33) = 33 \pmod{10} = 3$ (Collision with 23)
- Calculate $h_2(33) = 7 - (33 \pmod{7}) = 7 - 5 = 2$
- $i = 1: \text{New Index} = (3 + 1 \cdot 2) \pmod{10} = 5$
- Index 5 is empty. Insert 33.
- **Probes:** 2

7. Insert 53:

- $i = 0: h(53) = 53 \pmod{10} = 3$ (Collision with 23)

- Calculate $h_2(53) = 7 - (53 \bmod 7) = 7 - 4 = 3$
- $i = 1$: New Index = $(3 + 1 \cdot 3) \bmod 10 = 6$
- Index 6 is empty. Insert 53.
- **Probes: 2**

Summary of Probes:

Key	23	43	13	27	37	33	53
Probes	1	2	2	1	2	2	2

Part 2: Deletion and Linear Probing

1. Delete Key 33:

- Locate 33 using double hashing: $h(33) = 3$ (Contains 23), then $h_2(33) = 2$.
- Next index: $(3 + 2) \bmod 10 = 5$.
- Key 33 is found at index 5. We remove it and mark index 5 as "Deleted" (so future linear or double hashing probes know to keep searching if needed, but it is treated as available space for new insertions).

2. Insert 63 using Linear Probing:

- *Formula for Linear Probing:* $H(k, i) = (h(k) + i) \bmod 10$
- $i = 0$: $h(63) = 63 \bmod 10 = 3$. (Collision, index 3 contains 23).
- $i = 1$: $(3 + 1) \bmod 10 = 4$. (Collision, index 4 contains 13).
- $i = 2$: $(3 + 2) \bmod 10 = 5$. Index 5 was marked as "Deleted", which means it is available for insertion.
- Insert 63 at index 5.
- **Probes required: 3**

Final State of the Hash Table

Index	Value
0	Empty
1	Empty
2	37
3	23
4	13
5	63
6	53
7	27
8	Empty
9	43

Q4: A student named Bhanu aims to improve his skills by enrolling in several online courses. Each course requires a registration fee, and upon successful completion, awards a certain number of credits. Bhanu has a limited amount of savings and must select from the available courses in a way that maximises the total credits earned while staying within budget/ Given that Bhanu has a budget of \$70, employ an appropriate algorithm to identify the courses he should pursue for the specified scenario given in format

Course ID (Fee- Credits) OC1 (\$40-12), OC2 (\$20-8), OC3 (\$30-16), OC4 (\$10-9) OC5(\$30-5)

Q5: The following tasks and dependencies are given for a project with five tasks- A, B, C, D, E. Express these dependencies in the form of a graph and its equivalent adjacency matrix.

Task	Dependencies	Task	Dependencies
B	A	D	B, C
C	A	E	D

Perform **topological ordering** to find a valid task execution sequence.

Gemini 3.1 Pro Extended

Here are the solutions to the questions presented in the second image:

Q4: Course Selection (0/1 Knapsack Problem)

This scenario is a classic example of the **0/1 Knapsack Problem**, where Bhanu must choose a subset of courses to maximize total credits (value) without exceeding his maximum budget of \$70 (weight capacity). He cannot take a fraction of a course (hence 0 or 1).

Given Data:

- **Budget:** \$70
- **OC1:** Fee = \$40, Credits = 12
- **OC2:** Fee = \$20, Credits = 8
- **OC3:** Fee = \$30, Credits = 16
- **OC4:** Fee = \$10, Credits = 9
- **OC5:** Fee = \$30, Credits = 5

Algorithm Application (Dynamic Programming / Systematic Evaluation): Because the number of items is very small ($n=5$), we can easily evaluate the most promising combinations based on their "credit-per-dollar" ratio or by calculating the total weights and values of valid subsets.

Let's look at the best combinations that stay **≤ \$70**:

1. Try combining the highest value items (OC3, OC1):

- $OC3 + OC1 = 30 + 40 = \$70$. Total Credits = $16 + 12 = 28$

2. Try combining highly efficient items (OC4, OC3, OC2):

- $OC4(10) + OC3(30) + OC2(20) = TotalFee60$. (Leaves \$10 unused).
- Total Credits = $9 + 16 + 8 = 33$

3. Try swapping OC2 with OC5 in the above:

- $OC4(10) + OC3(30) + OC5(30) = TotalFee70$.
- Total Credits = $9 + 16 + 5 = 30$

Conclusion: The maximum total credits Bhanu can earn is **33**. He should pursue **OC2, OC3, and OC4**.

Q5: Graph Dependencies, Matrix, and Topological Sorting

Based on the table provided, the tasks have the following dependencies:

- Task B depends on A $\Rightarrow$ Directed edge from A to B ($A \rightarrow B$)
- Task C depends on A $\Rightarrow$ Directed edge from A to C ($A \rightarrow C$)
- Task D depends on B and C $\Rightarrow$ Directed edges from B to D and C to D ($B \rightarrow D, C \rightarrow D$)
- Task E depends on D $\Rightarrow$ Directed edge from D to E ($D \rightarrow E$)

1. Equivalent Adjacency Matrix An adjacency matrix represents a graph using a 2D array. A value of 1 indicates an edge directed from the row's node to the column's node, and 0 indicates no edge.

$$\begin{pmatrix} & \mathbf{A} & \mathbf{B} & \mathbf{C} & \mathbf{D} & \mathbf{E} \\ \mathbf{A} & 0 & 1 & 1 & 0 & 0 \\ \mathbf{B} & 0 & 0 & 0 & 1 & 0 \\ \mathbf{C} & 0 & 0 & 0 & 1 & 0 \\ \mathbf{D} & 0 & 0 & 0 & 0 & 1 \\ \mathbf{E} & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

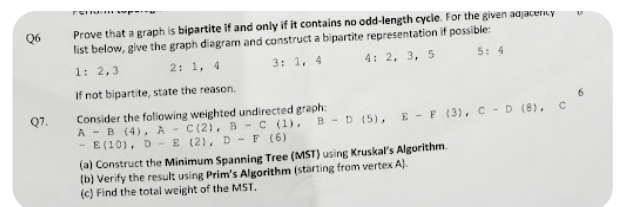
2. Topological Ordering Topological sorting provides a linear ordering of vertices such that for every directed edge $U \rightarrow V$, vertex U comes before V in the ordering. This is used to find a valid execution sequence for tasks.

We can solve this by tracking the "in-degree" (number of incoming edges) for each node:

- **Initial in-degrees:** A=0, B=1, C=1, D=2, E=1
- **Step 1:** Node **A** has an in-degree of 0. Execute A.
 - Sequence: A * Remove A's outgoing edges. New in-degrees: B=0, C=0, D=2, E=1.
- **Step 2:** Nodes B and C now have an in-degree of 0. We can pick either. Let's pick **B**.
 - Sequence: A, B
 - Remove B's outgoing edge. New in-degrees: C=0, D=1, E=1.
- **Step 3:** Node **C** has an in-degree of 0. Execute C.
 - Sequence: A, B, C
 - Remove C's outgoing edge. New in-degrees: D=0, E=1.
- **Step 4:** Node **D** now has an in-degree of 0. Execute D.
 - Sequence: A, B, C, D
 - Remove D's outgoing edge. New in-degrees: E=0.
- **Step 5:** Node **E** has an in-degree of 0. Execute E.
 - Final Sequence: A, B, C, D, E

(Note: Because B and C become available at the same time in Step 2, another perfectly valid topological order is A, C, B, D, E).

You Asked 



Gemini 3.1 Pro Extended

Here are the solutions to the questions presented in the image:

Q6: Bipartite Graphs and Cycles

Part 1: Proof that a graph is bipartite if and only if it contains no odd-length cycle

This is a standard biconditional proof, meaning we must prove both directions. Let $G = (V, E)$ be a graph.

- **Forward Direction ($\implies$): If G is bipartite, it has no odd-length cycles.** Assume G is bipartite. By definition, its vertex set V can be partitioned into two disjoint sets, X and Y , such that every edge connects a vertex in X to a vertex in Y . Suppose there is a cycle $v_1, v_2, \dots, v_k, v_1$ in G . Without loss of generality, let $v_1 \in X$. Because edges must alternate between sets, $v_2 \in Y, v_3 \in X, v_4 \in Y$, and so on. Thus, $v_i \in X$ if i is odd, and $v_i \in Y$ if i is even. For the cycle to close, the edge (v_k, v_1) must connect a vertex in Y back to X . Therefore, v_k must be in Y , which means k must be an even number. Thus, every cycle in G must have an even length.
- **Reverse Direction ($\impliedby$): If G has no odd-length cycles, it is bipartite.** Assume G has no odd-length cycles. Pick an arbitrary starting vertex u and place it in set X . For every other vertex v , find the shortest path from u to v .
 - If the shortest path length is even, place v in X .
 - If the shortest path length is odd, place v in Y . We must prove no two vertices in the same set share an edge. Suppose two vertices $v, w \in X$ share an edge. By our assignment, the shortest path from u to v is even, and the shortest path from u to w is even. The cycle formed by the path $u \dots v$, the edge (v, w) , and the path $w \dots u$ would have a length of: $(\text{even}) + 1 + (\text{even}) = \text{odd}$. This contradicts our initial assumption that there are no odd cycles. Therefore, no two vertices in the same set share an edge, and G is bipartite.

Part 2: Analyzing the Given Graph

Adjacency List:

- 1: 2, 3
- 2: 1, 4
- 3: 1, 4
- 4: 2, 3, 5
- 5: 4

Edges: $(1, 2), (1, 3), (2, 4), (3, 4), (4, 5)$

Is it Bipartite? Yes. We can check this by attempting a 2-coloring (Red/Blue) representing sets U and V .

1. Assign Node **1** to Set U (Red).
2. Nodes **2** and **3** connect to 1, so they must go to Set V (Blue).
3. Node **4** connects to 2 and 3. Since 2 and 3 are in V , 4 must go to Set U (Red).

4. Node 5 connects to 4. Since 4 is in U , 5 must go to Set V (Blue).

Bipartite Representation: The vertices can be strictly partitioned into two independent sets with all edges crossing between them:

- Set U : $\{1, 4\}$
- Set V : $\{2, 3, 5\}$

Edges connecting the sets:

- $1 \longleftrightarrow 2$
- $1 \longleftrightarrow 3$
- $4 \longleftrightarrow 2$
- $4 \longleftrightarrow 3$
- $4 \longleftrightarrow 5$

Q7. Minimum Spanning Tree (MST)

Given Edges & Weights: $A - C(2)$, $B - C(1)$, $D - E(2)$, $E - F(3)$, $A - B(4)$, $B - D(5)$, $D - F(6)$, $C - D(8)$, $C - E(10)$

(a) Construct MST using Kruskal's Algorithm

Kruskal's algorithm sorts edges by ascending weight and adds them to the tree sequentially, skipping any edge that forms a cycle.

Sorted Edges:

1. $(B, C) = 1$
2. $(A, C) = 2$
3. $(D, E) = 2$
4. $(E, F) = 3$
5. $(A, B) = 4$
6. $(B, D) = 5$
7. $(D, F) = 6$
8. $(C, D) = 8$
9. $(C, E) = 10$

Execution:

- Add (B, C) [Weight 1]. Tree: $\{B, C\}$
- Add (A, C) [Weight 2]. Tree: $\{A, B, C\}$
- Add (D, E) [Weight 2]. Tree: $\{A, B, C\} \cup \{D, E\}$
- Add (E, F) [Weight 3]. Tree: $\{A, B, C\} \cup \{D, E, F\}$
- Skip (A, B) [Weight 4]. Reason: Forms cycle A-C-B.
- Add (B, D) [Weight 5]. Tree: $\{A, B, C, D, E, F\}$. (All vertices connected).

Final MST Edges: (B, C) , (A, C) , (D, E) , (E, F) , (B, D)

(b) Verify result using Prim's Algorithm (starting from A)

Prim's algorithm starts at a chosen node and greedily expands the tree by adding the lowest-weight edge connected to the currently visited nodes.

- **Step 1:** Start at **A**. Available edges: $(A, C) = 2, (A, B) = 4$.
 - Choose minimum: (A, C) . Visited: $\{A, C\}$.
- **Step 2:** Available edges from $\{A, C\}$: $(C, B) = 1, (A, B) = 4, (C, D) = 8, (C, E) = 10$.
 - Choose minimum: (C, B) . Visited: $\{A, C, B\}$.
- **Step 3:** Available edges from $\{A, C, B\}$: $(A, B) = 4$ (cycle), $(B, D) = 5, (C, D) = 8, (C, E) = 10$.
 - Choose minimum: (B, D) . Visited: $\{A, C, B, D\}$.
- **Step 4:** Available edges from $\{A, C, B, D\}$: $(D, E) = 2, (D, F) = 6, (C, D) = 8$ (cycle), $(C, E) = 10$.
 - Choose minimum: (D, E) . Visited: $\{A, C, B, D, E\}$.
- **Step 5:** Available edges from $\{A, C, B, D, E\}$: $(E, F) = 3, (D, F) = 6, (C, E) = 10$ (cycle).
 - Choose minimum: (E, F) . Visited: $\{A, C, B, D, E, F\}$.

The resulting edges selected by Prim's are identical to Kruskal's, verifying the tree.

(c) Find the total weight of the MST

Summing the weights of the edges in our MST: $1(\text{for } B - C) + 2(\text{for } A - C) + 2(\text{for } D - E) + 3(\text{for } E - F) + 5(\text{for } B - D)$ **Total Weight = 13**

You Asked 

Q1 a) Consider the scheduling problem wherein you are given a single resource and a set of requests with deadlines. A request is said to be late if it misses the deadline. Your goal is to minimise the maximum lateness. With respect to a schedule S , idle time is defined as the time during which the resource is idle, in between two requests. S is said to have an inversion when request i has been scheduled before j and $d(i) > d(j)$, where $d(i)$ and $d(j)$ are the deadlines of the requests i and j , respectively. Argue that all schedules with no idle time and no inversions have the same maximum lateness. 4

b) Define **optimal substructure** with an example. 2

Q2 a) Define a **bipartite graph**. Determine whether the following graph is **bipartite**. Use BFS and show the vertex-colouring process step by step. 6

Vertices: $\{A, B, C, D, E\}$

Edges: $\{A-B, A-C, B-D, C-D, D-E\}$

Gemini 3.1 Pro Extended

Here are the detailed solutions to the questions from your exam paper:

Q1 a) Scheduling Problem: Schedules with No Inversions and No Idle Time

Objective: Argue that all schedules with no idle time and no inversions have the same maximum lateness.

Proof / Argument:

1. **Understanding "No Inversions":** An inversion occurs when request i is scheduled before j , but its deadline is later ($d(i) > d(j)$). Therefore, a schedule with *no* inversions means all jobs are sorted strictly in non-decreasing order of their deadlines. This is often called Earliest Deadline First (EDF) order, where $d(1) \leq d(2) \leq \dots \leq d(n)$.
2. **Understanding "No Idle Time":** This means the resource processes jobs back-to-back starting from time $t = 0$, without any gaps.

3. **Comparing two such schedules:** Let S_1 and S_2 be two distinct schedules that both have no idle time and no inversions. Because both follow the $d(1) \leq d(2) \dots$ rule, they can only differ in how they order jobs that have the **exact same deadline**.
4. **Analyzing identical deadline blocks:** Let R be a contiguous block of requests that all share the exact same deadline, D . Because both S_1 and S_2 have no idle time and have processed the exact same set of earlier jobs before reaching R , the start time for block R is identical in both schedules. Consequently, the completion time of the *entire block* R is also identical in both.
5. **Maximum Lateness within the block:** The lateness of a job is its completion time minus its deadline. For the block R , the maximum lateness is strictly determined by the completion time of the *last* job in that block, because every job in R shares the same deadline D .
6. **Conclusion:** Since block R completes at the exact same time in both S_1 and S_2 , the maximum lateness for the jobs within R is identical. Because this holds true for every block of jobs with identical deadlines, the overall maximum lateness across the entire schedule must be exactly the same for S_1 and S_2 .

Q1 b) Optimal Substructure

Definition: A problem is said to have an **optimal substructure** if an optimal solution to the overall problem can be constructed efficiently from the optimal solutions of its smaller subproblems. It is a core property required to solve problems using Dynamic Programming or Greedy Algorithms.

Example: The **Shortest Path Problem** in a graph. Suppose the shortest path from City A to City C goes through City B. The path can be broken into two segments: A to B, and B to C. For the overall path (A to C) to be the shortest possible, the sub-path from A to B must be the shortest path between those two cities, and the sub-path from B to C must also be the shortest. If there were a faster route from A to B, you could substitute it to make the total trip from A to C even shorter, which would contradict the initial assumption.

Q2 a) Bipartite Graphs and BFS Vertex-Colouring

Definition of a Bipartite Graph: A bipartite graph is a graph whose vertices can be divided into two disjoint (non-overlapping) sets, U and V , such that every edge in the graph connects a vertex in set U to a vertex in set V . Equivalently, it is a graph that can be colored using exactly two colors such that no two adjacent vertices share the same color.

Determining if the given graph is bipartite:

- **Vertices:** $\{A, B, C, D, E\}$
- **Edges:** $\{A - B, A - C, B - D, C - D, D - E\}$

Let's use **Red** (Set U) and **Blue** (Set V) for our colors. We will traverse the graph using Breadth-First Search (BFS):

1. **Step 1:** Start at an arbitrary vertex, say **A**. Color it **Red**.
 - State: A is Red. Queue: [A]
2. **Step 2:** Dequeue A. Find its uncolored neighbors (**B** and **C**) and assign them the opposite color, **Blue**.
 - State: A is Red. B, C are Blue. Queue: [B, C]
3. **Step 3:** Dequeue B. Its neighbors are A (already Red, which is valid) and **D** (uncolored). Assign D the opposite color to B, so **Red**.
 - State: A, D are Red. B, C are Blue. Queue: [C, D]
4. **Step 4:** Dequeue C. Its neighbors are A (already Red) and D (already Red).
 - Check: The edge C-D connects C (Blue) to D (Red). The colors alternate correctly, so there is no conflict.

◦ State: Queue: [D]

5. **Step 5:** Dequeue D. Its neighbors are B (Blue), C (Blue), and E (uncolored). Assign E the opposite color to D, so **Blue**.

◦ State: A, D are Red. B, C, E are Blue. Queue: [E]

6. **Step 6:** Dequeue E. Its neighbor is D (already Red, which is valid). The queue is now empty.

Conclusion: Because we successfully colored all vertices with 2 colors without any two adjacent vertices sharing the same color, **YES, the graph is bipartite**. The two sets are:

- **Set U (Red):** {A, D}
- **Set V (Blue):** {B, C, E}

You Asked 

Q3

Given an array A of n integers, you need to find the maximum sum of any contiguous subarray. For instance, the maximum sum of any contiguous subarray in the array [-1, 2, 3, -2, 5, 6, 7, -8] is 9 (which is the sum of the subarray [2, 3, -2, 5, -6, 7]). Complete the following Dynamic Programming solution for the above problem with proper explanation:

DP[0] = A[0]
for i=1 to n DP[i] = max(A[i], _____)

Q4

Will the greedy strategy with the greedy parameter being the value per unit weight of the items yield an optimal solution for the 0-1 knapsack problem? Justify your answer by solving the Knapsack for the following data:

Item	Weight	Value
1	10	60
2	20	100
3	30	120

Max Capacity of Knapsack=50

Gemini 3.1 Pro Extended

Here are the solutions to the questions presented in the image:

Q3: Maximum Contiguous Subarray (Dynamic Programming)

The Completed Code: for i=1 to n DP[i] = max(A[i], A[i] + DP[i-1])

Explanation: This problem is solved using **Kadane's Algorithm**. The array DP is used to store the maximum subarray sum that ends exactly at index i.

At any given index i, you have exactly two choices to maximize your sum:

1. **Start a new subarray:** You discard the previous sequence and start fresh with the current element, A[i].
2. **Extend the existing subarray:** You add the current element A[i] to the maximum sum of the subarray ending at the previous index, DP[i-1].

The algorithm takes the maximum of these two choices. If DP[i-1] is positive, adding A[i] to it will always yield a larger sum than A[i] alone. However, if DP[i-1] is negative, it will drag down the value of A[i], so it is better to start a new subarray at A[i].

Q4: 0-1 Knapsack Problem (Greedy Strategy vs. Optimal)

Answer: No, the greedy strategy based on value-per-unit-weight will **not** consistently yield an optimal solution for the 0-1 Knapsack problem.

Justification (Solving the Data):

First, let's calculate the Value-to-Weight ratio for each item:

- **Item 1:** $60/10 = 6$
- **Item 2:** $100/20 = 5$
- **Item 3:** $120/30 = 4$

1. The Greedy Approach: The greedy algorithm prioritizes items with the highest value-to-weight ratio.

- **Step 1:** Pick Item 1 (Ratio 6). Weight used = 10. Value = 60. Remaining capacity = 40.
- **Step 2:** Pick Item 2 (Ratio 5). Weight used = 10 + 20 = 30. Value = 60 + 100 = 160. Remaining capacity = 20.
- **Step 3:** Try to pick Item 3 (Ratio 4). Its weight is 30, but we only have 20 capacity left. Because this is the *0-1 Knapsack* problem (you cannot take fractions of an item), we must skip it.
- **Total Greedy Value = 160** (Total weight = 30)

2. The Optimal Solution: Let's evaluate other possible combinations that fit within the maximum capacity of 50:

- **Item 1 + Item 3:** Total Weight = 40, Total Value = $60 + 120 = 180$
- **Item 2 + Item 3:** Total Weight = 50, Total Value = $100 + 120 = 220$

Conclusion: The maximum possible value we can fit in the knapsack is **220** (by selecting Items 2 and 3). The greedy algorithm gets stuck with a suboptimal value of **160** because it greedily takes Item 1, leaving empty space (20 units of capacity) that it cannot fill. Therefore, the greedy strategy fails for the 0-1 Knapsack problem.

You Asked 

Q5 Consider a hash table of size 10 with hash function: $h(k) = k \bmod 10$. Insert the following keys in order: 23, 43, 13, 27, 37, 33, 53. For each element inserted, record the probes required.

- Insert all keys using Linear Probing.
- After all insertions, delete key 43 and insert 63 using linear probing.

Q6 Consider the following weighted undirected graph with the following:

Vertices: A, B, C, D, E

Edges: A-B (1), A-C (3), B-C (1), B-D (4), C-D (2), D-E (2), C-E (5), B-F (3), D-F (5)

- Represent the above graph using an equivalent adjacency list.
- Find the MST using Prim's algorithm, with node D as the start node, showing all intermediate steps.
- Find the MST using Kruskal's algorithm, and verify that both give the same total cost.

Gemini 3.1 Pro Extended

Here are the step-by-step solutions to the questions in the image provided:

Q5: Hash Table with Linear Probing

Initial Parameters:

- **Hash Table Size (m):** 10 (Indices 0 through 9)
- **Hash Function:** $h(k) = k \bmod 10$
- **Linear Probing Formula:** $H(k, i) = (h(k) + i) \bmod 10$

i) Insert all keys using Linear Probing

Keys to insert in order: 23, 43, 13, 27, 37, 33, 53

1. **Insert 23:** $h(23) = 3$. Index 3 is empty. Insert at 3.

- **Probes required:** 1
- 2. **Insert 43:** $h(43) = 3$. Collision at 3. Next index is 4. Index 4 is empty. Insert at 4.
 - **Probes required:** 2
- 3. **Insert 13:** $h(13) = 3$. Collisions at 3 and 4. Next index is 5. Index 5 is empty. Insert at 5.
 - **Probes required:** 3
- 4. **Insert 27:** $h(27) = 7$. Index 7 is empty. Insert at 7.
 - **Probes required:** 1
- 5. **Insert 37:** $h(37) = 7$. Collision at 7. Next index is 8. Index 8 is empty. Insert at 8.
 - **Probes required:** 2
- 6. **Insert 33:** $h(33) = 3$. Collisions at 3, 4, and 5. Next index is 6. Index 6 is empty. Insert at 6.
 - **Probes required:** 4
- 7. **Insert 53:** $h(53) = 3$. Collisions at 3, 4, 5, 6, 7, and 8. Next index is 9. Index 9 is empty. Insert at 9.
 - **Probes required:** 7

State of Hash Table after insertions: [0: Empty, 1: Empty, 2: Empty, 3: 23, 4: 43, 5: 13, 6: 33, 7: 27, 8: 37, 9: 53]

ii) Delete key 43 and insert 63

1. Delete 43:

- Locate 43: Probe $h(43) = 3$ (contains 23), probe 4 (contains 43).
- Remove 43 and mark index 4 as **"Deleted"**. (This marker ensures future searches don't prematurely stop, while allowing new insertions to reuse the space).

2. Insert 63:

- $h(63) = 3$.
- Probe index 3: Collision (contains 23).
- Probe index 4: Encountered the "Deleted" marker. We can safely insert 63 here.
- **Probes required:** 2

Final State of Hash Table: [0: Empty, 1: Empty, 2: Empty, 3: 23, 4: 63, 5: 13, 6: 33, 7: 27, 8: 37, 9: 53]

Q6: Minimum Spanning Tree (MST)

(Note: The problem description lists vertices A through E, but the edge list includes vertex F. The solution below correctly incorporates vertex F as implied by the edges).

a) Equivalent Adjacency List

An adjacency list represents each vertex and lists all its directly connected neighbors along with the edge weights.

- **A:** (B, 1), (C, 3)
- **B:** (A, 1), (C, 1), (D, 4), (F, 3)
- **C:** (A, 3), (B, 1), (D, 2), (E, 5)
- **D:** (B, 4), (C, 2), (E, 2), (F, 5)
- **E:** (D, 2), (C, 5)
- **F:** (B, 3), (D, 5)

b) Find MST using Prim's Algorithm (Start at Node D)

Prim's algorithm builds the MST by starting at a given node and always picking the smallest available edge that connects a visited node to an unvisited node.

- **Step 1: Start at D.**
 - Available edges: D-B(4), D-C(2), D-E(2), D-F(5).
 - Action: Choose the minimum edge **D-C (2)**.
 - Visited: {D, C}. MST Edges so far: {D-C}
- **Step 2:**
 - Available edges from {D, C}: D-B(4), D-E(2), D-F(5), C-A(3), C-B(1), C-E(5).
 - Action: Choose the minimum edge **C-B (1)**.
 - Visited: {D, C, B}. MST Edges so far: {D-C, C-B}
- **Step 3:**
 - Available edges from {D, C, B}: D-E(2), D-F(5), C-A(3), C-E(5), B-A(1), B-F(3). (Ignoring D-B as it forms a cycle).
 - Action: Choose the minimum edge **B-A (1)**.
 - Visited: {D, C, B, A}. MST Edges so far: {D-C, C-B, B-A}
- **Step 4:**
 - Available edges from {D, C, B, A}: D-E(2), D-F(5), C-E(5), B-F(3). (Ignoring C-A).
 - Action: Choose the minimum edge **D-E (2)**.
 - Visited: {D, C, B, A, E}. MST Edges so far: {D-C, C-B, B-A, D-E}
- **Step 5:**
 - Available edges from {D, C, B, A, E}: D-F(5), B-F(3). (Ignoring C-E).
 - Action: Choose the minimum edge **B-F (3)**.
 - Visited: {D, C, B, A, E, F}. All vertices connected.

Prim's MST Total Cost: $2 + 1 + 1 + 2 + 3 = 9$

c) Find MST using Kruskal's Algorithm & Verify Total Cost

Kruskal's algorithm sorts all edges in the graph by weight and greedily adds the smallest edge to the tree as long as it does not form a cycle.

Sorted Edges:

1. A-B (1)
2. B-C (1)
3. C-D (2)
4. D-E (2)
5. A-C (3)
6. B-F (3)
7. B-D (4)
8. C-E (5)
9. D-F (5)

Step-by-step Selection:

1. Select **A-B (1)**.
2. Select **B-C (1)**.
3. Select **C-D (2)**.
4. Select **D-E (2)**.
5. *Skip A-C (3)* because it connects A and C, which are already in the same tree (forms cycle A-B-C).
6. Select **B-F (3)**.
7. All 6 vertices are now connected. The algorithm terminates.

Selected Edges for MST: {A-B, B-C, C-D, D-E, B-F} **Kruskal's MST Total Cost:** $1 + 1 + 2 + 2 + 3 = 9$

Verification: Both Prim's Algorithm and Kruskal's Algorithm yield the exact same set of edges and the identical total minimum cost of **9**.

You Asked 

Solve the subset problem using dynamic programming for the set (4, 2, 9, 6) with an intended sum 17.

Gemini 3.1 Pro Extended

To solve the **Subset Sum Problem** using Dynamic Programming, we need to find if there is a subset of the given set {4, 2, 9, 6} that adds up to exactly 17.

1. Dynamic Programming Setup

Let our set be $A = \{4, 2, 9, 6\}$ with $n = 4$ elements, and our target sum $W = 17$.

We create a 2D boolean table `dp` of size $(n + 1) \times (W + 1)$, which is 5×18 .

- `dp[i][j]` will be `True` if there is a subset of the first i elements that sums up to exactly j .
- `dp[i][j]` will be `False` otherwise.

Base Cases:

- `dp[i][0] = True` for all $0 \leq i \leq 4$. (A sum of 0 is always possible by choosing an empty subset).
- `dp[0][j] = False` for all $1 \leq j \leq 17$. (A non-zero sum is impossible with 0 elements).

Transition Rule: For each element $A[i - 1]$ (since arrays are 0-indexed) and each target sum j from 1 to 17:

- If the current element is greater than the target sum ($A[i - 1] > j$), we cannot include it: `dp[i][j] = dp[i-1][j]`
- If the current element is less than or equal to the target sum ($A[i - 1] \leq j$), we check if we can form the sum by either **excluding** it OR **including** it: `dp[i][j] = dp[i-1][j] OR dp[i-1][j - A[i-1]]`

2. Filling the DP Table

Let's build the table step-by-step. `T` represents True, `F` represents False.

- **Row 0:** $\{\}$ (Empty set)
- **Row 1:** Include $\{4\}$
- **Row 2:** Include $\{4, 2\}$
- **Row 3:** Include $\{4, 2, 9\}$
- **Row 4:** Include $\{4, 2, 9, 6\}$

$i \setminus \text{Sum } j$	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
0 ($\{\}$)	T	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F
1 ($\{4\}$)	T	F	F	F	T	F	F	F	F	F	F	F	F	F	F	F	F	F
2 ($\{4, 2\}$)	T	F	T	F	T	F	T	F	F	F	F	F	F	F	F	F	F	F
3 ($\{4, 2, 9\}$)	T	F	T	F	T	F	T	F	F	T	F	T	F	T	F	T	F	F
4 ($\{...6\}$)	T	F	T	F	T	F	T	F	T	T	T	T	T	T	F	T	F	T

3. Conclusion & Backtracking

Look at the bottom-right cell: $\text{dp}[4][17] = \text{True}$. This means **it is possible** to get a sum of 17 using the given set.

To find the actual subset, we backtrack from $\text{dp}[4][17]$ to see which elements were included to reach that state:

1. **Start at $\text{dp}[4][17]$** : It is **True** .
2. Look at the row above it, $\text{dp}[3][17]$. It is **False** .
 - *Meaning:* We couldn't make 17 without the 4th element (which is **6**). Therefore, **6 is in the subset**.
 - *Remaining target:* $17 - 6 = 11$.
3. **Move to $\text{dp}[3][11]$** : It is **True** .
4. Look at the row above it, $\text{dp}[2][11]$. It is **False** .
 - *Meaning:* We couldn't make 11 without the 3rd element (which is **9**). Therefore, **9 is in the subset**.
 - *Remaining target:* $11 - 9 = 2$.
5. **Move to $\text{dp}[2][2]$** : It is **True** .
6. Look at the row above it, $\text{dp}[1][2]$. It is **False** .
 - *Meaning:* We couldn't make 2 without the 2nd element (which is **2**). Therefore, **2 is in the subset**.
 - *Remaining target:* $2 - 2 = 0$.
7. The remaining target is 0, which means we have found our complete subset.

Final Answer: The subset that sums to 17 is $\{6, 9, 2\}$.

Q1 a) Which sorting algorithm out of insertion and quick-sort will sort an array of size n faster if the list is already sorted? Why? 2

b) Discuss the running time of the following snippet of code: 3

```
count = 0
for (i=1, i<=n, i++)
    for (j=1, i<=n, j = 2*j)
        count++
```

Q2 Sort the array {7, 3, 2, 28, 11} using the Selection sort, and Merge Sort techniques and write the status of the array during every iteration of each pass. Also, find the cumulative number of comparisons at each iteration. 5

Q3 Use Strassen's algorithm to compute the product of the following matrices: 5

$$\begin{pmatrix} 2 & 3 \\ 5 & 1 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 2 & 2 \end{pmatrix}$$

Gemini 3.1 Pro Extended

Here are the solutions to the questions presented in the image:

Q1 a) Insertion Sort vs. Quick-Sort on a Sorted Array

Insertion sort will sort an already sorted array of size n much faster than quick-sort.

Why?

- **Insertion Sort:** When an array is already sorted, insertion sort only needs to compare each element to the one immediately before it. Since it's already in the correct order, no shifting or swapping is required. The inner loop never executes, resulting in a best-case time complexity of $O(n)$.
- **Quick-Sort:** The performance of quick-sort heavily depends on pivot selection. If a standard quick-sort algorithm (which often picks the first or last element as the pivot) is run on a sorted array, it hits its worst-case scenario. It will partition the array into heavily unbalanced sub-arrays (one of size 0, and one of size $n-1$). This degrades its performance to $O(n^2)$. Even with optimized pivot selection (like median-of-three or random pivot), its best possible time complexity is $O(n \log n)$, which is still slower than insertion sort's $O(n)$.

Q1 b) Running Time of the Code Snippet

(Note: There appear to be typographical errors in the printed snippet's syntax, such as using commas instead of semicolons, unmatched parentheses, and the variable i being used in the inner loop condition $i \leq n$ where $j \leq n$ is mathematically intended to prevent an infinite loop. We will analyze this based on the standard algorithmic structure intended by such a problem).

Intended Logic:

```
C
count = 0;
for(i=1; i<=n; i++) {
    for(j=1; j<=n; j = 2*j) {
        count++;
    }
}
```

Analysis:

1. **Outer Loop:** The loop `for(i=1; i<=n; i++)` executes exactly n times.
2. **Inner Loop:** The loop `for(j=1; j<=n; j = 2*j)` does not increment by 1; it doubles its value on each iteration (1, 2, 4, 8... up to n). The number of times this loop runs is the number of times you can multiply 1 by 2 until you reach n , which is exactly $\log_2(n)$ times.
3. **Total Running Time:** Because the loops are nested, you multiply their execution frequencies.

Time Complexity: $n \times \log_2(n) = O(n \log n)$

Q2) Sorting the Array {7, 3, 2, 28, 11}

1. Selection Sort

Selection sort repeatedly finds the minimum element from the unsorted part and puts it at the beginning.

Pas s	Array Status (Unsorted portion analyzed)	Action	Comparisons (Current I teration)	Cumulative Compa risons
Initi al	[7, 3, 2, 28, 11]			0
1	[2, 3, 7, 28, 11]	Find min (2), swap wit h 7	4	4
2	[2, 3, 7, 28, 11]	Find min (3), no swap needed	3	7
3	[2, 3, 7, 28, 11]	Find min (7), no swap needed	2	9
4	[2, 3, 7, 11, 28]	Find min (11), swap w ith 28	1	10

2. Merge Sort (Iterative / Bottom-Up)

To clearly show the "status during every iteration of each pass," a bottom-up iterative merge sort approach illustrates the merging levels perfectly.

Initial Array: [7, 3, 2, 28, 11]

Pass 1: Merge sub-arrays of size 1 into size 2

- Merge [7] and [3] → **[3, 7]** (1 comparison)
- Merge [2] and [28] → **[2, 28]** (1 comparison)
- Element [11] is left over.
- **Array Status:** [3, 7, 2, 28, 11]
- **Cumulative Comparisons:** 2

Pass 2: Merge sub-arrays of size 2 into size 4

- Merge [3, 7] and [2, 28].
 - 3 vs 2 (pick 2)
 - 3 vs 28 (pick 3)
 - 7 vs 28 (pick 7)

- Append remaining 28.
- Result → **[2, 3, 7, 28]** (3 comparisons)
- Element [11] is left over.
- **Array Status:** [2, 3, 7, 28, 11]
- **Cumulative Comparisons:** $2 + 3 = 5$

Pass 3: Merge sub-arrays of size 4 into final size

- Merge [2, 3, 7, 28] and [11].
 - 2 vs 11 (pick 2)
 - 3 vs 11 (pick 3)
 - 7 vs 11 (pick 7)
 - 28 vs 11 (pick 11)
 - Append remaining 28.
 - Result → **[2, 3, 7, 11, 28]** (4 comparisons)
- **Final Array Status:** [2, 3, 7, 11, 28]
- **Cumulative Comparisons:** $5 + 4 = 9$

Q3) Strassen's Algorithm Matrix Multiplication

Given Matrices: $A = \begin{pmatrix} 2 & 3 \\ 5 & 1 \end{pmatrix}$ $B = \begin{pmatrix} 1 & 1 \\ 2 & 2 \end{pmatrix}$

Step 1: Calculate the 7 intermediate variables (P_1 to P_7)

- $P_1 = A_{11}(B_{12} - B_{22}) = 2(1 - 2) = 2(-1) = -2$
- $P_2 = (A_{11} + A_{12})B_{22} = (2 + 3)(2) = 5(2) = 10$
- $P_3 = (A_{21} + A_{22})B_{11} = (5 + 1)(1) = 6(1) = 6$
- $P_4 = A_{22}(B_{21} - B_{11}) = 1(2 - 1) = 1(1) = 1$
- $P_5 = (A_{11} + A_{22})(B_{11} + B_{22}) = (2 + 1)(1 + 2) = 3(3) = 9$
- $P_6 = (A_{12} - A_{22})(B_{21} + B_{22}) = (3 - 1)(2 + 2) = 2(4) = 8$
- $P_7 = (A_{11} - A_{21})(B_{11} + B_{12}) = (2 - 5)(1 + 1) = -3(2) = -6$

Step 2: Compute the resulting matrix C

- $C_{11} = P_5 + P_4 - P_2 + P_6 = 9 + 1 - 10 + 8 = 8$
- $C_{12} = P_1 + P_2 = -2 + 10 = 8$
- $C_{21} = P_3 + P_4 = 6 + 1 = 7$
- $C_{22} = P_5 + P_1 - P_3 - P_7 = 9 + (-2) - 6 - (-6) = 9 - 2 - 6 + 6 = 7$

Final Result Matrix: $C = \begin{pmatrix} 8 & 8 \\ 7 & 7 \end{pmatrix}$

Q4 Given the set of following activities, find the set of activities and thus the number of activities that can be performed by a person, given that he/she can perform only one task at a time:

Activity Name	Start time	Finish time
A	1	2
B	3	4
C	0	6
D	5	7
E	8	9
F	5	9

What will be the criteria of picking up the activities? Which algorithmic approach you followed?

Q5 Demonstrate the procedure for finding any one topological order for the following graph, clearly showing the graph at each step. Also, find all possible topological orderings.

Gemini 3.1 Pro Extended

Here are the solutions to the questions presented in the image:

Q4: Activity Selection Problem

Algorithmic Approach: The problem is solved using the **Greedy Algorithm** approach, specifically known as the Activity Selection Problem.

Criteria for picking up activities: To maximize the number of activities performed, the greedy choice is to always select the next possible activity that finishes the earliest.

1. Sort all activities in ascending order of their **Finish Time**.
2. Select the first activity from the sorted list.
3. For all remaining activities, select an activity if its **Start Time** is greater than or equal to the Finish Time of the previously selected activity.

Step-by-Step Execution:

First, we sort the table based on Finish Time:

Activity	Start Time	Finish Time
A	1	2
B	3	4
C	0	6
D	5	7
E	8	9
F	5	9

Note: E and F tie for finish time. It does not matter which is listed first for the algorithm to work, but standard sorting keeps them in input order.

Now, we apply the greedy criteria:

1. **Select A:** It finishes first at time 2 . (Selected: A)
2. **Evaluate B:** Starts at 3 . Since $3 \geq 2$ (A's finish time), we select it. (Selected: A, B)

3. **Evaluate C:** Starts at 0 . Since $0 < 4$ (B's finish time), it overlaps. We skip it.
4. **Evaluate D:** Starts at 5 . Since $5 \geq 4$ (B's finish time), we select it. (Selected: **A, B, D**)
5. **Evaluate E:** Starts at 8 . Since $8 \geq 7$ (D's finish time), we select it. (Selected: **A, B, D, E**)
6. **Evaluate F:** Starts at 5 . Since $5 < 9$ (E's finish time), it overlaps. We skip it. (Even if evaluated before E, it starts at 5 which is < 7 , so it would be skipped regardless).

Final Answer:

- **Set of activities:** {A, B, D, E}
- **Number of activities:** 4

Q5: Topological Ordering

A topological sort creates a linear ordering of vertices such that for every directed edge $U \rightarrow V$, vertex U comes before V . We find this by iteratively removing nodes with an "in-degree" of 0 (nodes with no incoming edges).

Part 1: Demonstrating the Procedure for ONE Topological Order

Initial State:

- In-degrees: Node 1: 0 , Node 2: 1 , Node 3: 1 , Node 4: 2 , Node 5: 2 , Node 6: 2
- Current Sequence: []

Step 1: * Action: Node 1 has an in-degree of 0. Remove Node 1 and its outgoing edges ($1 \rightarrow 2, 1 \rightarrow 3$).

- **New Graph State:** The remaining nodes are {2, 3, 4, 5, 6}.
- **Updated In-degrees:** Node 2: 0 , Node 3: 0 , Node 4: 2 , Node 5: 2 , Node 6: 2
- **Current Sequence:** [1]

Step 2:

- **Action:** Nodes 2 and 3 now have an in-degree of 0. Let's pick Node 2. Remove Node 2 and its outgoing edges ($2 \rightarrow 4, 2 \rightarrow 5$).
- **New Graph State:** The remaining nodes are {3, 4, 5, 6}.
- **Updated In-degrees:** Node 3: 0 , Node 4: 1 , Node 5: 1 , Node 6: 2
- **Current Sequence:** [1, 2]

Step 3:

- **Action:** Node 3 has an in-degree of 0. Remove Node 3 and its outgoing edges ($3 \rightarrow 4, 3 \rightarrow 6$).
- **New Graph State:** The remaining nodes are {4, 5, 6}.
- **Updated In-degrees:** Node 4: 0 , Node 5: 1 , Node 6: 1
- **Current Sequence:** [1, 2, 3]

Step 4:

- **Action:** Node 4 has an in-degree of 0. Remove Node 4 and its outgoing edges ($4 \rightarrow 5, 4 \rightarrow 6$).
- **New Graph State:** The remaining nodes are {5, 6}. No edges remain.
- **Updated In-degrees:** Node 5: 0 , Node 6: 0
- **Current Sequence:** [1, 2, 3, 4]

Step 5:

- **Action:** Nodes 5 and 6 have an in-degree of 0. Let's pick Node 5. Remove Node 5.
- **Current Sequence:** [1, 2, 3, 4, 5]

Step 6:

- **Action:** Pick the final node, Node 6. Remove Node 6.
- **Final Sequence:** [1, 2, 3, 4, 5, 6]

Part 2: Finding ALL Possible Topological Orderings

By analyzing the dependencies, we can map out all valid paths:

- **Node 1** must always be first because it's the only starting node.
- After Node 1, we can process **Node 2** and **Node 3** in any order, because neither depends on the other. (Creates branches: 1, 2, 3... or 1, 3, 2...).
- **Node 4** depends on *both* 2 and 3. Therefore, Node 4 must always come exactly after both 2 and 3 are processed.
- After Node 4 is processed, its prerequisites are cleared for **Node 5** and **Node 6**. Since 5 and 6 have no dependencies on each other, they can be processed in any order. (Creates suffix branches: ...4, 5, 6 or ...4, 6, 5).

Combining these independent branches ($1 \times 2 \times 1 \times 2$), we get exactly **4 possible topological orderings**:

1. 1, 2, 3, 4, 5, 6
2. 1, 2, 3, 4, 6, 5
3. 1, 3, 2, 4, 5, 6
4. 1, 3, 2, 4, 6, 5

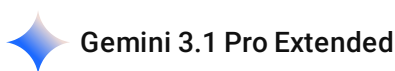
You Asked



Q6 State and describe the two properties of Greedy approach of problem solving 5

Q1 For each of the following sorting algorithms, merge sort and insertion sort, discuss whether or not it is 3
a) (i) stable. (ii) in-place

b) State the cut property of graph. 2



Here are the solutions to the questions from the images provided:

Q6: Properties of the Greedy Approach

For a problem to be successfully solved using a Greedy algorithm, it must typically exhibit two fundamental properties:

1. Greedy Choice Property: This property states that a globally optimal solution can be reached by making a locally optimal choice. In other words, at each step of the algorithm, you pick the option that looks best right now, without worrying about future consequences or reconsidering past choices. If making these local "greedy" choices consistently leads to the overall best outcome, the problem possesses the greedy choice property.

2. Optimal Substructure: A problem has optimal substructure if an optimal solution to the entire problem contains the optimal solutions to its sub-problems. This means you can break the larger problem down into smaller, identical pieces, solve those pieces optimally, and combine them to form the optimal solution for the whole.

Q1 a) Merge Sort and Insertion Sort Properties

Merge Sort:

- **(i) Stable: Yes.** Merge sort is a stable sorting algorithm. During the merge phase, when comparing two equal elements, the algorithm strictly prefers the element from the left sub-array over the right sub-array. This ensures that identical elements retain their original relative order.
- **(ii) In-place: No.** Standard merge sort is not an in-place algorithm. It requires $O(n)$ auxiliary memory space to create temporary arrays where the elements are placed during the merging process.

Insertion Sort:

- **(i) Stable: Yes.** Insertion sort is stable. As it inserts an element into the sorted portion of the array, it only shifts elements that are *strictly greater* than the current key. Equal elements are never swapped past each other, preserving their original sequence.
 - **(ii) In-place: Yes.** Insertion sort operates in-place. It only requires a constant amount of extra memory space $O(1)$ to hold the current key being inserted, rearranging the elements within the original array boundaries.
-

Q1 b) Cut Property of a Graph

Definition: The cut property is a fundamental concept used in finding Minimum Spanning Trees (MSTs). A "cut" is any partition of a graph's vertices into two mutually exclusive, disjoint sets.

The cut property states that for any given cut, the crossing edge (an edge that connects a vertex in one set to a vertex in the other) with the **minimum weight** is guaranteed to be a part of the Minimum Spanning Tree of that graph.

You Asked 

Q2 Demonstrate the working of Insertion sort, Quick-Sort algorithms on the array { 3, 7, 9, 12, 15}. write the status of the array during every iteration of each pass. Also, find the cumulative number of comparisons at each iteration. 5

Q4 Illustrate the Counting sort algorithm on the following array:
A= {2, 4, 1, 2, 9, 1, 5, 8, 2, 3, 8, 9, 5, 2, 1} 5

 Gemini 3.1 Pro Extended

Here are the step-by-step solutions to the algorithms presented in your questions.

Q2: Insertion Sort and Quick-Sort on {3, 7, 9, 12, 15}

Observation: The given array is already sorted. This represents the **best-case scenario** for Insertion Sort $O(n)$ and, depending on the pivot selection, the **worst-case scenario** for standard Quick-Sort $O(n^2)$.

1. Insertion Sort

Insertion sort iterates through the array, comparing the current element (key) to its predecessors and inserting it into the correct position. Because the array is already sorted, the key will only ever be compared to the element immediately before it, finding it is already in the right place.

Pass (Iter.)	Key	Array Status at End of Pass	Comparisons (This Iter.)	Cumulative Comparisons
Initial	-	[3, 7, 9, 12, 15]	0	0
1 ($i = 1$)	7	[3, 7, 9, 12, 15]	1 (7 vs 3)	1
2 ($i = 2$)	9	[3, 7, 9, 12, 15]	1 (9 vs 7)	2
3 ($i = 3$)	12	[3, 7, 9, 12, 15]	1 (12 vs 9)	3
4 ($i = 4$)	15	[3, 7, 9, 12, 15]	1 (15 vs 12)	4

2. Quick-Sort

For this trace, we will use the standard Lomuto partition scheme, where the **last element** of the current sub-array is selected as the pivot. Because the array is already sorted, the pivot will always be the largest element, resulting in highly unbalanced partitions (one of size $n - 1$, the other of size 0).

Note: Partitioning a sub-array of length k requires $k - 1$ comparisons against the pivot.

Call Level / Partition	Pivot	Array Status after Partition	Comparisons (This Iter.)	Cumulative Comparisons
Initial	-	[3, 7, 9, 12, 15]	0	0
1: [3, 7, 9, 12, 15]	15	[3, 7, 9, 12, 15]	4 (vs 3,7,9,12)	4
2: [3, 7, 9, 12]	12	[3, 7, 9, 12, 15]	3 (vs 3,7,9)	7
3: [3, 7, 9]	9	[3, 7, 9, 12, 15]	2 (vs 3,7)	9
4: [3, 7]	7	[3, 7, 9, 12, 15]	1 (vs 3)	10
5: [3]	-	[3, 7, 9, 12, 15]	0 (Base Case)	10

Q4: Illustrate the Counting Sort Algorithm

Given Array: $A = \{2, 4, 1, 2, 9, 1, 5, 8, 2, 3, 8, 9, 5, 2, 1\}$

Counting sort is a non-comparison-based sorting algorithm. Here are the steps to illustrate its working:

Step 1: Find the maximum element Scanning array A , the maximum value is **9**. This tells us we need a counting array C of size 10 (indices 0 through 9).

Step 2: Frequency Count Array (C) We initialize an array C with zeros and tally the occurrences of each number from array A .

- 1 appears 3 times

- 2 appears 4 times
- 3 appears 1 time
- 4 appears 1 time
- 5 appears 2 times
- 8 appears 2 times
- 9 appears 2 times
- 0, 6, and 7 appear 0 times

Array C (Frequencies):

Index	0	1	2	3	4	5	6	7	8	9
Count	0	3	4	1	1	2	0	0	2	2

Step 3: Cumulative Count Array (C') We modify the frequency array by adding the previous counts to the current count. This tells us the actual positions of the elements in the final sorted array.

- $C'[0] = 0$
- $C'[1] = 0 + 3 = 3$
- $C'[2] = 3 + 4 = 7$
- $C'[3] = 7 + 1 = 8$
- ...and so on.

Array C' (Cumulative Frequencies):

Index	0	1	2	3	4	5	6	7	8	9
Sum	0	3	7	8	9	11	11	11	13	15

(Interpretation: The cumulative count of 7 for index 2 means the last occurrence of the number '2' will be placed at position 7 in a 1-based index).

Step 4: Build the Output Array (B) We iterate through the original array A , look up its mapped position in the cumulative array C' , place the element in output array B , and then decrement the count in C' .

Following this process yields the final sorted array: $B = \{1, 1, 1, 2, 2, 2, 2, 3, 4, 5, 5, 8, 8, 9, 9\}$

You Asked 

Q5 Show the result of running BFS on the following graph (Figure-1) given below using the vertex D as source. Show the tree after each step

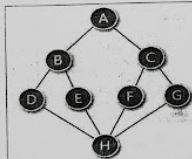


Figure 1: Graph for BFS Traversal

Q6 For the graph given in Figure 2, find a minimum spanning tree using the Prim's algorithm considering node 1 as the start node. Also, state the time complexity of the same.

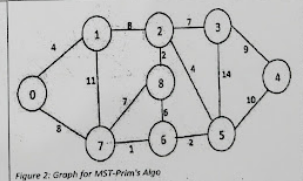


Figure 2: Graph for MST-Prim's Algo

Here are the step-by-step solutions to the graph traversal and minimum spanning tree problems presented in the image.

Q5: Breadth-First Search (BFS) Traversal

Objective: Perform a BFS on Figure 1 starting from vertex **D** and show the resulting tree. *Note: When multiple neighbors are available, we will visit them in alphabetical order, which is the standard convention.*

Initialization:

- **Queue:** [D]
- **Visited Set:** {D}
- **Tree Edges:** (Empty)

Step 1:

- **Action:** Dequeue **D**. Check its unvisited neighbors: **B** and **H**.
- **Update:** Enqueue B and H. Mark them as visited.
- **Queue:** [B, H]
- **Visited:** {D, B, H}
- **Tree Edges Added:** D - B , D - H

Step 2:

- **Action:** Dequeue **B**. Check its unvisited neighbors: **A** and **E**. (D is already visited).
- **Update:** Enqueue A and E. Mark them as visited.
- **Queue:** [H, A, E]
- **Visited:** {D, B, H, A, E}
- **Tree Edges Added:** B - A , B - E

Step 3:

- **Action:** Dequeue **H**. Check its unvisited neighbors: **F** and **G**. (D and E are already visited).
- **Update:** Enqueue F and G. Mark them as visited.
- **Queue:** [A, E, F, G]
- **Visited:** {D, B, H, A, E, F, G}
- **Tree Edges Added:** H - F , H - G

Step 4:

- **Action:** Dequeue **A**. Check its unvisited neighbor: **C**. (B is already visited).
- **Update:** Enqueue C. Mark it as visited.
- **Queue:** [E, F, G, C]
- **Visited:** {D, B, H, A, E, F, G, C} (All vertices are now visited)
- **Tree Edge Added:** A - C

Subsequent Steps (5-8): Dequeuing the remaining nodes (**E, F, G, C**) yields no new unvisited neighbors. The algorithm terminates.

Final BFS Tree Structure:

- **Root:** D
- **Level 1 Children:** B, H (connected to D)
- **Level 2 Children:** A, E (connected to B) and F, G (connected to H)
- **Level 3 Child:** C (connected to A)

Q6: Minimum Spanning Tree (Prim's Algorithm)

Objective: Find the MST for Figure 2 starting from node 1, and state the time complexity.

Step-by-Step Execution: We maintain a set of visited vertices (V) and always greedily pick the edge with the lowest weight that connects a visited vertex to an unvisited vertex.

- **Start:** $V = \{1\}$
1. **Edges available:** (1,0) weight 4; (1,2) weight 8; (1,7) weight 11.
 - **Pick min:** Edge (1, 0), Weight = 4.
 - **V becomes:** {1, 0}
 2. **Edges available:** (1,2) weight 8; (1,7) weight 11; (0,7) weight 8.
 - **Pick min:** There is a tie between (1,2) and (0,7). We can pick either; let's pick (1, 2), Weight = 8.
 - **V becomes:** {1, 0, 2}
 3. **Edges available:** (0,7) weight 8; (1,7) weight 11; (2,8) weight 2; (2,5) weight 4; (2,3) weight 7.
 - **Pick min:** Edge (2, 8), Weight = 2.
 - **V becomes:** {1, 0, 2, 8}
 4. **Edges available from V :** (0,7): 8; (2,5): 4; (2,3): 7; (8,7): 7; (8,6): 6.
 - **Pick min:** Edge (2, 5), Weight = 4.
 - **V becomes:** {1, 0, 2, 8, 5}
 5. **Edges available from V :** (0,7): 8; (2,3): 7; (8,7): 7; (8,6): 6; (5,6): 2; (5,4): 10; (5,3): 14.
 - **Pick min:** Edge (5, 6), Weight = 2.
 - **V becomes:** {1, 0, 2, 8, 5, 6}
 6. **Edges available from V :** (0,7): 8; (2,3): 7; (8,7): 7; (6,7): 1; (5,4): 10; (5,3): 14. (Note: (8,6) is ignored as it forms a cycle).
 - **Pick min:** Edge (6, 7), Weight = 1.
 - **V becomes:** {1, 0, 2, 8, 5, 6, 7}
 7. **Edges available from V :** (2,3): 7; (5,4): 10; (5,3): 14. (Note: (0,7) and (8,7) are ignored as they form cycles).
 - **Pick min:** Edge (2, 3), Weight = 7.
 - **V becomes:** {1, 0, 2, 8, 5, 6, 7, 3}
 8. **Edges available from V :** (3,4): 9; (5,4): 10.
 - **Pick min:** Edge (3, 4), Weight = 9.
 - **V becomes:** {0, 1, 2, 3, 4, 5, 6, 7, 8} (All nodes visited)

Resulting MST Edges: $\{(1,0), (1,2), (2,8), (2,5), (5,6), (6,7), (2,3), (3,4)\}$ *Total Minimum Cost:* $4 + 8 + 2 + 4 + 2 + 1 + 7 + 9 = 37$

Time Complexity: The time complexity of Prim's Algorithm depends heavily on the data structures used to implement it:

- $O(E \log V)$: This is the standard time complexity if the algorithm is implemented using an **Adjacency List** and a **Min-Priority Queue** (Binary Heap), where E is the number of edges and V is the number of vertices.
- $O(V^2)$: This is the time complexity if the graph is represented using an **Adjacency Matrix** and an unsorted array is used for distance lookups (often used for dense graphs).